PLAUSIBLE INFERENCE: A KNOWLEDGE-INTENSIVE APPROACH TO INDUCTION (AN APPLICATION OF PI TO THE PROBLEM OF HIDDEN HOMOLOGY MODELING)

BY

DANIEL A. OBLINGER

B.S., Northern Kentucky University, 1987
M.S., Ohio State University, 1989

THESIS

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Computer Science
in the Graduate College of the
University of Illinois at Urbana-Champaign, 1998

Urbana, Illinois

ABSTRACT

We adopt the perspective of learning as a user-controlled tool that assists in complex domain modeling. Using this perspective we develop Plausible Inference (PI)—a framework based on the tradeoffs suggested by this learning-as-a-tool perspective. Four aspects of the approach distinguish it from other learning approaches, and are responsible for its improved learning performance: (1) PI uses problem-decomposition information. (2) It accepts local bias information on a domain-term by domain-term basis. (3) It allows the user to fashion a task-specific learning system by composing algorithms from a library of learning algorithms, and (4) it allows the user to make efficient use of the fixed learning resource by allowing the user to interleave directly encoded and induced portions of the domain model being learned. Increased performance from these aspects of PI is measured in empirical comparisons with two well known learning algorithms: FOIL and FOCL. We use PI to draw a parallel between induction and inference, and use this parallel to illustrate how each discipline can benefit from the important ideas in the other field. Assumptions made by our tool approach are tested by encoding and using knowledge from a complex modeling task. The last third of the dissertation is devoted to the prediction of Protein Structure. Available expertise on the protein-folding process is encoded for use by PI and is tested and refined using our framework.

For Mom and Dad

ACKNOWLEDGMENTS

A large number of people have contributed to the technical content of this thesis. The greatest single source has been my advisor, Jerry DeJong. His ability to take a fresh perspective on current work in the field has lead us in many fruitful directions. Nina Mishra and Mark Brodie both provided significant help through discussions of the work and specifically in organizing the prelim and final defense presentations. They encouraged me to pair down the many ideas developed in the course of my work into a concise package, thereby strengthening the thesis.

As I am a relative novice in the field of bioinformatics, my contribution to that field was only feasible through the generous assistance of a number of people. Tom Ioerger first introduced me to the area. In fact it was well beyond midnight on a marathon session at Mary Ann's diner that we first saw in a concrete way how PI could (and eventually would) be applied to protein structure prediction. Shankar Subramaniam provided a speedy and friendly introduction to this area; he and the members of his research group made many essential resources available to me. Computer accounts, access to many hours of high performance computer time, electronic data, hours of time spent explaining important concepts in the area, and references to just the right article at the right time were all made available easily. Tom Ioerger provided two of the data sets used in my experiments, and source code for the linear space alignment, and George Pappas provided the MinRMS procedures used to analyze protein similarity.

I am also indebted to several people who are in neither of my research groups. Although not even in AI or Bioinformatics, Josh Yelon has suffered more hours of discussion about Plausible Inference than any other. He has always been available on demand for discussion, dinner, or late night hacking. Marsha Brofka contributed to my social life, especially during the last phases of those graduate studies. Her pleas of "come on, come on, come on, come on" resulted in salsa classes, many parties, movies, and lots of good though sometimes bizarre vegetarian food. My Mentor Dr. Klembara at Northern Kentucky University also deserves a note of recognition. Although he still probably doesn't recognize Computer Science as a Science (Mathematics being the only true discipline), he did instill in me a sense of what research is, and what we as a research community are trying to accomplish. His influence has shaped my graduate career and still provides a counter-point to the purely utilitarian view of my career.

I owe a great debt of thanks to my parents. Without their thousands of hours of tutoring, its not clear that this dyslexic boy would have ever graduated from high school, much less from graduate school. They encouraged both my brother and myself to follow our natural inclinations without undo regard for time or money. While this provided years of worry, it has also provided us with a potential lifetime of satisfaction. I also want thank my parents for years of coming up with creative answers to the question "so what exactly is he doing there in Illinois anyway?" More than that, for gracefully accepting the time and distance constraints that this dissertation and career choice place on me. It is to them that I dedicate this thesis.

TABLE OF CONTENTS

LIST OF FIGURES

Figure **Page**

CHAPTER 1

INTRODUCTION

In this dissertation we adopt the perspective of machine learning as a *tool* under the explicit control of the expert. By comparison many adopt the view of learning as back-box process where training-data is fed in as input, and domain models are generated by the learning system as output. According to our perspective, learning is a tool in the same way that a flashlight is a tool. A flashlight illuminates an area pointed at, in the same way our learning system should search those hypotheses "pointed at" by the expert. Notice the shift in responsibility here: we do not expect a flashlight to try to illuminate the "right" area, we expect it to illuminate the area that's designated by its user. In the same way, we want our learning tool to search the area designated. As with the flashlight, the responsibility for correct designation rests with the user. A black-box learning system, on the other hand, *is* responsible for finding the "right" output given its input data. So, given the option, why would a user ever choose a tool that must be carefully pointed over a black-box that requires little guidance? By analogy, imagine a flashlight with a bulb that sheds light equally in all directions. It does not need to be correctly pointed, but in many cases this would not be desirable since the amount of light directed at the area of interest would be much weaker. The same tradeoff exists with automated learning systems: shifting the responsibility of "pointing" to the user

can increase search in areas of interest. Such a learning tool needs a precise and flexible control language in order to allow the user to point the learning system. We will see that this reduction in the system's responsibility and increase in user control can greatly extend its applicability—we will see that a Machine Learning tool can be applied in domain far too complex for a black-box type learning system. *Thus the goal of this dissertation is to develop a learning tool, that can search a focused or diffuse area as designated by the expert through a precise and flexible control mechanism.*

Because of this shift in perspective, we must reevaluate many design decisions typically made in Machine Learning. Completeness for example, is desirable for a black-box learner, but *in*completeness is required in cases where an expert is to tightly control what is searched. Precise control of a tool is necessarily more complex than control of a black-box, thus a good deal more attention needs to be drawn to the software engineering aspects of the language (bias) used to direct the learning. Indeed we will argue that no single universal language exists that is ideal for precise control across all domains.

1.1 Contributions

The contribution of this dissertation to the field of Machine Learning is *Plausible Inference*—a framework for user-directed inductive learning. There are four objectives which we will argue are important if a learning system is to be used by a domain expert as a modeling tool. We summarize the tradeoffs necessary for expert-directed inductive learning as a set of four primary objectives for Plausible Inference (PI). Each of these objectives are formalized and tested in a number of way throughout the dissertation. We briefly describe each objective here so that they may serve as touchstones for understanding the work itself.

1. Learning bias is often specified as parameters that affect the global behavior of the learning system. A flexible learning tool should allow the user to exercise local control over the learning system. Further, that control should be expressed using the vocabulary of the target domain (which is most familiar to the expert that must supply this learning bias). We refer to this as a *term-level bias*. A learning tool should employ a term-level bias.

2. We show that dramatic performance improvement is possible by providing the learning system with problem-decomposition information—a learning bias that describes how the learning task is best decomposed into smaller sub-tasks. We refer to this as *problem structure*. We show examples of complex, practical tasks where such problem structure is available; thus an important goal for any learning tool is to capitalize on this decomposition information when it is available.

3. The field of inductive learning has long acknowledged the inherent tradeoff between the generality of a particular bias and the learning performance that results from that bias. A learning tool should have general applicability, but at the same time should allow the user to encode a very specialized (high-performance) learning bias. One way to accomplish both of these objective is to provide a diverse set of different learning algorithms, and let the user combine these into a specialized structure for each learning task. We refer to this as a *building-block* approach. We will show that specifying a structure of learning systems is easier than building a problem-specific learning algorithm, yet it still results in very problem specific learning performance.

4. We formalize the notion of the "amount" of automated learning as a numeric quantity, and show that any given training set will only support a limited "amount" of learning according to this metric. "Black box" learning systems use a fixed policy for allocating this learning resource. Allowing the expert to control the application

of this fixed resource enables a system to apply it to portions of the task where automated refinement is most needed. A learning tool should permit the expert to apply this fixed learning resource, to portions of the model where learning is most needed.

We have provided a high-level description of these four objectives: term-level bias, use of problem structure, building-block approach, and flexible allocation of learning resource since they are the defining aspects of our approach. In our formal characterization of PI we will describe how it achieves each of these objectives, and in our empirical evaluation, we show how this tighter, more flexible control by the user translate the additional information provided by the user into dramatically improved learning performance.

1.2 Control Mechanisms for Learning Systems

We use a simple intuitive learning task to illustrate the defining aspects of Plausible Inference (PI), our framework for controlled learning. The objective in this learning task is to predict whether or not an automobile will start, given a number of observations about that automobile. In this task we assume a basic understanding of the car as our "expertise" for the domain. The sentences in Figure 1.1 convey a very basic understanding of what factors affect whether a car will start. Notice, the third rule will not apply to most car, since they will be able to start regardless of how hard it has rained. Nonetheless, that rule is important, since it enables us to model older cars as well—cars that may have cracks in their wiring that keep them from operating in the rain. The task of the learning system, for any given car, is to determine what combination of these attributes predicts its starting behavior. This task fits the conventional characterization of inductive learning nicely. Assume that a set of cars are labeled as starting or not-starting, and that each car has three associated real-value attributes: gas-level, battery-level, and

"A car will not start if it has no gas."
"A car will not start if its battery is drained." And,
"A car will not start if it has rained hard."

Figure 1.1 An "expert's" theory on car starting

inches-of-rain. This is a textbook example of a learning task that is ideal for any number of conventional inductive learning algorithms. C4.5, for example, is a widely used inductive learning system developed by Ross Quinlan[1]. It is designed to take real-valued attributes like those shown here, and construct a decision tree that "splits" cars into starting and non-starting cars. It learns rules like "a car starts if it rained less than one inch or if the battery level is greater than 80%."

At first glance, it seems that by looking for splits over the three relevant attributes C4.5 has captured much of a layman's understanding what affects car starting. Upon closer inspection, however, even for this trivial task there are a number of important things that we, as car experts, would like to "tell" our learning tool in order to direct its search. We do not know how the input terms combine to predict in this task, so it seems to follow that we cannot direct the learning toward an appropriate decomposition of the problem. But this is not true: in cars with old, cracked wiring it is plausible that having additional energy in the car battery will compensate for weakened current going to spark plugs because of the rain. Thus, combinations of these terms makes sense. Once the car has enough gas to start, however, no amount of additional gas will compensate for deficiencies in the other parameter values—it makes no sense to consider combinations between gas-level and either of the other parameters. We refer to this type of knowledge as *[partial] model decomposition* information. A model decomposition specifies functional dependency networks in the same way that a baysian network does, but we to not assume (or treat) the values in this network as probabilities[2, 3]. Figure 1.2 graphically depicts

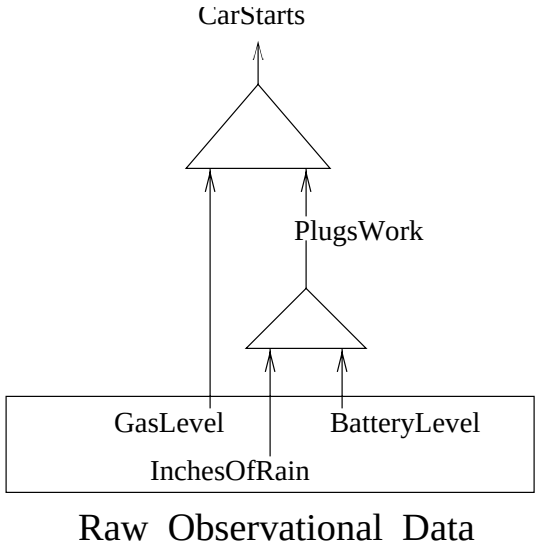

Figure 1.2 A partial decomposition for the car-starting task

how we might express this partial decomposition. Each cone represents a smaller sub-learning task for plausible inference. C4.5, like many other learning systems, has no vocabulary for specifying partial decomposition information like this. This information is critical, however. As seen here, it is often available even when a full model of the domain is not available. In addition to being readily available in many domains, this information is important because it has the potential to split a large learning task into a set of smaller learning tasks. This splitting can make a large, practical impact on learning performance, since there are many cases where both training data needed and learning time are exponential in the size of the model being learned.

Though the structure shown in Figure 1.2 captures more than can be provided to C4.5, it still ignores much about these attributes that might help focus a learning system. It provides functional dependence information, but says nothing about what functions are possible at each point within that decomposition. Yet there is much that we know about these relationships. For example, if we observe a car that starts given some gas-level x, then, all things being equal, we expect that that same car will also start given some

gas-level y, where $y \geq x$. Stated differently, car starting depends on some unknown threshold on the gas-level parameter. The set of threshold functions is a very small subset of the set of all possible real valued functions. Simply asserting that the function sought is a threshold function can greatly reduce the difficulty of the subsequent learning task. We refer to this additional information on the functional relationships as *annotation* information. Unfortunately it seems there is no single language that captures the diversity of annotation information that an expert might want to use to control their learning systems. In order to accommodate such information in a flexible manner, Plausible Inference employs an extensible language for specifying this annotation information. Of course extensions to the annotation language must also specify how the annotation is to be used in learning, and not every kind of annotation can be incorporated into the framework. Nonetheless, we believe that this approach results in one of the most rich bias specification languages considered within Machine Learning.

We have seen methods the expert can use to control how learning is applied. Just as important for a tool, however, is the ability to specify when it should *not* be applied. This is accomplished within our framework by providing definitions for terms in part of the model, and declaring that the learning system should not consider alternatives for that portion of the model. PI attempt to take advantage of all expertise available for each part of the theory. If some portions are completely understood it make sense that PI should be capable of simply incorporating that information directly into the model it is learning.

Interestingly, a "black-box" learning system, like C4.5, is capable of inducing a model for the car-starts task without using any of the control information described above, even though it is spending its time considering hypotheses that would be ruled out by our more comprehensive expertise. In fact, the additional cost incurred by C4.5 on this trivial example is only a modest increase in the training data required for learning. We will show

domains only slightly more complex than the one shown here, however, where systems like C4.5 quickly become overwhelmed by the space of hypotheses they must consider. Our approach, on the other hand, provides several mechanisms that allow the expert to focus the learning using available expertise. Further, we will show that dramatic learning performance improvements are possible through the use of this additional expertise. PI allows the expert greater flexibility in gradually assuming greater responsibility over the model being generated as the complexity of the model outstrips the computational and training-data resources available. This avoids the need to completely discard learning as the domain increases in complexity. Both of these advantages are geared toward larger, more complex domain modeling tasks, which are exactly the tasks that motivated our shift in perspective. Because a learning tool shifts responsibility to its user it can succeed in cases where a black-box learning system cannot.

1.3 Modeling Protein Structure Using Plausible Inference

As with many systems, the ultimate success or failure rests on how well the assumptions made by the system match the world in which it they used. Understanding a system's range of possible behaviors can often be done formally, but the best way to understand how the underlying assumptions hold up in practice is to apply it to some real-world modeling task. What properties such a test domain should possess? Our perspective on learning was adopted in order to push the envelope of domains where Machine Learning can be applied. The tradeoff between user responsibility and domain complexity made by our approach is only appropriate for domains that are too complex for more autonomous forms of learning. In addition to high complexity, the domain must also be

well-studied by a body of experts. PI can reach further than other learners only when it is provided with useful problem structures, and such structures can only be generated by those with considerable experience with the complex domain—PI is best applied to problems currently on the frontier of human understanding. We have identified such a problem within the field of computational biology.

The task of predicting the general 3D-shape of a folded protein given only its primary sequence (its chemical makeup) serves as an ideal application domain for a number of reasons. The task of predicting a protein's structure is far too complex for any learning system that does not have some very strong bias (structural) knowledge provided. Because structural prediction is an area of very active study there are a number of theories about different aspects of protein folding. These partial theories provide the ideal hypothesized structural knowledge for PI's learning.

The intense activity in determining protein structures belies the importance of the area. Both the biological and medical sciences are turning with increasing frequency to the molecular level for explanations of important phenomenon. Experimental science has yielded very cheap methods for determining primary sequence information. Structural information, by comparison, is very difficult or even impossible to obtain. It is the protein's structure, however, that provides clues about its function. Thus predicting the structure of a protein given only it sequence has become something of a holy grail within the field of computational biology.

The fields of chemistry, physics, and quantum mechanics, have a great deal to say about the forces involved in shaping a protein. Each of these areas introduces a great deal of structure (as an interrelated set of intermediate terms), but none of these theories can tractably make any predictions about a protein's overall shape. Nonetheless each discipline has considerable evidence that the structures (and intermediate terms) introduced by that discipline are indeed relevant in structure prediction. A predictive model

of protein structure will undoubtedly contain portions of each of these partial theories, but the exact combination is not known. This is the type of partial knowledge PI is designed for. We apply PI to this task by encoding information from these areas. Some portions of these theories are understood in complete detail; these are simply encoded as functions that should not be learned. Often, however, these well-understood portions of the domain still have parameters or other unknown portions that must be instantiated before they can be applied in any a particular situation. It is over these enumerated "gaps" that PI does its learning. PI also learns by joining existing theories; often there are cases where several partial theories lend evidence for the value of some intermediate term. Since the theories themselves are separate, none of them discusses how they should be combined. PI learns what combination of these theories is most predictive.

Note that these parameter-setting, gap-filling, combining roles allow PI to avoid the really nasty tasks like inducing a model of the three-dimensional steric constraints that result from the thousands of bonds within a protein. Such computation are simply provided by experts as procedural attachments and are made available to PI. Note also that while PI has avoided many "tough" parts of the problem, it is still serving a crucial role: Any advancement in "merely filling the gaps" to build a predictive model is quite important—it is exactly what a great many researchers are currently working toward.

Our work on protein structures has a two-fold benefit. The direct benefit is a better understanding of how various partial theories apply to the structure prediction task. For example, are substitution patterns most affected by the steric or electrical properties of the amino acids? The second benefit of our work on structure prediction is exploration of how a large variety of partially understood theories can (and cannot) be encoded as a plausible knowledge.

1.4 Dissertation Outline

Our exploration of the framework for Plausible Inference, and the task of Protein Structure Prediction is divided into seven chapters. Chapter 2 provides a formal characterization of the PI framework, including with a detailed example of its application to a simple learning task. Chapter Three uses experimental comparisons between PI and two other learning algorithms in order to understand how PI's comparative performance will vary over the space of possible learning tasks. Chapter Four compares Plausible Inference with other approaches developed within Artificial Intelligence. PI is compared with learning systems that do not employ any structured bias knowledge (like Figure 1.2), as well as several classes of learning systems that do utilize different forms of structured bias. We also use Plausible Inference as a bridge between inductive learning and defeasible inference. Using the parallel drawn between these two areas, we consider how the central ideas from each could be applied to the other. Chapter Five introduces Protein Science and the specific task of Hidden Homology Modeling (a special case of protein structure prediction). Prior work on Hidden Homology Modeling (HHM) is briefly described. How well the knowledge utilized by these prior approaches fits into the PI framework is also considered. Chapter Six describes our application of PI to HHM. It provides detail about the algorithms used, data available, and one of the experiments performed. Chapter Seven looks back at our comparisons, applications, and analyses of PI to discuss the overall strengths and weaknesses of the approach. This discussion is used to characterize those tasks for which our approach is best suited.

CHAPTER 2

A FRAMEWORK FOR PLAUSIBLE INFERENCE

2.1 Introduction

We have demonstrated the need for and benefit of using a structured, annotated background knowledge as bias. In this chapter we present plausible inference, a model of induction that drives specialized, high-performance induction using a combination of specialized learning algorithms and structured, annotated background knowledge as bias. We (1) introduce the important concepts within the framework, (2) provide formal specification of the framework, and (3) demonstrate the use of the framework through its application to a simple learning task. Throughout the chapter we use the car-starting example from the introduction to make various components of the framework concrete.

As described above, PI relies on the structure that exists between terms within a domain to simplify learning in that domain. This structure is encoded using annotated horn clauses called *influents*. These influents describe the functional dependencies that exist within the domain. In the car-starting example we are told that a car starts when it has gas. This could be encoded as: "`GasLevel` *influences* `CarStarts`". This declared relationship is very similar to the determination described by Ben Grosof and Stuart Russell[4, 5]. Their determination expresses a functional relationship between its an-

tecedents and its consequent; e.g. a person's country of birth *determines* their native language. One important distinction between these two types of bias is the way their antecedents are used. The antecedents of the determination completely determine the consequent of the determination. The influent's antecedent, on the other hand, is not required to completely determine its consequent. In our example we see that gas level *influences* but does not completely *determine* whether a car will start. Thus it would be incorrect to use a determination in place of an influent in this case. In Chapter 4 we make a comparison of these two forms of bias.

The influent, in addition to constraining what terms may be considered when during induction, also constrains what types of combinations of those terms are considered. Both constraints are also used within the framework to simplify the learning task. We use the car-starting example above to demonstrate both types of constraint. We use terms like `PlugsWork` and `HasGas`; observations, on the other hand, are described using real-valued terms like `InchesOfRain` and `GasLevel`. These relationships can succinctly be represented by noting that "`InchesOfRain` *influences* `HardRain`," and "`GasLevel` *influences* `HasGas`," where `InchesOfRain` and `GasLevel` are real-valued observations, and `HardRain` and `HasGas` are inferred Boolean attributes. While these influents capture the dependency structure that exists between their terms, they do not capture all that is known about that relationship. In particular, we expect that `HasGas` is some threshold function on `GasLevel`; i.e., if `HasGas` = TRUE for `GasLevel` = 10%, then `HasGas` = TRUE must also hold for `GasLevel` = 20%. We capture this additional information by annotating the influents by the *type* of relation they describe. The assertion `HasGas` is a threshold between 0 and 100 on the `GasLevel` term is written "`HasGas` $\prec_{\texttt{th}}$ $[0-100]$ `GasLevel`." Later we will discuss influent syntax; for now we just consider it meaning. This influent asserts that there exists some N between 0 and 100 such that such that `HasGas` iff `GasLevel` $\geq N$. Such an assertion constrains PI's learning since the space of threshold

functions is a small subset of the possible relations that might exist between these two terms. We will later see how annotations like these which restrict the specified functional relationship have can greatly simplify the learning task. After all there space of thresholds is a tiny fraction of the larger space of all real-valued functions.

Each influent's consequent (like `HasGas`) represents some intermediate concept within the domain. By assumption, the domain described by these influents is only partially understood, thus many of these intermediate terms have no computational model—they represent opportunities for refinement (learning). The influent above declares `HasGas` is a real-valued threshold on `GasLevel`, but does not specify *which* real-valued threshold. Instead it specifies a family (all thresholds with cutoff between 0 and 100). This family of alternatives is called a *refinement space*—each refinement space represents an ambiguity in the domain expertise and a potential mini-induction problem for PI. The hypotheses in this mini-induction problem are those combinations of influent antecedents consistent with influents' type.

Different influents may suggest completely different refinement spaces for their constituent terms. Since `InchesOfRain` and `BatteryLevel` are real-valued terms and both relate to `PlugsWork`, perhaps some combination of the two terms is the best predictor of the intermediate term `PlugsWork`. This would suggest a completely different refinement space for this term, and would require a completely different learning algorithm to select optimally from this space. Thus, each influent is associated with an induction algorithm and a set of dependant terms specific to that position within the domain theory.

Having a hand-coded induction algorithm for each node within some model of a domain would surely result in stellar learning performance over that domain. Such an interconnected network of specialized learning algorithms would also undoubtedly be prohibitively labor intensive to construct. One doubts whether the benefits of learning with such a system could ever justify the cost of constructing all of those specialized learning

algorithms. Fortunately, the refinement spaces in the domains we have considered naturally fall into sets of similar refinement space "types." For example, in domains with both numeric and Boolean attributes, threshold-type refinement spaces abound. These similar spaces can be characterized as instances of some generic threshold refinement space with its generic threshold learning algorithm.

The PI framework assumes a library of refinement space types and associate learning algorithms. The domain expert then uses these refinement space types as building blocks for constructing a problem-specific high performance learning algorithm. The specificity of the resulting algorithm results from structure provided by the expert relating the constituent refinement space types; it also results from the refinement space terms provided by the expert. We have found that a small library of refinement space types is sufficient for a wide range of learning tasks. Later in this chapter we present eight general influent types.

These eight are interesting because they are ones used repeatedly in our application of PI to protein structure prediction. While they were developed specifically for this particular domain, we will see that they are quite domain *independent*. This reuse of refinement space types is a crucial aspect of our approach; it is what allows us to supply a high performance, problem-specific learning algorithm without requiring the domain expert to code problem-specific learning algorithms. We explore the reuse of refinement space types in detail in the analysis chapter, Chapter 7.

PI performs its learning by iterating over this structured set of refinement spaces. Induction is performed on each refinement space, and an updated definition for that spaces term is obtained. This iteration is repeated until PI converges on a predictive model of the target domain term or no further improvement is possible within any of the refinement spaces. In the car-start example, PI would use its training examples to select cutoffs for `HardRain`, `HasGas`, and `BatteryLevel`. The most predictive combina-

tion of these intermediate terms would then be selected as the definition of `CarStarts`. This description of the algorithm is simplistic; in particular, we have not dealt with the interactions that inherently exist between these sub-induction tasks. After a formal and complete description of the approach is provided in the next section, we will return to the car-start problem to see how these interactions are handled by the approach.

2.2 Formal Specification of the PI Framework

As described above, the PI framework expresses a learning task as a set of smaller refinement problems. The framework places very little additional constraint on the representation of bias and training data used by these subordinate refinement space induction algorithms. Given a plausible theory (a set of influents), PI must determine (1) the relationship between refinement spaces, and (2) the appropriate induction algorithms and terms for those spaces. Thus only the general structure of the knowledge and learning algorithms are fixed by the framework itself. The specifics of PI's behavior is captured in several functions that are instantiated as appropriate by each particular refinement space.

As we have seen, domain expertise is provided to PI as a set of horn-like rules, called influents. **I** denotes the set of well-formed influents whose syntax is shown below:

$$\texttt{<RefSpaceName>} \quad \prec_{\texttt{type}} \quad \texttt{<Parameters>} \ \texttt{<SubRefSpaces>}$$

The influent declaring `HasGas` as a threshold between 0 and 100 over `GasLevel` would be written as:

$$\texttt{HasGas} \ \prec_{\texttt{th}} [0 - 100] \, \texttt{GasLevel}$$

Each type of refinement space may require parameters that are specific to that type of refinement space. For example, the notion of a maximum and minimum in the influent

above is appropriate for space of real-valued thresholds, but is meaningless for a space of Boolean combinations.

Because of this we do not fix the influent's syntax in our framework; instead, we define several functions over the space of influents that control PI's behavior given a set of influents. If a new refinement space is created, these functions must be instantiated for the new type of influent. Each influent, `I`, may have a set of antecedents, a consequent, a type, and mapping. The consequent of an influent `I`, CONSEQUENT(`I`), is the name of the refinement space described by the influent. So CONSEQUENT("`CarStarts` $\prec_{\texttt{bool}}$ `HasGas`") = `CarStarts`. Each influent also has a set of antecedents, ANTECEDENTS(`I`). These are the names of the sub-refinement spaces upon which this refinement space depends (In the example above this would be {`HasGas`}. The type of an influent, TYPE(`I`), denotes the type of alternatives appropriate at that point in the domain model. This type determines which specialized learning algorithm should be applied to refine its consequent (for example, TYPE("`CarStarts` $\prec_{\texttt{bool}}$ `HasGas`") = `BooleanCombination`. The first part of Figure 2.1 summarizes these functions defined over the space of influents. The notation used in the figure describes the input/output behavior of the various functions making up the framework. So the notation, "CONSEQUENT: $\mathbf{I} \rightarrow$ `term`" describes a function, CONSEQUENT, that accepts an influent from $\mathbf{I}$, and returns consequent term, `term`.

Like conventional rule-based induction systems[6, 7, 8, 9, 10], PI accepts some explicit representation of bias (background knowledge), a set of training examples, and a set of target concepts. In our approach, background knowledge is provided as a set of influents, `K`. This set of influents is called its *plausible theory* because this theory describes the set of *plausible* definitions for each domain term.

The training data provided to the system is encoded as a set of assignments of values to terms. We denote an assignment as: $\{term_1 = value_1, term_2 = value_2, \ldots\}$. Each train-

ing instance available to PI is represented as such an assignment. So {CarStarts = TRUE, GasLevel = 25%, InchesOfRain = .3*in*, BatteryLevel = 90%} might be a training instance for the car-starting task. The set of all training instances (assignments) available to PI is denoted by O, the observable data. The target concept or goal of a PI system, G, is the set of top-level domain terms PI is charged with predicting. In the example above, G, is the singleton set {CarStarts}. The three sets K, O, and G specify a learning task; the set of such triples is denoted **P**—the set of well-formed problem specifications for the framework. Figure 2.1 summarizes these inputs to PI.

We draw on constructs from a conventional first-order language, **L**, in defining various components of our framework.[11] PI uses terms, $\mathbf{L}_{term}$, from **L** in several ways: they are used to denote observations in O like InchesOfRain = 1.0, and intermediate terms like HasGas, and goals in G are expressed as terms like CarStarts. In order to distinguish the appropriate use for each term in the framework, each term is associated with a type. If a term, T, is the consequent of some influent I in K, then TYPE(T) = TYPE(I); otherwise, it is assumed to be an observable aspect of the domain, so TYPE(T) = Observable. In the example above, TYPE(HasGas) = Threshold and TYPE(GasLevel) = Observable.

The *refinement space* is the basic building block in our framework. Each refinement space represents a mini-induction task. As shown in Figure 2.1, each refinement space is tuple with the following form: ⟨term, ID, STATE, TARGET⟩. The first position is some domain term from **L** whose value is being induced by this refinement space. For example, the term, HasGas, might be predicted by some refinement space.[1] The second position is an arbitrary identifier for the space. The third position holds the internal state for the induction algorithm for this space. It always starts as ⊥ (undefined). The fourth position is a weighted set of TARGET values for the term predicted by this space. This

[1]For simplicity, our example employs propositional refinement spaces; later in more complex applications of PI we will see refinement spaces which are denoted by first-order terms (e.g. $Friend(x, y)$).

TARGET serves as the training data for this space. Recall that some of these refinement spaces may be charged with predicting terms that are not directly observable (not in O). Expected values for these intermediate terms are propagated between refinement spaces and are captured in the weighted TARGET vector. An example of a possible refinement space from the car domain might be:

$$\langle \texttt{CarStarts}, \textit{goal}, \bot, \langle 1 \text{ TRUE}, 3 \text{ FALSE}, 1\text{TRUE} \rangle \ \rangle$$

The numbers in the TARGET vector refer to the weights associated with each TRUE/FALSE value. Each value in the TARGET vector is a prediction of CarStarts for the corresponding training instance in the vector of training examples, O.

Plausible Inference works by applying induction to each refinement space relevant to the current set of goals. Each iteration will provide better assignments for intermediate terms upon which the next iteration can be based. We use **S** to denote the entire state of the PI system. This state is simply the set of currently active refinement spaces (including the internal state for their associated refinement algorithms, and their target values).

The framework defines four functions that operate on refinement spaces in this global state: HYP, RELEVANT, REFINE, and BEST. Collectively these functions control PI's behavior. The first function HYP(r) is not actually computed by PI, it simply denotes the hypotheses potentially considered by that space's refinement algorithm; i.e., its hypothesis space. The second function determines which refinement spaces should be considered by PI. The RELEVANT function accepts a refinement space and returns a set of sub-refinement spaces *relevant* to predicting the input space. PI determines what sub-induction problems are relevant to G, by first constructing refinement spaces for each goal in G, and then recursively calling the RELEVANT function on these spaces and on the spaces generated by these spaces. This chaining process continues until each refinement space depends only on other refinement spaces or on the observable terms in O. In the example above, CarStarts depends on GasLevel and PlugsWork,

so RELEVANT(CarStarts) = $\{\langle$HasGas, ...$\rangle, \langle$PlugsWork, ...$\rangle\}$. Both of these refinement spaces, however, only depend on observable terms (GasLevel, InchesOfRain, and BatteryLevel) so no new refinement spaces are generated by calls to RELEVANT on HasGas or PlugsWork.

Once PI has computed the set of relevant refinement spaces, induction is performed over these spaces using the REFINE function. REFINE accepts a refinement space, R, and a set of observations, O, and attempts to select some function in HYP(R) that best predicts the TARGET in R given the observations, O. Whatever progress (if any) made toward this end is recorded by updating the STATE of the refinement space. We provide no additional information about this state since each specific refinement algorithm is likely to have its own unique requirements for it local state. The only way the framework operate on a refinement space's STATE is by apply the BEST function to that state. This function either returns $\perp$ if no "best" hypothesis has been computed, or its returns some function from HYP(R). If returned, this hypothesis will compute values for the refinement space's term directly from observations in O. So, BEST($\langle$HasGas, ...$\rangle$) might return HasGas := (GasLevel $\geq 7\%$). Figure 2.1 summarizes all four of these functions that manipulate refinement space objects.

How do these functions combine to form a model of structured induction? As a first approximation, PI computes the space of RELEVANT refinement spaces, these spaces are REFINEd, and the BEST hypotheses for each are returned. A detailed account of PI's top level control is presented at the end of this chapter.

2.3 Generic Refinement Space Types

One central aspect of our approach is the use of specialized learning algorithms for each refinement space. These specialized algorithms are integrated into the framework

I is the set of well-formed influents. Functions defined over the influents:

TYPE:	$\mathbf{I} \rightarrow \mathbf{T}$
ANTECEDENTS:	$\mathbf{I} \rightarrow \{\texttt{term}_1, \texttt{term}_2, \ldots, \texttt{term}_n\}$
CONSEQUENT:	$\mathbf{I} \rightarrow \texttt{term}$

P is the set of well-formed problem specifications. Each P in **P** is a tuple $\langle \texttt{K}, \texttt{G}, \texttt{O} \rangle$ where

$\texttt{K} = \{\texttt{I}_1, \ldots, \texttt{I}_n\}$	A background a tuple $\langle \texttt{K}, \texttt{G}, \texttt{O} \rangle$ where
$\texttt{K} = \{\texttt{I}_1, \ldots, \texttt{I}_n\}$	A background theory of influents ($\texttt{I}_i \in \mathbf{I}$).
$\texttt{G} = \{\texttt{term}_1, \ldots, \texttt{term}_n\}$	A set of goals (domain terms to predict).
$\texttt{O} = \langle \texttt{OBSERVATION}_1, \ldots, \texttt{OBSERVATION}_n \rangle$	A set of training instances.

R is the set of possible refinement spaces.
S is the set states for the PI system. Each state is some set of refinement spaces $\{\texttt{R}_1, \ldots, \texttt{R}_n\}$.
Each refinement space, R, in **R** is a tuple: $\langle \texttt{term}, \texttt{ID}, \texttt{STATE}, \texttt{TARGET} \rangle$

$\texttt{term} \in \mathbf{L}_{term}$	The term computed by the refinement space.
ID	A Unique identifier for the refinement space.
STATE	An Internal state for a the induction of a refinement space term.
TARGET =	$\{weight_1\ term_1 = val_1, \ldots, weight_n\ term_n = val_n\}$
	A weighted set of "target" values for some unobserved term.

Framework Control:

RELEVANT:	$\mathbf{P} \times \mathbf{R} \rightarrow \{\texttt{R}_1, \ldots, \texttt{R}_n\}$	where $\texttt{R}_i \in \mathbf{R}$
PICK:	$\mathbf{P} \times \mathbf{S} \rightarrow \mathbf{R}$	
REFINE:	$\mathbf{P} \times \mathbf{R} \rightarrow \mathbf{R}$	
HYP:	$\mathbf{P} \times \texttt{term} \rightarrow \{\texttt{FN}_1, \ldots, \texttt{FN}_n\}$	
BEST:	$\texttt{STATE} \rightarrow \texttt{FN}$	"Best" refinement space predictor given its STATE

Base Types:

FN :	$\texttt{term}_1 \times \ldots \times \texttt{term}_n \rightarrow \texttt{term}_0$ Computes some domain term $\texttt{term}_0$ from a set of other term values.
OBSERVATION =	$\{term_1 = val_1, \ldots, term_n = val_n\}$ where $term_i \in \mathbf{L}_{term}$, and $val_i \in \mathbf{L}_{val}$
	A set of assignments that represent a single training instance.
$\mathbf{L}_{term}$	The set of terms in the first-order language **L**.[2]
$\mathbf{L}_{val}$	The domain of **L** (possible values for the terms, $\mathbf{L}_{term}$).
T	The set of refinement space types.

Figure 2.1 Definitions Required for the PI framework

through a common interface. This interface consists of four functions (HYP, RELEVANT, REFINE, and BEST) that are specified for each refinement space type. These functions capture the inductive learning algorithms and influent representation used by each type of refinement space. As we discussed in the introduction, the expert instantiates a small library of these types in different combinations in order to construct a task-specific learning algorithm over a wide range of learning tasks.

In the next several subsections we present the refinement types that have been used in our applications of PI which we also believe have general applicability. Each section defines a different refinement space type. The specialized syntax for the influents used by that space is defined. Sample domain expertise captured by the refinement space type is given, and the four functions needed to instantiate the refinement space type within our framework are discussed.

2.3.1 Parametric Refinement Space

A parametric refinement space represents induction over a parametric set of real-valued functions. Performing induction over hypotheses from the set of all quadratic functions of the form $y = \mathtt{a}_1 x^2 + \mathtt{a}_2 x + \mathtt{a}_3$, for example, could be expressed as a parametric refinement space with $\mathtt{a}_i$ as parameters. Refinement algorithms for this space would choose values for $\mathtt{a}_i$ that optimally predicted values of y given values of x.

The parametric refinement space is interesting because of its wide applicability in physical domains. The laws that govern physical objects often describe some family of functions, and the specific physical details of the objects provide various coefficients for those functions. Both the law and the specific description are needed to make predictions about the domain, but since a complete physical description is often much more complex and task-specific than the description of the governing law, the expert may be able

to specify the law but not the coefficients. This type of ambiguity is captured by a parametric refinement space.

An example of a parametric refinement space is an expert's model of the force required to push a concrete block across a flat table. The force required is a function of the texture and material of both the block and table as well as the weight of the block. The expert that knows friction is linearly related to the normal force applied, but does not know the specifics of the surfaces, could concisely express this partial understanding as a space of linear relationships between the weight of the block and the force needed to push the block. In this case the influent description of the expert's knowledge is ideal; it allows them to encode all constraints available without forcing commitment to unmotivated constraints (e.g. particular values for coefficient of friction). This flexibility is the goal of the various refinement space types. In our exploration of the biological domain we found numerous instances of expertise that is captured by a space of parametric functions. We believe that the prevalence of this type of ambiguity in the HHM domain is ultimately tied to the fact that protein folding is a physical domain with complex objects whose behavior is based on simpler laws of physics. Because many domains have a physical component we include parametric as one type of refinement space.

A parametric refinement space is declared using the following influent:

$$\texttt{consequent} \quad \prec_{\texttt{para}} \ \langle \texttt{skolem}_1, \ ..., \texttt{skolem}_n \rangle \ \texttt{antecedent\_expression}$$

It declares that the consequent term on the left can be computed as a function of the antecedent terms on the right hand side of the sentence by correctly instantiating the `skolem` parameters in the `antecedent_expression`. The `antecedent_expression` can be a composition of any mathematical operators defined in the instantiated PI system. The range of values that may be adopted by each $\texttt{skolem}_i$ is encoded using one of the following:

$\texttt{skolem}_i \prec_{\texttt{range}} [\texttt{low} - \texttt{high}]$

$\texttt{skolem}_i \prec_{\texttt{range}} [\texttt{low} - \texttt{high}, \texttt{step}]$

$\texttt{skolem}_i \prec_{\texttt{range}} \{\texttt{value}_1, \ldots, \texttt{value}_n\}$

Our expert's understanding of the concrete block on a table might be encoded as:

$\texttt{PushingForce}(brick, table) \prec_{\texttt{para}} \langle \texttt{ContactCoefficient}(brick, table) \rangle$

$\texttt{Weight}(brick) \cdot \texttt{ContactCoefficient}(brick)$

$\texttt{ContactCoefficient}(x, y) \prec_{\texttt{range}} [0.0 - 1.0]$

The space of hypotheses, $\textsc{Hyp}(r)$, considered by a parametric refinement space, r, defined by the influent, `I`, is simply the set of all possible combinations of the skolem terms in `I`:

$\{low_1, ..., high_1\} \times ... \times \{low_n, ..., high_n\}$

A number of REFINEment procedures are well suited to this particular type of induction[12]. In our application of PI, we have used two of these. We have quantized the multi-dimensional space, and exhaustively tested the grid points. We have also used a simulated-annealing hill climbing method[13, 14]. In either case, the BEST function simply returns the antecedent of `I` instantiated with the $\texttt{skolem}_i$ values that best predicted the `TARGET` of the refinement space.

2.3.2 Linear Factors Refinement Space

Induction over a space of linear factors is a commonly occurring specialization of the general class of parametric functions. When an expert is confident that a given prediction task involves several *non-interacting* factors, but does not know the relative strength of those factors, the space of weighted averages of those factors is a very low-dimensional (strongly biased) option. The following influent declares a linear-factor that can be combined with others to form a linear-factors refinement space:

$\texttt{consequent} \prec_{\texttt{factor}} \langle \texttt{weight} \rangle \quad \texttt{antecedent}$

$\texttt{weight} \prec_{\texttt{range}} \ldots$

Its semantics are identical to the parametric refinement space where all $\prec_{\texttt{factor}}$ influents that share the same CONSEQUENT serve as the factors in an implicit weighted average parametric function:

$$\texttt{weight}_1 \cdot \texttt{antecedent}_1 + \ldots + \texttt{weight}_n \cdot \texttt{antecedent}_n$$

A related refinement space type is the linearly weighted threshold. It has the same syntax and semantics, but its output term is Boolean, so it is some threshold over a linear combination of its inputs. We use the influent symbol $\prec_{\texttt{lin}}$ to denote this refinement space. Its coefficients are also assumed to be positive, so it can be used to specify the polarity between its inputs and outputs. Although both of these spaces are completely contained within the more general parametric space we do not used the same algorithms for these spaces since constrained REFINEment algorithms are available. Both of these spaces have a known monotonic relationship between the input and output terms. Winnow[15], and the perceptron learning algorithm[16, 17] are two examples of algorithms developed specifically to take advantage of this monotonic relationship.

2.3.3 Threshold Refinement Space

Another specialization of the parametric space that admits an efficient refinement algorithm is the threshold refinement space. When integrating symbolic and numeric information, a threshold is often used. We have already seen examples of the threshold refinement space in our running example; `HardRain` is a Boolean term that is computed from a real-valued term `InchesOfRain` by inducing an appropriate threshold. The following influent declares that `HardRain` is computed from `InchesOfRain` using a threshold

between 0.1 and 4.0 inches of rain:

$$\texttt{HardRain} \prec_{\texttt{th}} \langle \texttt{HardRainThreshold} \rangle \, \texttt{InchesOfRain}$$

$$\texttt{HardRainThreshold} \prec_{\texttt{range}} [0.1 - 4.0]$$

In general a threshold influent is written:

$$\texttt{consequent} \prec_{\texttt{th}} \langle \texttt{threshold} \rangle \, \texttt{antecedent}$$

$$\texttt{threshold} \prec_{\texttt{range}} \ldots$$

The HYPotheses in the threshold refinement space are the values in the <range> of its threshold. The space can be REFINEd very efficiently using a binary search. The number of false positive and false negative predictions are compared for each possible threshold in the binary search. If the number of false positives is greater, the threshold is moved lower, if the number of false negatives is greater, the threshold is raised.

2.3.4 Boolean Refinement Space

The Boolean refinement space is the space of all logical combinations of a set of two-valued terms. Formally, the `BooleanCombination` space for a set of terms, T, is its transitive closure over the logical-and ($\wedge$), logical-not ($\neg$), and variable renaming.[3] This is the space of hypotheses considered by conventional non-numeric induction algorithms. Decision tree algorithms[19, 20], inductive logic programming[21, 22, 23], artificial neural networks[24, 25], and rule induction engines[26, 27], are examples from the inductive learning literature that operate over the Boolean space. Furthermore, most of these algorithms are *complete* with respect to the Boolean space of hypotheses. The notion of completeness is central to our analysis of various learning algorithms, thus we take a moment now to provide a formal definition for the concept. An inductive learning

[3] If the terms, T, are first-order (have variables) then the variable renaming operator is needed to obtain all possible variable bindings. So $P(u)$ and $Q(v, w)$ could be combined as $P(x) \wedge Q(x, y)$ or as $P(y) \wedge Q(x, y)$. (See page 65 of *Logical Foundations of Artificial Intelligence* for a discussion of variable renaming[18].)

algorithm, A, is *complete* with respect to a set of hypotheses, H, iff for all hypotheses h in H, there exists a set of observations, O, such that A outputs h given O. That is, an algorithm is complete with respect to a space if it can, in principle, induce *every* member of that space. In the next subsection we define a refinement space algorithm that has increased performance precisely because it is *not* complete with respect to the Boolean space. An algorithm is simply said to be *complete* if the space of hypotheses covered by the algorithm is any fixed set, i.e., the space of hypotheses is not a function of explicit bias provided to the algorithm.

Indeed we have included the Boolean refinement space here primarily to show how complete Boolean induction fits within our framework, and also to show how these well-known induction algorithms relate to the refinement spaces we have considered in terms of the strength of bias used in each. We have not used this weakly-biased form of induction in our applications of PI to date. The following influent encodes a Boolean refinement space:

$$\texttt{consequent} \prec_{\texttt{bool}} \texttt{antecedent}_1, \ldots, \texttt{antecedent}_n$$

2.3.5 Overriding Refinement Space

An *overriding combination* is a restricted type of Boolean expression. We have focused a considerable amount of attention on this class of expressions because of their utility in the decomposition of symbolic (non-numeric) domain expertise. In fact, in an earlier paper we describe a restricted form of plausible inference that relies solely on this type of refinement[28, 29].

An overriding refinement is a Boolean expression that is consistent with a set of polarity statements supplied by the expert. For example, if an expert declares *birdness* to be positively related to *flight*, and *deadness* to be negatively related, the PI would

consider hypotheses like "non-dead bird's fly," and "dead, non-bird's don't fly," but not consider hypotheses like "non-birds fly" because that violates the bird-fly positive polarity.

Each influent in an overriding refinement space defines a relevant combination of terms and its polarity in any hypothesis considered. Each polarity assertion is declared with the following influent:

$$\texttt{consequent} \prec_{\texttt{ovr}} \texttt{antecedent\_expression}$$

All $\prec_{\texttt{ovr}}$ influents with the same `consequent` define the space of hypotheses considered in the refinement of that term.

Formally the space of hypotheses in an overriding refinement space defined by $\texttt{K} = \{\texttt{conseq} \prec_{\texttt{ovr}} \texttt{ante}_1, ..., \texttt{conseq} \prec_{\texttt{ovr}} \texttt{ante}_n\}$ is the transitive closure of the operators shown in Figure 2.2. By eliminating the '$\neg$' operator from this restricted space of hypotheses, we ensure that the polarity constraints of the influents are not violated by the hypotheses in $\textsc{Hyp}(\texttt{K}, r)$. Precisely, we ensure that a $\texttt{term}_i$ in some hypothesis, h in $\textsc{Hyp}(\texttt{K})$ occurs negated[4] iff $\texttt{term}_i$ occurs negated in some k in `K`. As an example, consider the background knowledge, $\texttt{K} = \{\texttt{Fly} \prec_{\texttt{ovr}} \texttt{Bird}, \texttt{Fly} \prec_{\texttt{ovr}} \neg\texttt{Dead}\}$. The space of overriding refinements, $\textsc{Hyp}(\texttt{K}) = \{\texttt{Fly} \equiv \texttt{Bird}, \texttt{Fly} \equiv \neg\texttt{Dead}, \texttt{Fly} \equiv \texttt{Bird} \wedge \neg\texttt{Dead}, \texttt{Fly} \equiv \texttt{Bird} \vee \neg\texttt{Dead}\}$

The refinement space algorithm we employ for this space is similar to Michalski's "star" algorithm used in his AQ* systems[30]. In this algorithm, a single influent from `K` is used to explain (correctly predict) some unexplained observation in `O`. The operators shown in Figure 2.2 are used to restrict the applicability of the influent to avoid other predictions inconsistent with the training data, `O`. Once no further restrictions are needed

[4] A term, $\texttt{term}_i$, occurs in negated form in an expression, E, iff it has a negation sign immediately preceding an occurrence of it when writing E in canonical CNF form. (CNF form is a single conjunction of disjunctions of possibly negated terms.)

$\textsc{Hyp}(\mathtt{K})$ is the smallest set that satisfies:
$\forall\ q \prec_{\mathtt{ovr}} \alpha,\ r \prec_{\mathtt{ovr}} \beta\ \subseteq \textsc{Hyp}(\mathtt{K})^5$

1. $\mathtt{K}\ \subseteq \textsc{Hyp}(\mathtt{K})$
2. $r\theta \prec_{\mathtt{ovr}} \beta_{q'|(\alpha)}\ \in \textsc{Hyp}(\mathtt{K})^6$ where θ is the mgu for f and r **Chaining**
3. $q\theta \prec_{\mathtt{ovr}} \alpha\theta \wedge \beta\theta\ \in \textsc{Hyp}(\mathtt{K})$ where θ is the mgu for q and r **Conjoining**
4. $q\theta \prec_{\mathtt{ovr}} \alpha\theta \vee \beta\theta\ \in \textsc{Hyp}(\mathtt{K})$ where θ is the mgu for q and r **Disjoining**

$q := \alpha\ \in \textsc{Hyp}(\mathtt{K}, r)$, iff q unifies with r and $q \prec_{\mathtt{ovr}} \alpha\ \in \textsc{Hyp}(\mathtt{K}, r)$

Figure 2.2 Definition of the overriding refinement space

or are possible, another unexplained training instance from `O` is selected and the process is repeated until no unexplained instances exist or no explanations remain to be tried.

Figure 2.3 shows the pseudo-code for this algorithm. The first procedure, `RefineDisjunct`, grows a disjunct whose terms "cover" the positive (TRUE) observations. For each term, it calls the dual procedure `RefineConjunct` to grow that term in a so as to restrict its applicability over the negative (FALSE) observations. As the algorithm executes, new "relevant" refinement spaces are generated in the first step of both procedures. These spaces must be refined before the selection step in the second step of the procedures. Thus the execution of this refinement algorithm must be interleaved with the refinement of other spaces. (This is exactly why each refinement space has an independent internal state.) We demonstrate this interleaving in our sample trace of the PI at the end of this chapter.

The algorithm that was implemented and discussed in earlier publications[31] is slightly more complex than what is described here in two ways: it employed iterative deepening in both the derivation of relevant term combinations and in the overriding of overrides, and it allowed more complex influents—their antecedents could include arbitrary Boolean expressions. In Chapter 3 we empirically demonstrate how this restricted space of hypotheses can result in dramatically improved learning performance.

```
RefineDisjunct(K, O, term)
   BestTerms ← ⊥
   A ← RefSpace for each antecedent for rules in K whose consequent is term
   while (A ≠ ⊥) ∧ BestTerms is not a perfect predictor of O
      Select a in A that best predicts O not covered by ⋃BestTerms
      O' ← { o in O | o satisfies A }
      RefineConjunct(K, O', term)
   end
   return ⋃BestTerms
end

RefineConjunct(K, O, term)
   dual of the RefineDisjunct procedure ...
```

Figure 2.3 Refinement algorithm for the `OverridingCombination` space

2.3.6 Enumerative and Definitional Refinement Spaces

The enumerative refinement space is the most basic specification of a space of hypotheses. In this refinement type, the possible hypotheses are explicitly enumerated within the plausible theory. This set of hypotheses is then exhaustively searched. In our later application of PI you will see a number of occurrences of this type of refinement space. It occurs when the expert has reason to suggest two or more ways of computing some intermediate term. The expert can encode all available alternatives, and let induction test to see which is most predictive for each learning task. Each alternative in an enumerative refinement space is written:

`consequent` $\prec$ `antecedent—expression`

The set of all influents with the same CONSEQUENT term defines the space of hypotheses searched by enumerative refinement. Thus, given $\texttt{K} = \{\texttt{conseq} \prec \texttt{ante}_1, ...\}$ the space of hypotheses is simply the explicitly encoded alternatives: $\textsc{Hyp}(\texttt{K}, \texttt{conseq}) = \{\texttt{ante}_1, ...\}$

Definitional refinement is a degenerate refinement space with only one hypothesis; it is used to encode deductive knowledge in the PI framework. A definitional refinement is

written as:

`consequent` ← `antecedent—expression`

Anytime `consequent` occurs in a hypothesis it is replaced with `antecedent—expression`. No evaluation is performed, PI simply assumes the resulting hypothesis is optimal. This behavior is appropriate when the expert knows the provided definition does not need to be refined by learning. If multiple definitions are provided, PI for a given consequent is free to PICK any of these definitions without search.

2.4 Top-Level Control for the PI Framework

Plausible Inference, our generic framework for learning decomposition and specialization, has been presented along with eight particular refinement space types that fit within this framework. We now tie this formalism together visually using a block diagram pictorial of the framework, and follow that with a hypothetical trace of the car-starting problem using the PI framework.

Figure 2.4 provides a block diagram of PI's behavior in terms of the model presented above. Parameters that vary with each learning task are listed on the top left, and include the background theory of influents, `K`, the target concepts, `G`, and the training data, `O`. The Stack of boxes labeled "Car Starts," "Protein Prediction," etc. denotes the various tasks that might be assigned to the PI system, and their associated inputs. These inputs are combined into an initial state for PI. The Stack of boxes labeled "`Threshold`," "`Parametric`," etc. denote the refinement space types available within the framework. The functions listed in these boxes, RELEVANT, REFINE, and BEST are instantiated once for each refinement space—they *define* that refinement space type. PI iterates through a relevance-pick-refine loop as shown in the figure. The function RELEVANT is recursively applied to the goal terms, `G`, to arrive at a set of intermediate terms and

associated evaluation metrics which may need to be predicted as a subgoal of the main prediction task. The PICK function is used to select one of these spaces, an appropriate REFINEment procedure, and an evaluation `TARGET`. These are applied to the current state of the PI system resulting in a new state where the selected refinement space has been updated. A new definition for the intermediate term associated with that refinement space is computed using the BEST function associated with that type of refinement space; the observations, `O`, are then updated to reflect the new definition for this intermediate term. At any point, a current "best" hypothesis can be generated by using the BEST function to operationalize the target concept(s), `G`. Starting with the predicates in `G`, each non-operational predicate is expanded by replacing it with the BEST definition of that predicate (from its associated refinement space). This process is repeated until the entire definition for the target is expressed using only operational predicates. (Here we use the term operational to refer to those predicates that are available in the original observations, `O`.)

The block diagram shown in Figure 2.4 nicely depicts the object-oriented nature of the approach and because it provides a visual depiction of PI's top-level control. Figure 2.5 provides an algorithmic specification of PI's top-level control. This characterization is more precise in specifying the data flow within the PI. The state is a set of refinement spaces in $State_i$ that is updated each iteration. During each iteration the spaces from the previous iteration are queried to see if there are any new RELEVANT sub-refinement spaces. Then a space is chosen and refined. O' represents the initial observations augmented with instantiations for all intermediate terms that have some BEST hypotheses selected from previous iterations. The results of the refinement are stored in $State_i$—PI's internal state at the end of this iteration.

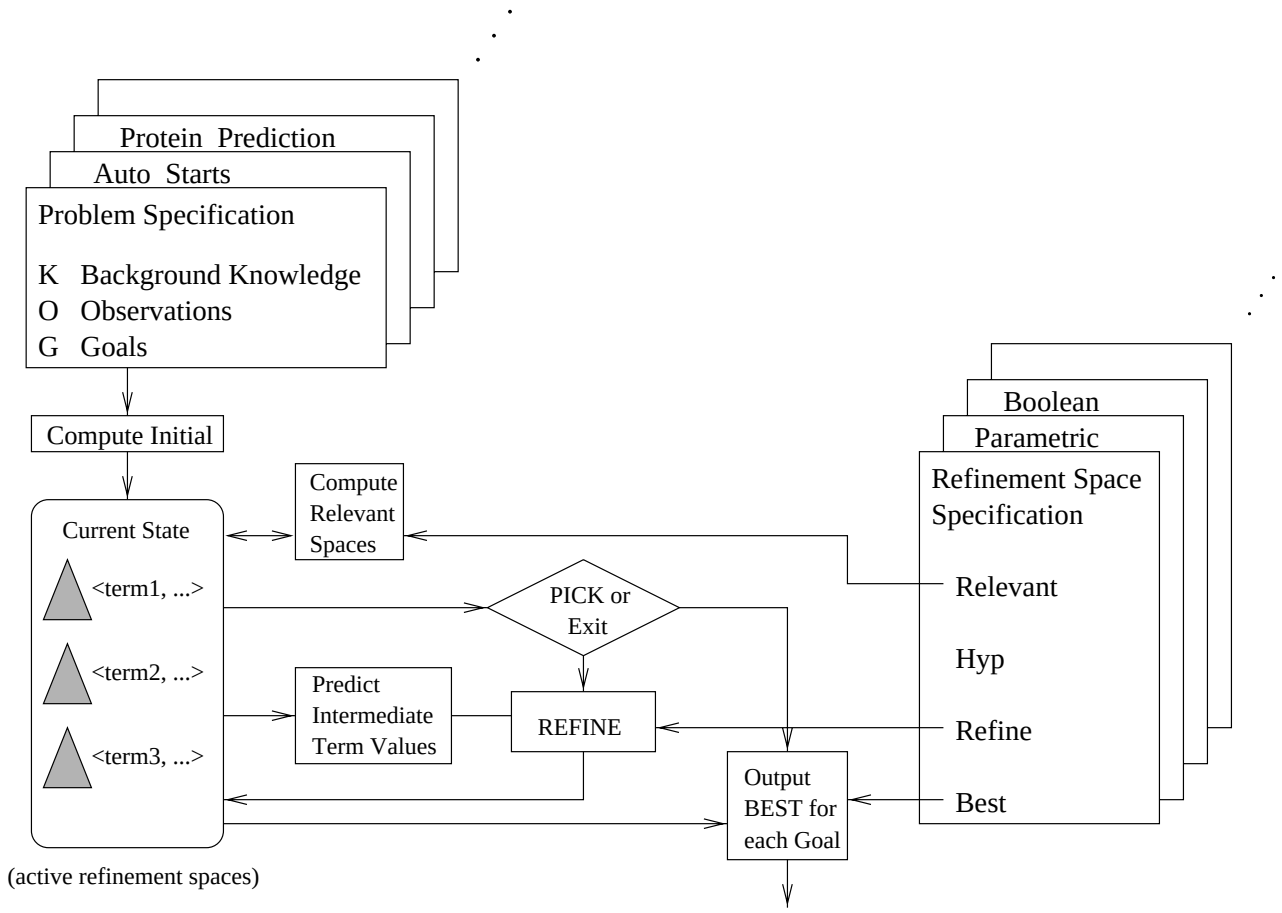

Figure 2.4 Data flow diagram for the PI framework

Input: K, O, G
Output: BEST predictors for each $g \in$ G

$State_0 \leftarrow \{ \langle term_1, goal, \perp, \texttt{ExtractValuesFor}(term_1, \texttt{O})\rangle,$
$\langle term_2, goal, \perp, \texttt{ExtractValuesFor}(term_2, \texttt{O})\rangle \ldots\} \quad \forall term_i \in \texttt{G}$
For i **from** 1
$State_i \leftarrow \bigcup$RELEVANT$(r) \quad \forall r \in State_{i-1}$
$\texttt{O}' \leftarrow \texttt{O} \bigcup \texttt{ComputeObservationsFor}(\text{BEST}(r), \texttt{O}) \quad \forall r \in State_i$
$CurrentSpace \leftarrow$PICK$(state_i)$
If $(State_i = State_{i-1})$ or $(CurrentSpace = \perp)$ **then**
Return $\bigcup$BEST$(r) \quad \forall r \in State_0$
REFINE(R, O′)
End

Figure 2.5 Top-level control flow within the PI framework

2.5 Putting It All Together: Car Starting Revisited

We have used the problem of predicting automobile starting behavior to motivate PI's use of structured knowledge. We now expand this example into a concrete learning task, and trace PI's behavior on that example.

According to Figure 2.1, a learning task is expressed as a triple $\langle \texttt{K}, \texttt{G}, \texttt{O}\rangle$. For this learning task we use the domain knowledge shown in Figure 2.6 as K. Each element of O, shown in Figure 2.7, is a description of an observed automobile (including whether it started or not). G is the attribute being predicted, in this case $\texttt{G} = \{\texttt{CarStarts}\}$. The initial state ($State_0$) is the set of relevant refinement spaces for the goals in G. Each goal is G is converted into a refinement space. In this example, only one space is constructed so $State_0$ is a set containing only one instantiated refinement space:

$$\langle \texttt{CarStarts}, goal, \perp, \langle 1 \text{ TRUE}, 1 \text{ FALSE}, 1 \text{ FALSE}, 1 \text{ TRUE}, 1 \text{ TRUE}\rangle\rangle$$

The fourth position in each refinement space tuple is the TARGET for that space. Top-level (GOAL) spaces simply use the observed values for its term in the observations O. The

1. `CarStarts` $\prec_{\texttt{ovr}}$ `HasGas`
2. `HasGas` $\prec_{\texttt{th}}$ $[0, 100]$ `GasLevel`
3. `CarStarts` $\prec_{\texttt{ovr}}$ `PlugsWork`
4. `PlugsWork` $\prec_{\texttt{lin}}$ `BatteryLevel`
5. `PlugsWork` $\prec_{\texttt{lin}} -$`InchesOfRain`

Figure 2.6 Domain structure for the car start example

vector shown is composed of the five `CarStarts` observations in `O`. The idea here is that models of each of these terms are sought that result in the specified values for the training data. The numbers in the target specify a weight for each training example. Since we have no initial information regarding the training data, all weights are set equally at 1. Refinement for each space is guided by the weighted predictive accuracy using the target defined for that space.

At the beginning of each interation of the PI algorithm a new set of relevant refinement spaces is computed by applying the RELEVANT procedure to the previous set of refinement spaces. Back chaining through the rules shown in Figure 2.6 suggests two additional refinement spaces—one for `HasGas` and one for `PlugsWork`. `TARGET`s for each refinement spaces are also computed by the RELEVANT procedure. The `OverridingCombination` relevance procedure initially propagates `TARGET`s that simply mirror that parent refinement space's `TARGET`.

As shown in Figure 2.4, each iteration is accomplished by computing the RELEVANT refinement spaces, PICKing a space, REFINEing that space, and UPDATEing the observations according to the new hypothesis for the subspace. One of the two subordinate spaces must be selected first since they are the only spaces whose antecedents are directly observable. We assume that the `PlugsWork` space is chosen first. Figure 2.8 shows a plot

	$x =$	CarStarts	GasLevel	BatteryLevel	InchesOfRain
1.	HealthyCar,	TRUE ,	20%,	80%,	0.0
2.	EmptyCar,	FALSE ,	**2**%,	70%,	0.0
3.	WeakWetCar,	FALSE ,	45%,	**40**%,	**1.0**
4.	WeakCar,	TRUE ,	40%,	30%,	0.0
5.	WetCar,	TRUE ,	50%,	80%,	1.5

Figure 2.7 Observations, O, for the Car Starting example

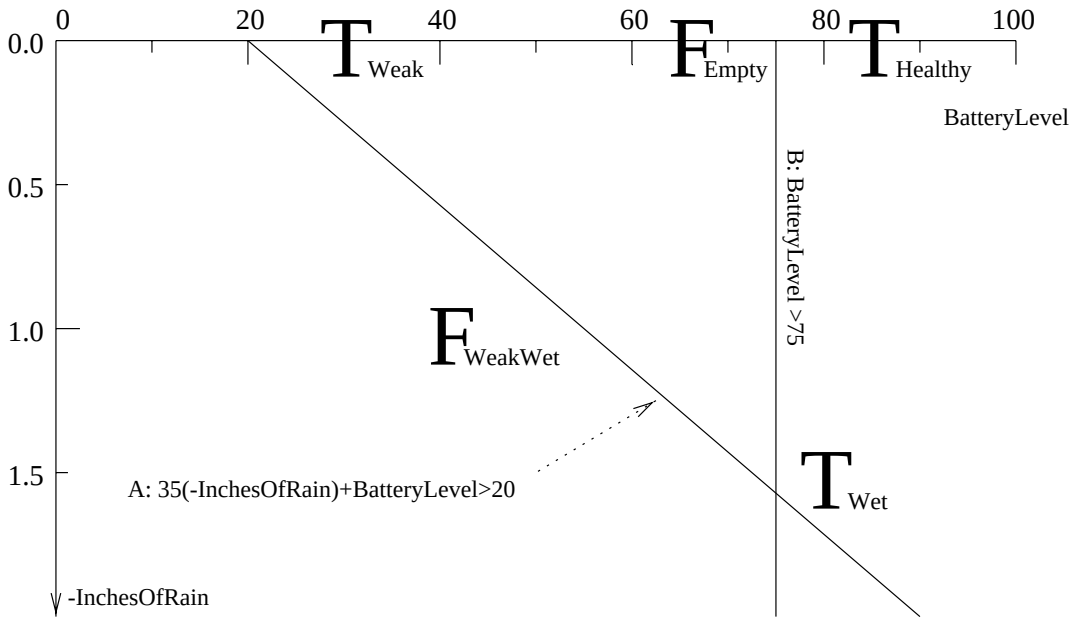

Figure 2.8 Initial split for the PlugsWork space

of the PlugsWork target values against that refinement spaces two numeric input parameters. The LinearFactor refinement space attempts to find a linear separator that best predicts its target. In this case several lines are equally predictive. (The problem here is that the EmptyCar is mislabeled as having plugs that don't work.) Let us assume that an incorrect predictor (line B) is first chosen. The PlugsWork refinement space is updated with this current predictor:

$$\texttt{PlugsWork} := (\texttt{BatteryLevel} \geq 75\%)$$

Car Instance		Empty		Healthy		Weak		WeakWet		Wet	
HasGas		F		T		T		F		T	
GasLevel		2		20		40		45		50	
	↑		↑		↑		↑		↑		↑
Score for split	2		1		2		3		2		3

Figure 2.9 Weighted error rates for each possible splits

	$x =$	CarStarts	HasGas	PlugsWork
1.	HealthyCar,	TRUE ,	TRUE ,	TRUE
2.	EmptyCar,	FALSE ,	FALSE ,	FALSE
3.	WeakWetCar,	FALSE ,	TRUE ,	FALSE
4.	WeakCar,	TRUE ,	TRUE ,	FALSE
5.	WetCar,	TRUE ,	TRUE ,	TRUE

Figure 2.10 Predicted values for intermediate terms

In the second iteration the HasGas space is refined. Since existing terms for the CarStarts space correctly predict all but the EmptyCar correctly, that example is given increased weight in our target for HasGas. Here all possible thresholds are considered for HasGas. Figure 2.9 shows the training data ordered by their GasLevel. It also shows the weight on each example, and the score each possible split would generate. In this case the weighting had no effect, and the best split is between 2 and 20, so its midpoint is used as the best model: HasGas := (GasLevel ≥ 11).

As the models for HasGas and PlugsWork were chosen above, the observations, O, were updated with indirect observations for those terms. Figure 2.10 shows the computed values for those terms.

Now that values are available for its antecedents, the CarStarts space may now be refined. According to the algorithm in Figure 2.3 the best single rule predictor is selected. With only five training examples, both PlugsWork and HasGas have the same predictive accuracy. We assume that CarStarts $\prec_{\texttt{ovr}}$ HasGas is chosen to cover the three

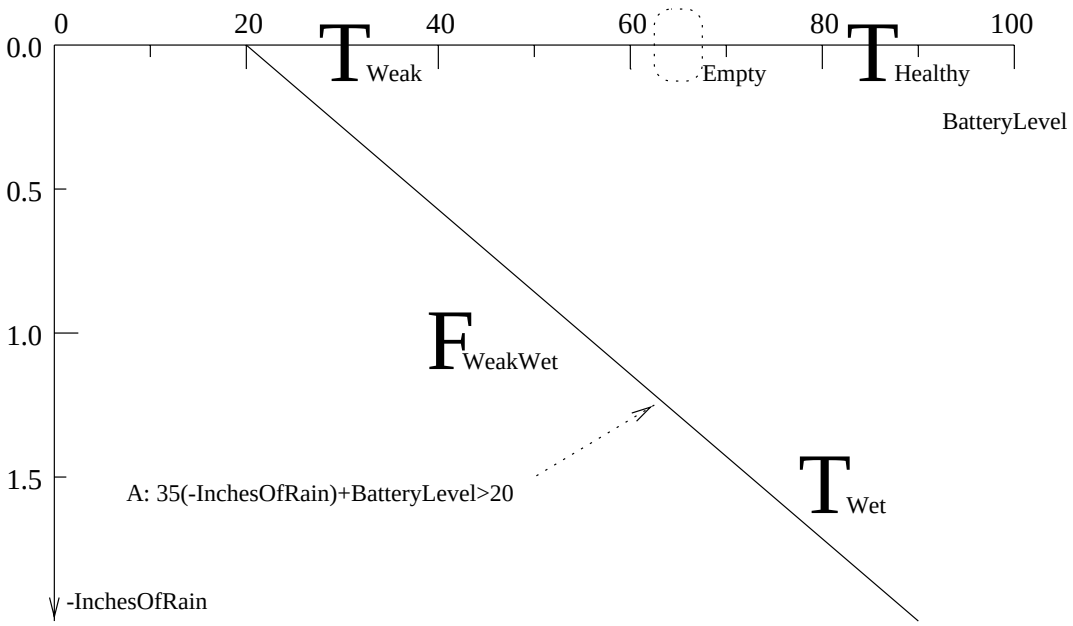

Figure 2.11 Second split for the `PlugsWork` space

positive examples of starting cars. The `OverridingCombination` calls the RefineConjunct procedure in Figure 2.3 which attempts to find other influents to constrain the `HasGas` term so it does not cover any non-starting cars. This is done by looking for refinement spaces that predict well specifically over the subset of observations that satisfy `HasGas`. On the next iteration the following refinement space will be added to PI's state:

$$\langle \texttt{PlugsWork}, \texttt{CarStarts} \prec_{\texttt{ovr}} \texttt{PlugsWork}, \bot, \langle 1\ \textsc{true},\ 0\ \textsc{false},\ 1\ \textsc{false},\ 1\ \textsc{true},\ 1\ \textsc{true} \rangle \rangle$$

Notice that it is very similar to the previous `PlugsWork` space but no weight is placed on the `EmptyCar` since that observation is not covered by the `HasGas` rule.

Figure 2.11 shows observations being split in the new `PlugsWork` space. Notice that, with no weight on the `EmptyCar`, only one split is now possible:

$$\texttt{PlugsWork} := \ 35 \cdot (-\texttt{InchesOfRain}) + \texttt{BatteryLevel} \geq 20$$

Now that its dependant refinement spaces have been computed, the `CarStarts` space may again be refined. The first refinement resulted in the `HasGas` term being added.

Now conjunctions for that rule may be considered. The `PlugsWork` definition computed in the last iteration excludes the `WeakWetCar` example as desired, so the following new refinement is adopted:

$$\texttt{CarStarts} := \texttt{HasGas} \wedge \texttt{PlugsWork}$$

At this point the BEST hypothesis for the `CarStarts` space correctly predicts all of the training data, so the algorithm halts. Applying the BEST function recursively starting that the goal term `CarStarts` results in the following output:

$$\texttt{CarStarts} := (\texttt{GasLevel} \geq 11) \wedge (35 \cdot (-\texttt{InchesOfRain}) + \texttt{BatteryLevel} \geq 20)$$

CHAPTER 3

EMPIRICAL EVALUATION OF THE PI FRAMEWORK

Our exploration of the PI framework is driven by a belief that specialized algorithms, specialized representations, and problem decomposition have essential roles in extending the power of existing machine learning into domains of greater complexity. This view prompts a number of more specific questions regarding its behavior: A central tenant of the PI approach asserts that decomposition of complex learning tasks into smaller refinement spaces allows the expert to use specialized algorithms and representations for each subspace in order to improve performance. How much performance gain over existing methods can be achieved through use of this specialization? The answer to this question is critical; overcoming the challenges associated with the combination of multiple, specialized learning algorithms is only justified if we expect the performance of these specialized algorithms to be significantly superior.

We take a learning task from a simple blocks world as our learning domain. An algorithm for PI's overriding refinement space is presented, along with two well-known, general-purpose learners from the learning community. These two learners were chosen because they are good representatives of the field, and because they are well suited to

the blocks domain we have chosen. The comparative performance of these systems is measured and reported.

3.1 A Test Bed for Comparison

The overriding refinement space was developed to handle discrete, first-order domains where the polarity relationships relevant for some prediction task are available, but a sound and complete theory is not available. We found this situation to be common in complex Boolean domains. The 1D-blocks world described below is the simplest domain we found which exhibited these features, thus it is used in our comparative testing.

Our test bed task is that of learning to project the effect of action in a simple one-dimensional blocks world. This world consists of fixed-sized blocks which can rest in integral positions along a horizontal surface (the table). A robot can push these blocks either left or right, along this surface. Figure 3.1 shows several possible configurations for this world. The state of our 1D blocks world is represented using situation calculus[32]. Situation calculus was chosen as a representational language for its simplicity and it generality. Consistent with situation calculus, the `At` predicate accepts three arguments: a block, a location, and a situation. The meaning is that the specified block is "`At`" the specified location, in the specified situation. Figure 3.1 provides the corresponding `At` predicates for each of the situations shown. The task is to learn to predict the `At` values in the final situation given the robot's action and the `At` predicates that describe the initial situation; i.e., the task is to predict the effect of actions in the 1D blocks world.

Each learning algorithm is presented with the predicates in Figure 3.1 as training data. Learning curves are constructed by looking at how performance changes as progressively larger subsets of this training data are presented. Some of the algorithms also accept knowledge about the domain as a hint to inform the learning. Figure 3.2 shows the rules

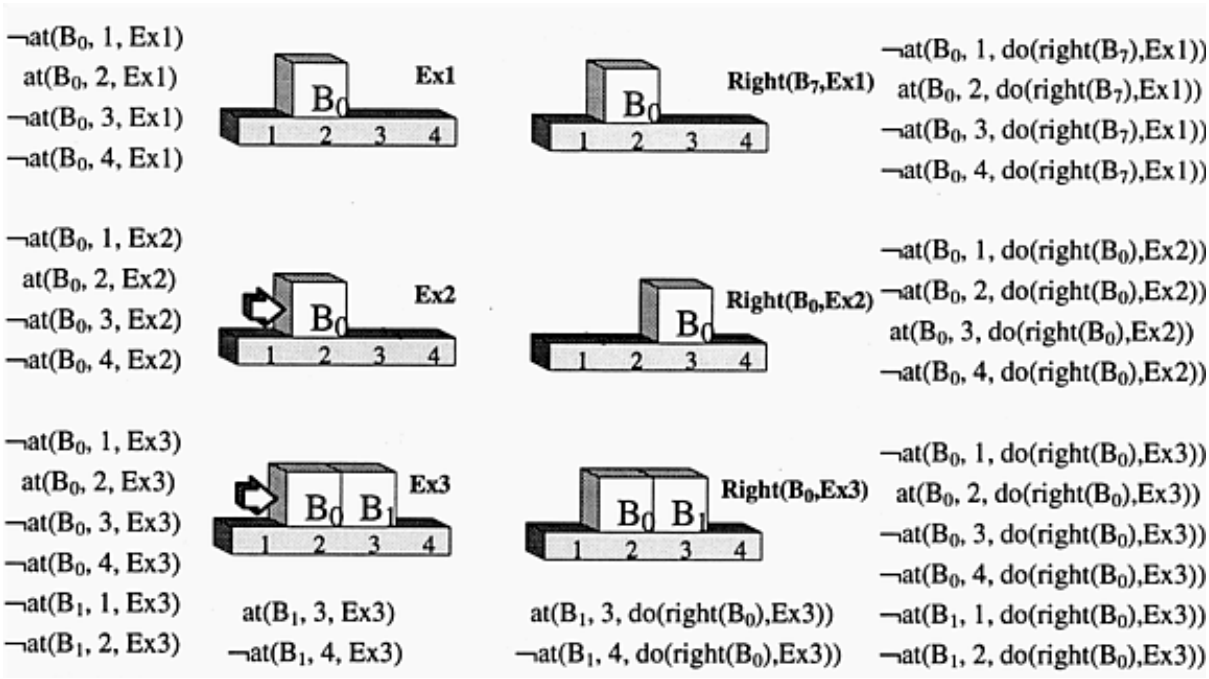

Figure 3.1 The 1D blocks world learning task

1. $\neg\texttt{At}(b, l, s) \prec_{\texttt{ovr}} \texttt{At}(b_1, l, s) \wedge \neg\texttt{Eq}(b, b_1)$ — No 2 blocks in one place.
2. $\texttt{At}(b, l, s) \prec_{\texttt{ovr}} \texttt{At}(b, l_0, s) \wedge \neg\texttt{Eq}(l, l_0)$ — No block in 2 places.
3. $\texttt{At}(b, l, s) \prec_{\texttt{ovr}} \texttt{At}(b, l, s_0) \wedge \texttt{Eq}(s, \texttt{Do}(a, s_0))$ — Blocks stay put.
4. $\texttt{At}(b, l, s) \prec_{\texttt{ovr}} \texttt{At}(b, l_0, s_0) \wedge \texttt{Add}(l, l_0, 1) \wedge \texttt{Eq}(s, \texttt{Do}(\texttt{Right}(b), s_0))$ — Move right.
5. $\texttt{At}(b, l, s) \prec_{\texttt{ovr}} \texttt{At}(b, l_0, s_0) \wedge \texttt{Add}(l, l_0, -1) \wedge \texttt{Eq}(s, \texttt{Do}(\texttt{Left}(b), s_0))$ — Move left.

Figure 3.2 A partial theory of the 1D blocks world domain

provided to those learners. This knowledge contains some rules that are deductively accurate descriptions of the world like rule #1. This rule is the first-order assertion equivalent to the English statement: "No two blocks occupy the same location in any given situation." Other rules are clearly not complete and accurate descriptions of the world; rule #3, "Blocks stay put," is violated in cases where an unobstructed block is explicitly pushed by the robot. Notice, like the learning task itself, these rules relate the initial situation with the final situation, but they do not form a sound and complete theory. Such a set of rules is what each learner is charged with inducing.

3.2 Experiment 1: The Utility of A Priori Domain Structure

Plausible Inference is predicated on the notion that induction constrained by background knowledge will perform significantly better than induction not constrained by an explicit model of the domain. Experiment 1 aims at measuring the performance gains actually attainable using background knowledge. After all, supplying such a priori models is a burden not required by some other approaches. As discussed above, one would be reluctant to embrace an approach that adds this requirement unless it also had the potential for significant performance improvements.

For this experiment we employ FOIL for comparison. FOIL is a well-known first-order rule induction engine developed by Ross Quinlan. It operates by greedily adding antecedents to Horn rules whose consequent is the predicate being predicted. So in the task above, FOIL would grow rules whose consequent was the `At` predicate. FOIL attempts to add all possible attributes as antecedents to the rules being grown, using all possible variable binding combinations. At each step it adds the attribute that maximizes the number of false positive eliminated while minimizing the number of true positives eliminated from the rule's predictions. FOIL continues to grow new rules as long it is possible to cover positive examples not covered by existing rules. Notice that this process only requires a list of predicates and their arguments. It does not (indeed cannot) accept any other knowledge about the domain. Thus, a comparison of PI and FOIL over the 1D blocks world prediction task is more a test of the utility of that background knowledge rather than a test of the inherent performance of either algorithm.

Experimental Setup

This experiment uses the `At` predicates shown in Figure 3.1 as its training instances.

A random set of these predicates are presented to each learning algorithm; then each algorithm is asked to predict some set of the unseen `At` predicates. Learning curves are generated for each system by measuring performance predicting unseen `At` predicates after an increasing number of training examples are presented. In order to gain statistical significance, ten-fold cross-validation is employed.[1] By assuming these fold measurements are statistically independent, we are able to use the T-distribution to compute confidence intervals around each measurement[33]. All differences shown in this and the next experiment are significant at the 95% confidence level.

Results

Figure 3.3 presents the results of this experiment. The average predictive accuracy for both algorithms is shown in the graph on the left. The horizontal axis represents the number of training instances (`At` predicate values) available to each learning algorithm; the vertical axis denotes the fraction of the test queries that are correctly predicted on average by each algorithm. For example, the point at (30, 95%) means that after 30 training instances were presented the algorithm had an average predictive accuracy of 95%. The dashed line at 83% represents the accuracy obtained by always predicting the a priori most probable class.[2] Figure 3.4 shows the lexically simplest set of rules that correctly predict this domain. Notice, while this theory is fairly complex, PI is able to quickly arrive at a predictive model of the domain. Figure 3.3 shows that after an average of 10 training instances PI attains a 95% average predictive accuracy.

FOIL's performance, on the other hand, remains below that of the hypothesis that simply predicts the a priori most probable class. FOIL grows rules to predict the available

[1] Using ten-fold cross-validation we partition the training data randomly into ten roughly equal sets. Nine sets are combined as a training set, and one is left as a testing set. This is repeated once for each of the ten sets. The results from these runs are averaged.

[2] In this domain, most blocks are not at most locations in any given situation so simply answering "false" to an `At` query (which specifies a specific block and location) is correct 83% of the time.

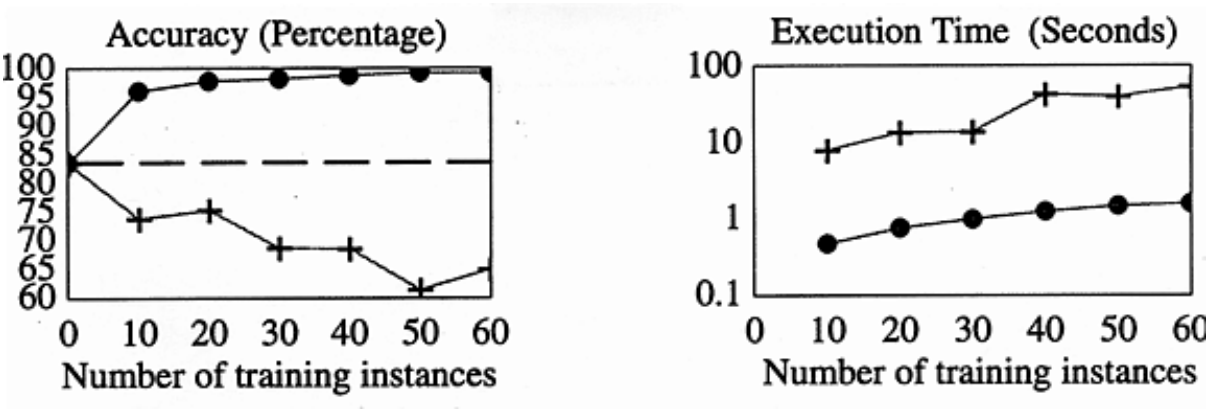

Figure 3.3 Accuracy and execution times for PI and FOIL

Unobstructed blocks move if pushed.
$at(b, l_0, s_0) \wedge add(l, l_0, 1) \wedge eq(s_1, do(right(b), s_0)) \wedge \neg at(b_1, l, s_0) \quad \vee$
Obstructed blocks stay put.
$at(b, l_0, s_0) \wedge eq(s_1, do(right(b), s_0)) \wedge at(b_1, l_1, s_0) \wedge add(l_1, l, 1) \quad \vee$
Blocks stay put if not pushed.
$at(b, l, s_0) \wedge eq(s1, do(a, s_0)) \wedge \neg eq(s_1, do(right(b), s_0)) \rightarrow at(b, l, s_1)$

Figure 3.4 Shortest first-order model of the 1D blocks world domain

training data. For this learning task, however, FOIL is completely unable to grow an adequate representation of the domain. The data available is far too limited to guide FOIL from an empty theory to a set of rules like those shown in Figure 3.4. Because it has no background information telling it which combinations of predicates are *plausible* combinations of predicates, it must indiscriminately try adding all possible predicates as it grows each rule. In this domain there are simply too many alternatives and too little data.

The execution times for these algorithms also highlights the disparity between the space of hypotheses considered by these two approaches. The graph on the right of Figure 3.3 shows the execution times for the two algorithms. As before, the horizontal axis denotes the number of training examples provided. The vertical axis is the log of the

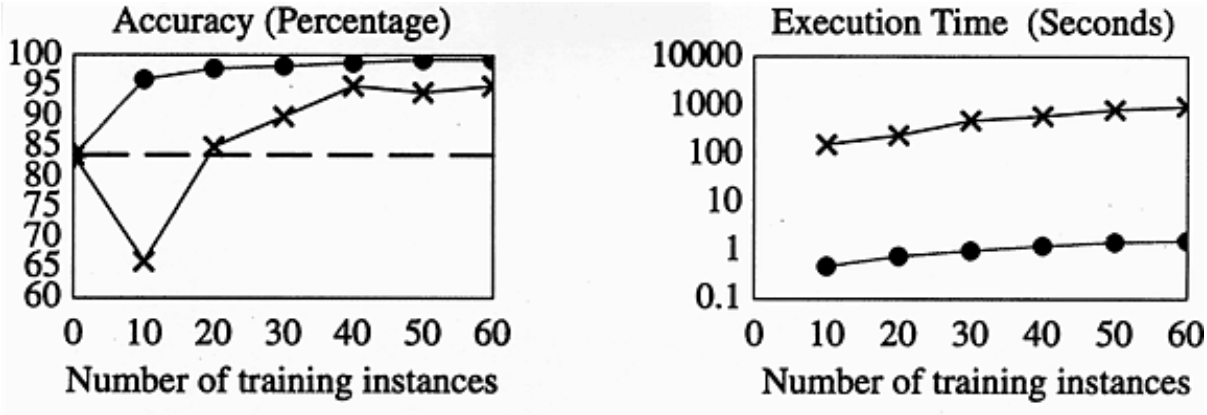

Figure 3.5 Accuracy and execution times for PI and FOCL

number of seconds required for learning.[3] This experiment provides a nice demonstration of a well-accepted point within machine learning: stronger bias results in stronger learning performance. It goes beyond this, however, it shows that learning with a weak bias breaks down very quickly as the complexity of the task increases. If we are to apply learning to assist experts in complex practical domains, use of prior knowledge is not only beneficial, *it is essential.*

3.3 Experiment 2: Specialized Versus Complete Induction

In our second experiment, we measure the comparative advantage of using *specialized* refinement algorithms. Our previous experiment demonstrates the benefit of using prior knowledge as building blocks that are combined to form complex predictive domain models. Recall from Chapter 2, influents describe plausible combinations of predicates (the building blocks); they also describes the class of combination *types* appropriate on

[3]These timings were obtained using a Sparc 10 with 32 megabytes of RAM. Both algorithms were written in LISP and run as compiled Lucid LISP code.

a rule-by-rule basis through an annotation on the rules themselves. In this experiment we measure the benefit of specializing induction based on those annotations.

A number of inductive learning algorithms employ prior knowledge in the form of deductively incomplete or incorrect rules as a bias for induction[34, 35].[4] These approaches, like PI, rely on a set of background rules as a hint for induction. Unlike PI, however, these approaches do not annotate the knowledge with information regarding plausible alternatives appropriate at that point in the model. Instead they have a single, generic procedure for generating hypotheses from the background knowledge.[5] Thus, we can measure the benefit of specializing our use of prior knowledge by comparing PI's performance with these generic rule-based learning algorithms.

The FOCL system, developed by Michael Pazzani, is an example of a system that accepts background knowledge[26]. FOCL is built on top of the FOIL system; like FOIL it attempts to "grow" first-order rules that accurately model the domain. As PI does, it accepts a rule-based background knowledge similar to the one shown in Figure 3.2. Like PI it uses these rules as building-blocks—it attempts to add entire antecedent clauses from this background knowledge to the rules being grown. FOCL is designed as a generic first-order induction system; it makes weaker assumptions about the prior knowledge it employs. In particular, FOCL like FOIL is a *complete* induction algorithm over the space of first-order theories. (An algorithm is complete over some hypothesis space, H, if for each hypothesis, h, in H there is some training set, S, that results in the induction of h.) FOCL uses available prior knowledge as a hint about the domain, but still considers non-prior-knowledge motivated hypotheses, thus it is complete—it is able to learn in

[4] A theory, T, is *incomplete* iff there exists an assertion, a, about the modeled world, W, not entailed by T. A theory is *incorrect* iff there exists an assertion, a, entailed by T that does not hold in the modeled world, W.

[5] This is not always the case; some of these algorithms have global "switches" that affect the types of hypotheses considered. Unlike PI, however, these affect behavior globally—across the entire theory being induced.

spite of *any* possible deficiency in the theory given the right training data. Most of PI's refinement spaces are *not* complete, indeed they obtain their increased performance by being incomplete. They exclude hypotheses that the expert has deemed implausible through the annotations placed on the influents. What comparative benefit does PI obtain from the specialization resulting from these rule annotations? This is the question addressed by the second experiment.

Setup for Experiment Two

We employ the experimental setup used in experiment one for our comparison to FOCL. The FOCL learning algorithm is limited to the induction of function-free horn theories. For this comparison we flattened the theory shown in Figure 3.2. Each functional term in the rules is replaced by a new predicate conjoined to the rule; the functional constant "1" is eliminated through the use of specialized increment and decrement predicates. Like the first experiment, the results of ten-fold cross validation were averaged to produce predictive-accuracy and execution-time learning curves.

Results

Figure 3.5 shows the results of this comparison. As expected, PI's use of specialized refinement algorithms does translate into better learning performance. Like PI, FOCL searches for patterns in the training data consistent with hypotheses generated from its background rules; unlike PI, it also considers hypotheses that have nothing to do with the expert-generated background rules. It is this second class of hypotheses that are responsible for the consistently lower accuracy for any given number of training examples. That said, its interesting to note how much better FOCL performed when compared to FOIL. Since FOCL, like PI, considers combinations of predicates suggested

by the background theory, it is able to partially avoid the local maximums that plagues FOIL's learning.

The execution time comparison of PI and FOCL shown in Figure 3.5 demonstrates remarkably bad performance on the part of FOCL. In fact, its performance is almost an order of magnitude worse than FOIL, and is three orders of magnitude worse than PI. At first glance, this is counter-intuitive, One expects FOCL's execution time and predictive accuracy to both be improved or degraded relative to FOIL's performance depending on whether the rules provided contain useful information regarding the domain being modeled. From that perspective, FOCL's increase in predictive accuracy over FOIL combined with an order of magnitude decrease in execution time seems puzzling. This puzzle is resolved by considering the behavior of the FOCL algorithm. FOCL *augments* the hypotheses considered by FOIL using a set of heuristics that suggest groups of predicates to add when growing predictive rules, but it still considers all refinements considered by FOIL. Thus the FOCL's branching factor is increased, because a group of predicates generally contains more variables than a single predicate, and because the number of different bindings for a newly added group is exponential in the number of variables, this increase in branching factor is significant.

This augmentation behavior is appropriate for FOCL. Its goal is to be a generic first-order learning algorithm. It retains the completeness of FOIL while taking advantage of background rules—a useful and important goal to be sure. There will be cases where such guarantees will be desired and even required. This experiment simply demonstrates the cost associated with such guarantees and the potential benefit of using specialized induction algorithms which make stronger commitments to the background knowledge provided by the expert.

3.4 Analysis of the Experiments

These experiments measured the comparative advantage of the PI algorithm had on a particular learning task. What implications do these specific tests have for PI's behavior in general? To what problems can PI be best applied? How will it scale as problem complexity increases? And how will its performance compare with other learning algorithms over the space of possible of learning tasks?

In order to evaluate the implications of these two experiments we must understand the motivation for the learning task chosen. The 1D blocks theory shown in Figure 3.2 was chosen as a simple, straightforward description of the constraints in the blocks 1D world. The 1D blocks world itself was chosen because it is a situation where the uncertainty in rule combination is effectively captured by the overriding refinement space. PI's overriding refinement algorithm is designed for situations where the expert has a set of "polarity relationships" sufficient for prediction, but does not have a predictive model. The 1D blocks world is a very simple example of this situation. In this sense, the 1D blocks world task is favorable for PI since it does not violate the assumption made by the overriding refinement space algorithm.

The simplicity of this domain, on the other hand, favors both FOIL and FOCL. The flattened theory given to the learners contains exactly those mathematical operators required to express the target concept, but no more. It only mentions the move right operator, and uses a very small, one-dimensional space of possible locations. Extending any of these aspects of the task will result in large (in some cases exponential) increases in hypothesis space size for both FOIL and FOCL. Because PI's overriding refinement is strongly guided by the background theory, these changes will not affect PI's performance. Only changes in background theory complexity affect the hypotheses considered by PI, thereby affecting performance. As the number of additional (irrelevant) predicates are

scaled up, the performance gulf between PI and these algorithms also will widen. Although the induced theory shown in Figure 3.4 is a bit cumbersome, this domain is still trivial when compared with practical modeling tasks. If complete learners like FOIL and FOCL begin to have difficulty in the 1D blocks world shown here, they will be quickly overwhelmed when applied to practical models requiring many dozens of rules.

We have seen that complete first-order learners like FOIL and FOCL cannot be applied to complex learning tasks—their space of hypotheses simply become too unwieldy. We have also seen that PI has the potential for greatly improved performance relative to these other systems, but this is no guarantee of scalability. Indeed PI will also fail to scale up to complex learning tasks unless it is given strong guarantees about the domain through its expert-encoded background knowledge, and those guarantees accurately reflect the domain.

PI facilitates the encoding of two strong types of guarantees: By providing a structured set of refinement spaces, the expert is, guaranteeing that a particular decomposition of the learning task is adequate. By annotating the rules of a particular refinement space with a type, the expert is guaranteeing, that a specialized algorithm (like overriding refinement) is sufficient for the specified portion of the learning task. These guarantees are strong in the sense that there is no bound on the fraction of the complete first-order hypothesis space that they may eliminate. A particular refinement space, described by a fixed set of influents, has a constant complexity even as the problem it is embedded in increases in complexity. Thus PI can be employed in problems of arbitrary complexity, as long as the expert is able to provide strong guarantees (constraints) on those portions of the problem to which PI is being applied. Of course PI's performance will be severely degraded if the guarantees provided by the expert, are violated by the domain. PI's background theory dictates a restricted space of hypotheses considered when learning, if this set does not contain a hypothesis that adequately models the domain, then PI

will not adequately model the domain. PI's strong use of background knowledge is a double-edged sword. It is responsible for PI's improved performance, but is also responsible for degradation in performance when guarantees about the domain are violated by the domain. This is as it should be: PI is intended to be an expert's modeling tool; if the expert makes a guarantee about the domain, the tool should take full advantage of that guarantee. If the expert is uncertain about a particular guarantee, then a weaker constraint should be encoded in the background knowledge in its place.

It is important to note that there is nothing in the PI framework that *forces* an expert to make particular guarantees about the learning domain. If the expert cannot guarantee the polarity relationships necessary for overriding refinement, then that refinement space can be annotated by using more weakly-biased refinement space types like the complete Boolean space refinement. If a particular decomposition is not known, then several alternative decompositions can be described and joined using at their roots the `Enumerative` refinement space. If no decomposition information is available at all, then the entire learning problem can be characterized using a single refinement space. Of course removing all guarantees from the expert encoded background theory really defeats the purpose of the PI learning framework. Its performance improvements stem from these guarantees. If they are removed, then PI's performance will degrade to that of one of the other complete learning algorithms. We believe this is a nice combination of behaviors: PI can take advantage of strong guarantees when they are available, and has the degraded but complete behavior of a FOIL or FOCL systems when such guarantees are not available. The only way its performance is inferior to general purpose learners is if a guarantee is explicitly encoded by the expert, and is violated by the domain.

In this section we focused on the comparative performance and scalability of the PI framework. These experiments demonstrated the potentially significant improvement possible given expert-encoded guarantees about the learning task. They show that these

guarantees are strong enough to allow PI to be applied in complex domains. Empirical study of PI is an important part of our evaluation; it demonstrates the framework's potential.

In order for PI to be a practical modeling tool for these domains, however, the guarantees that underlie PI's performance must *actually* be available in these domains. Whether such strong guarantees can be obtained in practice for any given domain is an important question that is not addressed within this section. The second part of this thesis is a detailed investigation of this question for a particular complex domain: protein structure prediction.

CHAPTER 4

RELATION TO WORK WITHIN ARTIFICIAL INTELLIGENCE

In the previous chapter we made a detailed comparison between the PI approach and two other learning systems. In this chapter we make a much broader comparison relating PI to other approaches within the artificial intelligence community. PI's mission as an extensible domain modeling tool results in several interesting overlaps with areas both inside and outside of machine learning. We break these into three categories: weakly-biased or structure-free learning systems, structurally guided learning, and defeasible inference. Comparisons between this work and work in our application area (Protein Science) is handled in a subsequent chapter. This chapter only considers PI's relation to work within artificial intelligence.

4.1 A Comparison with Structure-Free Induction Systems

We use the terms *structurally-guided* and *structure-free* to distinguish between learners that accept and do not accept some form of structured (rule-like) background knowledge

as input to guide induction. Specifically, a learning system is structurally-guided if it accepts bias that specifies relationships among intermediate domain terms (terms not mentioned in the training data, nor are the learning target). So FOIL is structure-free and both FOCL and PI are structurally-guided. The background theory for structurally-guided learning systems can often be used as a source of strong bias. Thus, typical structurally-guided induction is more strongly biased induction than structure-free induction. In cases where we want to emphasize the difference in bias strength rather than syntactic difference in the type of bias we will use the terms *strongly-bias* and *weakly-biased* learning as synonyms. This distinction between learning systems is an important one; it strongly affects the requirements the systems place on its user, it affects the control the user has over the learning system, and it affects the learning performance possible using the system. The overriding difference between most weakly-biased learning systems, and PI is, of course, the structured background theory required by PI. This added requirement make PI harder to apply to a problem, but as we saw in the last chapter it will in some cases have dramatically poorer performance.

Structure-free learning systems differ from each other in the types of representations (e.g. numeric, horn, etc.) for which they were designed. Because PI is an extensible learning approach; each of its extensions (refinement spaces) may and often do employ their own representation for bias knowledge and for the hypotheses themselves. The framework itself simply assumes that both input (bias) representations and output (target concepts) representations are functional. This allows PI to control the interaction between refinement spaces and allows it to compose the sub-results from these spaces. This functional requirement is also flexible enough to allow PI to embed a variety of specialized representational schemes within different refinement space types. Thus particular refinement spaces will have similarities with particular structure-free learning algorithms. We consider these overlaps in more detail here.

Because of its generality and flexibility, first-order predicate calculus is used as the representational language for a number of learning systems. These include the FOIL system above[36], C4.5[1], and inverse resolutions systems[37]. These systems are most closely related to the overriding refinement space and the complete first-order refinement space. By restricting PI's terms to a two-valued range (TRUE, FALSE), we are able to model horn rules in a straightforward manner using the influent representation. As demonstrated in the previous chapter, the significant difference is the degree to which PI is guided by its background knowledge.

By using only variable-free terms, PI is able to learn in propositional domains as well. Systems like ID3 and C4.5 developed by Ross Quinlan[1] are examples of propositional learning algorithms. For weakly-biased learning systems the difference between a first-order and a propositional representation is significant. The recursive binding structure possible using first-order representations the space of hypotheses considered by a weakly-biased learner is much greater in the first order case. Because PI is primarily constrained by its background theory, the distinction between representational spaces is not significant, thus no refinement spaces are specialized for the propositional case—they don't need to be.

Another large class of learning algorithms focuses on numeric domains where real-valued functions are being modeled. Neural networks[25] that use differentiable weighting functions like the sigmoid are able to model arbitrary differentiable real values functions.[1]. Regression algorithms[38, 39] are other examples of systems that induce real-valued functions. PI's linear polynomial and parametric refinement spaces also induce real-valued functions. Again PI's induction is very constrained by its background knowledge. Both

[1] One might argue that several of the first-order systems could also be placed into the numeric learning category since they are able utilize continuous valued attributes. C4.5 is an example of one such system. We left them in the non-numeric group because they do not learn real valued functions, but simply learn to split continuous attributes and then learn non-numeric functions based on those splits.

refinement spaces are hill climbing over a very constrained space of real-valued functions. This results in quite a difference in learning performance. We can see how dramatic this difference can be by considering the induction of a real-valued function with a single input term. PI's more restricted linear space would require time logarithmic in the magnitude of the error tolerance using binary search, and linear time using its standard hill climbing. A general real-valued function learner could employ an unbounded amount of time to learn a function in the vastly larger space of all continuous real-valued functions. Of course this come as no surprise, but it does serve to highlight the dramatic difference in complexity between the space of some parametric family of functions and the space of all continuous functions.

Obviously when appropriate constrains are available, induction over a parametric space of real-valued functions is a big performance win. Just as important, however, is the use of these restricted spaces when limited amounts of training data are available. In these cases it is better to induce the best approximation from a restricted subspace rather than the random result of induction over the larger space with far too little data to constrain the search. In fact, the biological application considered later in the dissertation contains several examples where the expert does not know that simple linear relationships exist between domain terms, but does expect that a number of input terms have mutually independent effects on some output term and that the relation between input and output term is monotonic. Nonetheless the linear refinement space is employed because the best *linear* function can be found using available data, while less restricted forms of induction cannot. Nonetheless there are certainly domains with practical importance that have sufficient data for less constrained induction. For these domains it would be appropriate to incorporate some less constrained real-valued refinement space into PI's repertoire.

4.2 A Comparison with Other Structurally-Guided Inductive Learners

The learning community has developed a number of structurally-guided induction algorithms. Like PI, these systems enjoy greater flexibility and bias strength as a result of their expert supplied structural bias. The representations and algorithms used by these algorithms vary considerably from system to system within the general class. Perhaps this wide variability is due to the inevitable specialization that occurs when a particular representation and algorithm is chosen. Some representations (e.g. bayes nets) effectively capture probabilistic information while other representations (e.g. first-order predicate calculus) capture complex relationships between unbounded sets of objects better. These orthogonal strengths and weaknesses result in many alternatives approaches none of which is always better than the alternatives. As a result, members of the learning community have investigated a number of different structurally biased approaches to learning in parallel. We explore the relation between PI and several of the better-studied types of structural bias. Because PI attempts to encapsulate both representation and algorithm within each refinement space, it often makes most sense to compare another learning algorithm with an analogous refinement space type rather than with the whole PI framework.

4.2.1 Theory Refinement Systems

Examples of theory refinement systems include Pazzani's FOCL[26], Cohen's A-EBL[40], Mooney's Forte[41] and Either[27] systems. All theory refinement systems take a domain model as input and generate a refined domain model as output. Typically these systems employ a set of transformation operators to hill climb in the space of possible do-

main models. Training data is used to make a local transformation to its current domain model in order to generate a better (more predictive) new domain model. This process is iterated until no further local transformations are needed or are possible. Although theory refinement can be applied to any representation, most systems developed to date employ first-order horn theories as their modeling language. These theory refinement systems primarily differ in the types of transformation operators they employ.

The overall theory refinement approach differs from Plausible Inference in a number of critical ways. The first and perhaps most obvious difference is the way bias is specified in the respective approaches. Theory refinement learners specify bias simply as an initial domain model to be refined. PI, on the other hand, requires input using a separate bias language, this language captures the plausible alternatives for each uncertain term in the domain model. Use of a single language for both input and output for theory refinement systems is a significant advantage for the refinement approaches. If the expert has some guess as to what model may be appropriate for a given domain, this model can simply be supplied to a theory refinement system as input. Further the expert is not required to understand a language for expressing plausible alternatives. Unfortunately the use of an initial domain model as the source of bias has inherent limitations. Because bias is expressed as an initial domain model, there is no room for specifying which parts of the theory are known with high confidence and which parts should be the focus of refinement. Also knowledge about the types of refinements expected by the expert generally cannot be encoded in such an initial model. The expert must simply rely on the learner's use of a very general set of transformation operators. In general the theory refinement approach does not afford the expert much control over the space of theories explored by the algorithm. The expert supplies an initial theory and the system explores the space of syntactically similar domain models.

FOCL shares much with theory refinement approach. It accepts an initial theory using the same language used for output. It also uses a set of transformation operators to construct its output concept. The primary difference is that it does not apply transformations directly to its input theory to obtain an output theory. Instead it uses parts of the input theory in building its output "from scratch." This approach leaves FOCL somewhat less vulnerable to cumbersome and irrelevant knowledge within the input theory. FOCL will simply not add such knowledge to its output theory unless it is justified by training data. This difference does not greatly impact its relation with Plausible Inference. It shares theory refinement's advantage of a single language for input and output, and it shares theory refinement's inability to encode local information about the utility of any particular transformation. In Chapter 3 we saw empirically how dramatic this inability to encode local information can affect learning performance. In fact this difference in performance underscores how these systems are best used. Theory refinement systems are not appropriate in the earlier stages of theory development; they are appropriate only after a fairly good theory has been obtained by other means. They are ideal for later polishing of a theory, since they require nothing beyond the prior theory itself, and can always be applied as a last step with little additional cost to the entire process. PI, on the other hand, is really designed to assist earlier in the theory development process. It requires the expert to encode appropriate alternatives for each of the terms in the domain model, but as a result is capable of exploring theories that are not necessarily centered on some almost-correct initial theory.

4.2.2 Learning Systems That Employ an Explicit Term-Level Bias

While having an unfortunately cumbersome title, these systems are an important part of our comparison with prior work, since these systems are closest in spirit to Plausible Inference. These systems use a separate specialized bias language to encode the set of alternatives explicitly deemed by the expert to be worth of exploration. Like PI these systems are flexible enough to specify alternatives on a term-by-term basis within the theory. Like other structurally-guided systems, these systems employ a great variety of languages to express the expert-encoded background theory (bias).

One of the earliest induction systems to utilize a term-level bias was the work on determinations by Stuart Russell and Ben Grosof[4]. The determination, like the influent, expresses a functional relationship between domain terms. The well-known example of a determination is the relation between a person's country of birth and their native language: "A person's country of birth *determines* that person's native language." This determination asserts the existence of a function mapping countries of birth to native languages. Like the influent, the determination is used to restrict the structure of the target concept. In fact one can view the determination as the weakest form of an influent. It specifies a functional relationship between domain terms, but specifies nothing more. According to the determination, *any* function mapping from its antecedents to its consequent is equally plausible.

Because of this weaker bias, learning with determinations requires that the training data contain instances with every possible combination of antecedent values for each determination in order to completely learn the target concept. This exhaustive form of bias is ideal when little is known beyond the basic structure of the target concept. Nonetheless, the number of training instances required grows quickly as either the size

of the determination grows, or as the range of possible values for domain terms grows. The annotation information associated with the influents in our framework make them more restrictive than the determination. Nonetheless, a determination influent would fit quite naturally into the framework as an additional *type* of influent. Perhaps the best way to relate the two approaches is to say that learning with determinations is equivalent to Plausible Inference over a set of influents with a homogeneous determination type.

William Cohen's more recent work on GRENDEL is another example of a specialized bias language that explicitly describes a set of alternatives as the level of individual domain terms[42]. According to this approach, the expert-specified background knowledge is encoded as a Context Free Grammar. Sentences in the language defined by this grammar correspond to the hypotheses considered by GRENDEL. Each grammar rule specifies a different expansion for some non-terminal symbol. This is analogous to specifying a possible definition for some intermediate term in the domain theory. In fact, one can easily convert any grammar, G, given to GRENDEL into a set of influents, G', that will result in precisely the same space of hypotheses. Each grammar rule is converted into a propositional influent where the antecedents of the grammar rule are converted into a conjunction of terms; the non-terminal in the rule's consequent is used as the propositional consequent of the influent. The enumerative type is used for all influents in G'. The enumerative refinement space assumes that all possible definitions for an intermediate term are explicitly represented within the theory in exactly the same way that all possible expansions of a non-terminal are explicitly present in the context free grammar.

Some Inductive Logic Programming (ILP) systems, like PI, accept a flexible bias describing the structure of the space of hypotheses for consideration. The objective of an ILP system is to induce a first-order theory, that coupled with an available background theory, entails all of the training data. Earlier systems, like GOLEM[43] for example,

were closer to theory refinement systems. A background theory was provided, but little flexibility was available to facilitate the expert's control over which augmentations of that theory would be considered during induction. A more recent ILP system, PROGOL[44], on the other hand does allow great flexibility in specifying the space of hypotheses considered during induction. Like PI, it is able to accept a functional decomposition for the learning task in a way that restricts all learning to the specified decomposition. This decomposition can be specified as a set of determinations, and also using the more mechanism described below.

Like PI, PROGOL can also utilizes a term-level bias. Each intermediate literal specified in PROGOL's initial background domain knowledge can be annotated with *mode* information. This information specifies the arity of the literal, the sub-literals that are possible, and even the variable binding patterns possible among those sub-literals. The primary difference between PI and PROGOL, is PROGOL's use of a single bias language and induction algorithm across all of the intermediate literals. PI, on the other hand, can use a completely different bias representation and algorithm for each node in the problem decomposition. Because PROGOL relies on a single bias language and algorithm, induction at each node can make stronger assumptions about the surrounding nodes in the problem decomposition. Further, the data passed between these nodes is not limited to positive and negative labelings on intermediate literals. PI *is* limited in this way, the only information passed from subspaces is truth values assignments on intermediate terms, and from above `TARGET` truth values are provided.

Because PI make such weak assumptions about the communication between refinement spaces, however, it is able to incorporate very different forms of bias and learning algorithms in a single learning task. PI's ability to learning functions over real-valued parameters, for example, is based on the ability to integrate very different learning algorithms. PROGOL is not designed to handle this type of induction. Its bias language is

very flexible, but it assumes a first-order representation, and does not rely on the continuity assumptions needed for real-valued function approximation, thus it would be very weak or simply not applicable for such domains.

A number of other systems also use or learn domain decompositions similar to the decompositions accepted by PI. Marie desJardins has looked at learning with probabilistic functional decompositions[45]. Some baysian classifiers learn networks that are functional decompositions[46, 47]. Qualitative process graphs like those used in qualitative physics also have a similar functional decomposition[48, 49]. Peter Clark developed a learning system that used a bias specified by a set of rules (he independently entitled as influents)[50].

Like Plausible Inference, these systems simplify the learning task by relying on an expert-generated structure for the concept being induced. Using this expert-specified structure, these systems, like PI, are able to perform strongly biased learning even over complex learning tasks. The primary distinction is that each of these approaches advocate a *single* type of functional relationship, and provide a learning algorithm that takes advantage of that type of relationship. PI, on the other hand, is a framework for combining learning over different types of restricted functional relationships. Nonetheless, its bias language does afford PROGOL the flexibility needed to operate a tool for learning in domains naturally encoded using first-order language.

4.2.3 Plausible Inference as a Form of Defeasible Inference

This unique perspective taken by our work on Plausible Inference allows the framework to serve as a bridge between two important areas within the field of artificial intelligence: induction and defeasible inference. We make the parallel between these two

areas explicit, and then use this analogy to consider how important concepts from each of these areas impacts the other.

For this comparison we use two classical forms of defeasible inference: Ray Reiter's Default Logic[51, 52], and John McCarthy's Circumscriptive method[53, 54]. Default rules used by Reiter's logic fairly easy to describe; they are of the form: $\alpha(x) : \beta(x)/\gamma(x)$. Each of the three terms may be arbitrary first-order expressions. The idea is that for each instantiation of x, $\alpha(x)$ implies $\gamma(x)$ as long as the set of currently derived facts are consistent with $\beta(x)$. An *extension* of a default theory is a largest set of these implications that is internally consistent. Each extension represents one possible minimal set of exceptions to the default rules. The extension is minimal in the sense that adding "one more" default implication leads to internal contradiction. We avoid the formalism that underlies this notion as it is not needed to understand the examples we will employ in this section.

Circumscriptive approaches to defeasible reasoning also rely on this notion of computing a minimal set of exceptions. The simplest of this approaches is known as the *Closed World Assumption*[18]. According to this model of defeasible inference some predicate (usually the "abnormal" or `Ab` predicate) is minimized. Minimization in this case means that every ground instance of the `Ab` predicate is assigned a value (TRUE or FALSE), and the number of TRUE assignments is minimized. The canonical example is: $\forall x\ \texttt{Bird}(x) \wedge \neg\texttt{Ab}(x) \rightarrow \texttt{Fly}(x)$ with assertions about particular instances like: `Bird(Bill)`, `Bird(Fred)` and $\neg$`Fly(Fred)`. All `Ab` ground instances will are set to FALSE except those that would result in contradiction: $\neg\texttt{Ab}(\texttt{Bill}) \wedge \texttt{Ab}(\texttt{Fred})$. Thus `Bill` flies by default.

By comparison to these systems for defeasible inference, Plausible Inference accepts a rule-like domain model with uncertainty in some of the rules. It uses training data to select from the alternatives available for each of the uncertain rules. Defeasible inference

also accepts a set of rule, some of these logical statements that can be "defeated" by other inconsistent inferences. Defeasible inference uses observations (ground terms) to select a set of defeated and non-defeated instances consistent with the observations. Interestingly, this extension to deductive inference (defeasible inference), and the extension to inductive inference (plausible inference) share the same basic inputs and outputs. They both accept an ambiguous rule-like model of the world, and use actual observations from that world to disambiguate this model.

Induction and deduction are often seen as very different forms of inference. Thus it is a bit surprising that extensions to these alternatives seem to overlap to such a large extent. Perhaps most enlightening, however, is the ways in which these two areas differ. The differences between the work done to date in each area may be fruitfully applied to the other area. As noted, both use data to select from a set of alternatives, but the goal, and thus the methods used for selection in the two cases are very different. Inductive learning in general has the goal of predictive generalization—to select from a space of patterns that pattern represented by the data. The emphasis here is to *generalize* patterns from observed examples and use it to predict unobserved examples. Generalization is achieved by *constraining* the induction system to a restricted space of hypotheses; choosing hypotheses from this restricted space consistent with observed examples forces the system into predictions about *un*observed examples. Defeasible inference, on the other hand, has the goal of selecting a logical theory, *minimally different* from the input theory, and is also consistent with the observed data. The emphasis here is on minimal change and consistency. Within this framework, consistency and minimal change are achieved by "defeating" inferences that lead to inconsistency with other inferences until consistency is achieved. These different perspectives and goals result in different algorithms, and in some cases very different performance on similar problems.

What notions from the study of inference can be usefully mapped onto our current understanding of induction? We have identified two central ideas from inference: explicit ontological status and componential semantics. We take each of these in turn, define them, and discuss their relevance in inductive learning.

Both inductive and defeasible inference supply formal characterizations mapping possible input knowledge to derived (output) knowledge. Logical approaches, however, provide a second characterization (semantics) for the knowledge they employ. This semantics provides a link directly to the domain being modeled that is *independent* of processing that might be performed on that knowledge. Inductive approaches, on the other hand, often have no meaning associated with their input except what effects those inputs have on the inductive process (what bias they provide). Having a semantics independent of the inference/learning mechanism itself is important. It allows the user to create, debug, and understand the knowledge given to the system without having to reason about how the system will use this knowledge.

As extreme examples of this, consider setting the topology and initial weights of a neural network, versus encoding the statement "All men are mortal" in first-order predicate calculus. In the first case the knowledge cannot be understood except to reason about how the neural net's learning will be affected by the chosen topology. No semantics for these weights and net topology exist outside of the procedural relationship defined by the learning algorithm itself. The second case, like the first, has a procedural relationship between inputs like "All men are mortal," and the reasoner's output (entailed clauses). In this case, however, a second relationship (model theoretic semantics[18]) exists between this statement and the world being modeled. One can reason directly about this statement's relation to the modeled domain. In principle our understanding of the world is all that is needed to accept or reject the statement above; we do not need to reason about what conclusions it entails. If the statement is "true" in the model

theoretic senses, then we are promised that all conclusions derived by our reasoner will also be true in that same sense—we are freed from thinking about the details of this derivation process. Most learning systems, on the other hand, have no such semantics for their explicit bias (knowledge provided to the learning system). Correctly generating this bias may require considerable knowledge and reasoning regarding the learning algorithm itself. This limits the complexity of the bias knowledge and learning algorithms possible, since they will simply become too cumbersome for the human expert to control.

An important property of conventional deductive semantics is its decomposability. We will see that like the notion of semantics itself, decomposability also has practical consequences regarding an expert's understanding and use of a reasoning system. A semantics is *componential* if the knowledge provided to the reasoning system can be broken into pieces (facts), and these facts have a meaning according to the semantics independent of the other pieces of knowledge. Deductive semantics are componential, by construction—the meaning of a set of sentences is constructed from the meaning of the individual sentences. Inductive systems, on the other hand, typically do not have a componential semantics for their bias. One cannot look at a single rule given to a theory refinement system, for example, and express in some simple way what impact that rule will have on the hypotheses considered by that system. One must look at all of the rules given and the transformation operators; then one can characterize hypotheses considered by the system. So both the existence of a semantics and the decomposability of that semantics improve the usability of a knowledge representation.

Of course the clean theoretical dichotomies expressed here (semantics/no-semantics and decomposable/non-decomposable) are blurred when one looks in detail at the diversity of work done in both areas. First, the elegant model-theoretic semantics developed for deductive inference are not easily extendible to existing models of defeasible inference. They existing models of defeasible inference appeal to deductive semantics, and

claim that their system is like deductive inference "with exceptions." So even though we argue that a formal notion of meaning is important, we acknowledge that even within the knowledge representation community this goal is not always achieved. Second, not all learning systems are without a formal componential semantics. In particular Grosof's determination, $(\mathtt{P}(x) \succ \mathtt{Q}(x))$, can be expressed succinctly using first-order predicate calculus: $\forall x, y \ (\mathtt{P}(x) = \mathtt{P}(y)) \rightarrow (\mathtt{Q}(x) = \mathtt{Q}(y))$. This influent's semantics is also componential. Like the semantics for predicate calculus itself, the meaning of a set of determinations is constructed from the meanings of its components. Determinations are the exception, however, almost all other inductive systems accept bias that has no componential semantics.

Both semantics and decomposability are especially important for the PI framework. PI is intended as a modeling tool; it is designed for domains where most of the inductive constraint resides in the background knowledge, rather than in the training data. We have shown that PI can be applied to complex domains assuming that suitably complex (and constraining) background knowledge is provided. The expert's ability to construct such complex bias depends crucially on that expert's understanding of how that bias language relates to the domain being modeled. All of the issues regarding semantics and decomposability for reasoning systems also apply to PI's use of bias.

We have not attempted to develop a formal semantics for the knowledge used within our framework. For the reasons listed above we are certain such semantics would enhance the usability of the system. An important future direction for our work is the development of such a semantics. Nonetheless, we consider briefly what such a semantics might involve. Several aspects of our framework lead to non-decomposability. The pick function probably cannot be decomposed, and can only be understood procedurally. The interaction between subspaces as each converges is a very non-decomposable phenomenon. Any semantics for the approach would probably have to make strong enough

commitments to ensure that any pick order and any possible interaction still resulted in progress, and so reasoning about the details of these processes would be unneeded. Each of the subspaces themselves would need a semantic defined for their particular type of influent. For several of these influent types, such a semantic is easy to imagine. It would be an existentially quantified logical statement. In the case of the threshold, $\mathtt{P} \prec_{\mathtt{th}} \mathtt{Q}$, for example, the statement $\exists n\ (\mathtt{P} \geq n) \rightarrow \mathtt{Q}$ could be used as its semantic. The functional decomposition inherent in the organization used by the approach also lends itself to a simple componential semantic.

Let us now consider the inverse question: what notions from the study of inductive inference can be fruitfully applied to the area of defeasible inference? Perhaps the biggest contribution is the notion of inductive generalization from observation. Defeasible inference assumes some representation for the pattern of defeated inferences that is sufficiently *flexible* to capture the exceptions observed in the world. One can view a particular assignment of defeat/non-defeat to each rule as a "hypothesis" for a defeasible inference system. The size of the hypothesis space for a defeasible inference system is not a central aspect of these systems, and in some cases is not even a computable aspect of these systems. The primary concern according to these approaches is that enough distinctions are made to allow the system to defeat and not defeat inferences in a way that maintains consistency with all observed data. Induction, on the other hand, relies on an appropriately *restricted* space of hypotheses to obtain generalization from training data—too much flexibility here results in too little generalization. More alternatives are *not* necessarily a good thing according to the learning perspective; successful induction relies on a careful trading of the gains in flexibility against the loss in inductive generalization. This tradeoff is well understood by the learning community, and is central in any application of inductive inference. Because of the differing perspective, generalization is

typically not considered when applying defeasible inference, yet we will show that it has a significant impact on the types of inferences drawn as well.

We present a simple intuitive domain where defeasible inference is appropriate, and use this to study the effects of generalization. In this domain the reasoning systems attempt to predict which pieces of clothing will and will not be comfortable. Figure 4.1 shows a circumscriptive theory for this domain. Each of the rules provides a defeatable "reason" for comfort (or discomfort) afforded the wearer of the garment. A stream of observations are then used to resolve the pattern of defeated inferences. Figure 4.2 shows a possible set of such observations. Given any fixed set of observations the theory can be circumscribed in a way that is consistent with and entails that fixed set of observations. The circumscribed theory would have sentences like: $Flannel(x) \wedge \neg Ab_1(x) \rightarrow ComfyClothing(x)$, $Ab_1(Washed)$, $Ab_1(Washed)$, $\neg Ab_1(Favorite)$, $\neg Ab_1$, ... Notice this encoding is completely incapable of generalizing from observation that *all* wet flannels will be uncomfortable. One might argue that we have no justification for concluding any universal quantified statement of the set of wet flannels, and that inference systems have no business indulging in such speculation. Perhaps this argument holds for pure deductive inference, but once implications are made by "default" we are already drawing conclusions that are not strictly justified by the theory. In fact this is the whole point of defeasible inference. The important thing is to make reasonable generalizations given the data. In this domain a bias toward combinations of the defeasible rules seems better than a bias toward minimal change. Undoubtedly their are other domains where a bias towards minimal change is more appropriate. The point is not to argue that one bias is better than the other, but to realize that the bias resulting from any partial model of a domain is an important part of its appropriateness. Adding flexibility to a model without considering the loss in inductive generalization will yield weak models of that domain.

This realization also implies that we should consider what control a modeling framework gives the modeling expert. In this example it is hard to imagine how one would construct a defeasible theory that could resolve the general relationships between the defeasible rules without explicitly encoding all possible relationships as a large set of many different alternative rules. Its difficult to appropriately control the bias for this problem using circumscriptive encodings of the problem. This is not surprising, since this formalism was developed without explicit consideration of bias and generalization. There is no theoretical reason, however, why more controllable defeasible inference systems could not be constructed. In fact, Plausible Inference offers a bias that is ideal for this domain. Each of the rules could be expressed as an overriding influent (as in the blocks world example in Chapter 3). Such a theory could easily yield $Flannel(x) \wedge \neg Wet(x) \rightarrow ComfyClothing(x)$ using only a few observations.

We believe the relationship between induction and inference made explicit in this framework is an important contribution to both areas of study. This bridge has deep implications for both areas bridged. Both inductive bias and deductive knowledge serve as compact representations of a large set of facts about some complex domain. In both cases complexity requires that the expert be able to understand a piece of these compact models and evaluate how well it captures the world being modeled. Bias should have a componential notion of meaning with respect to the world being modeled. Like induction, defeasible inference strives to make increasingly accurate predictions about the world given an increasing set of observations. The constraint and flexibility inherent in a defeasible characterization is critical in evaluating the appropriateness of the encoding. And the control over bias a representation language affords its user is an important criteria to use in evaluating that representation.

1. $Wet(x) \wedge \neg Ab_1(x) \rightarrow \neg ComfyClothing(x)$ $\quad OR$
   $Wet(x) \wedge \neg Ab_1 \rightarrow \neg ComfyClothing(x)$
2. $Flannel(x) \rightarrow ComfyClothing(x)$
3. $DivingSuit(x) \rightarrow ComfyClothing(x)$
4. $IncorrectSize(x) \rightarrow \neg ComfyClothing(x)$
   ...

Figure 4.1 A theory about the garment domain

1.	$ComfyClothing(Favorite)$,	$\neg Wet(Favorite)$,	$Flannel(Favorite)$
2.	$\neg ComfyClothing(Washed)$,	$Wet(Washed)$,	$Flannel(Washed)$
3.	$\neg ComfyClothing(BurlapBag)$,	$\neg Wet(BurlapBag)$,	$\neg Flannel(BurlapBag)$
4.	$\neg ComfyClothing(Washed)$,	$Wet(Washed)$,	$Flannel(Washed)$

...

Figure 4.2 Assertions about observed garments

CHAPTER 5

THE PROBLEM OF HIDDEN HOMOLOGY MODELING

We have presented PI, as an inductive framework that facilitates the encoding of domain expertise as strong bias. The approach is predicated on the belief that domain expertise is fraught with many instantiations of a small set of recurring "ambiguity" types (e.g., the set of factors are known, but the correct Boolean combination is not known, or a parameter's range is known but its specific value is not known). Our approach advocates the use of specialized induction for each type of ambiguity as encoded by the refinement spaces in the background knowledge. Successful application in any particular domain, however, requires that the ambiguities that occur in human expertise about that domain are easily expressed as plausible refinement spaces. We motivated our approach noting that certain classes of ambiguities naturally arise when expressing domain knowledge, and experimentally measured significant performance improvements occurred when specialized learning was used that took advantage of these expert encoded ambiguity types (refinement spaces). Detailed examination of a practical modeling task provides a complement to the empirical and analytical evaluations performed in the last chapter.

Given the effort required to understand and effectively apply PI to any complex modeling task, we should carefully consider what attributes the ideal task should possess. As discussed above, the domain should have a wealth of background knowledge, and the application of that knowledge for any specific prediction problem should be poorly understood (otherwise learning is not needed). The strength of our analysis is further increased if: (1) the domain is complex enough that neither inference nor conventional induction can be effectively applied to it, and (2) the domain is a well-studied problem with practical importance outside the field of artificial intelligence. Each of these properties are exemplified by a problem from computational biology—protein fold class prediction. The remainder of this chapter is devoted to detailed description of this problem, and the application of plausible inference to it.

5.1 Background in Protein Science

All life uses a class of molecules known as proteins as its primary building block. Proteins are composed of a linear chain of amino acids. Each amino acid has two standard interfaces called its nitrogen and carbon terminals. The carbon terminal of any amino acid can be connected to the nitrogen terminal of any other amino acid. Thus arbitrary strings of amino acids are possible. Remarkably, all organisms are based on proteins constructed from a common set of twenty amino acids, called the *natural* or *naturally occurring* amino acids. Figure 5.1 shows the atomic makeup of the natural amino acids and the single-letter abbreviation that biologists have assigned to each acid. These abbreviations map all possible proteins (chains of amino acids) to unique character strings where each letter in the string corresponds to one of the twenty amino acids. This string

of amino acid identifiers is referred to as the protein's *primary sequence*, and it completely describes the chemical makeup of the protein.[1]

The physical, chemical, and electrical properties of the amino acids in a protein cause it to "fold up" into a three-dimensional configuration, known as the protein's *tertiary structure*. This three-dimensional configuration, is specified by giving the relative 3D-coordinates of every atom within every amino acid within the protein. Figure 5.2 depicts a cartoon-like picture of the tertiary structure of one of four sub-units that makeup human hemoglobin. Hemoglobin's function is realized by its specific 3D-shape which allows it to form two histidine bonds to a heme-group containing an iron atom used in oxygen transport in the blood. The two histidine amino acids are show as ball and stick pentagons pointing into the heme-group which is shown as a larger ball and stick group of atoms. Note it is the precise placement of the two histidines that allow this protein to bond to the heme group, and thus results in its very specialized functionality. Indeed, the diverse and specialized properties exhibited by each protein result solely from their respective tertiary structure. As one might imagine, the mapping from structure to function makes protein structure very important in the study and control of biological systems.

Just as a protein's tertiary structure determines its function, its primary sequence in turn determines its tertiary structure. This surprising functional relationship has been shown to hold experimentally by denaturing (e.g., heating) proteins until their structure is destroyed, but their primary sequence is retained. When allowed to refold, these proteins spontaneously revert back to their original structure[55]. Thus, in principle, the

[1]There are cases where the chemical makeup of a protein is not completely described by its primary sequence. In some cases particular acids within a protein are modified by enzymatic action; in other cases an amino acid chain will bind with other compounds (heterogeneous atoms) to form the protein. Both cases can still be handled by the approach we will present, but for the purposes of discussion we will ignore these complexities in our analysis here.

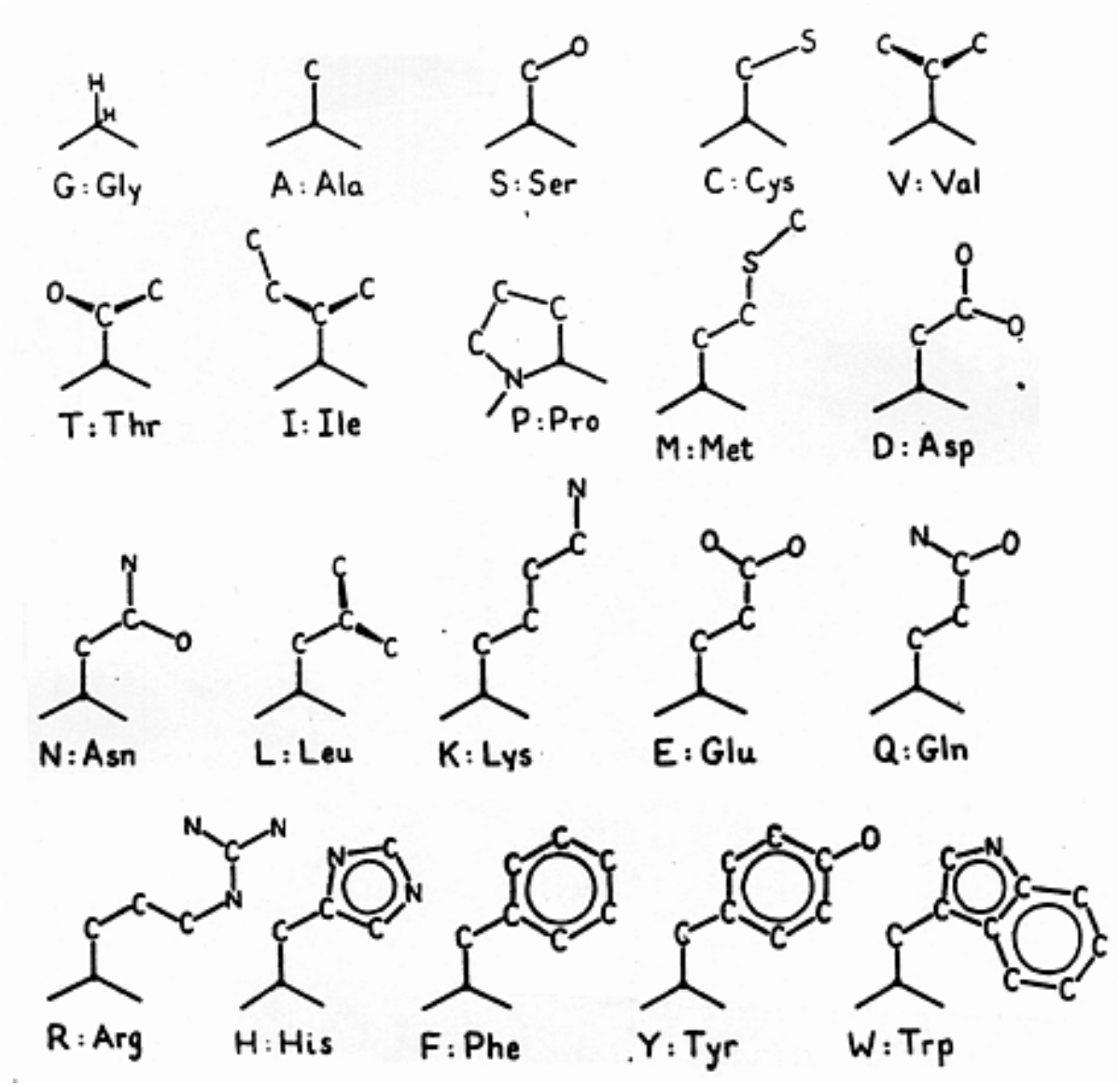

Figure 5.1 Atomic makeup of the twenty naturally occurring amino acids.

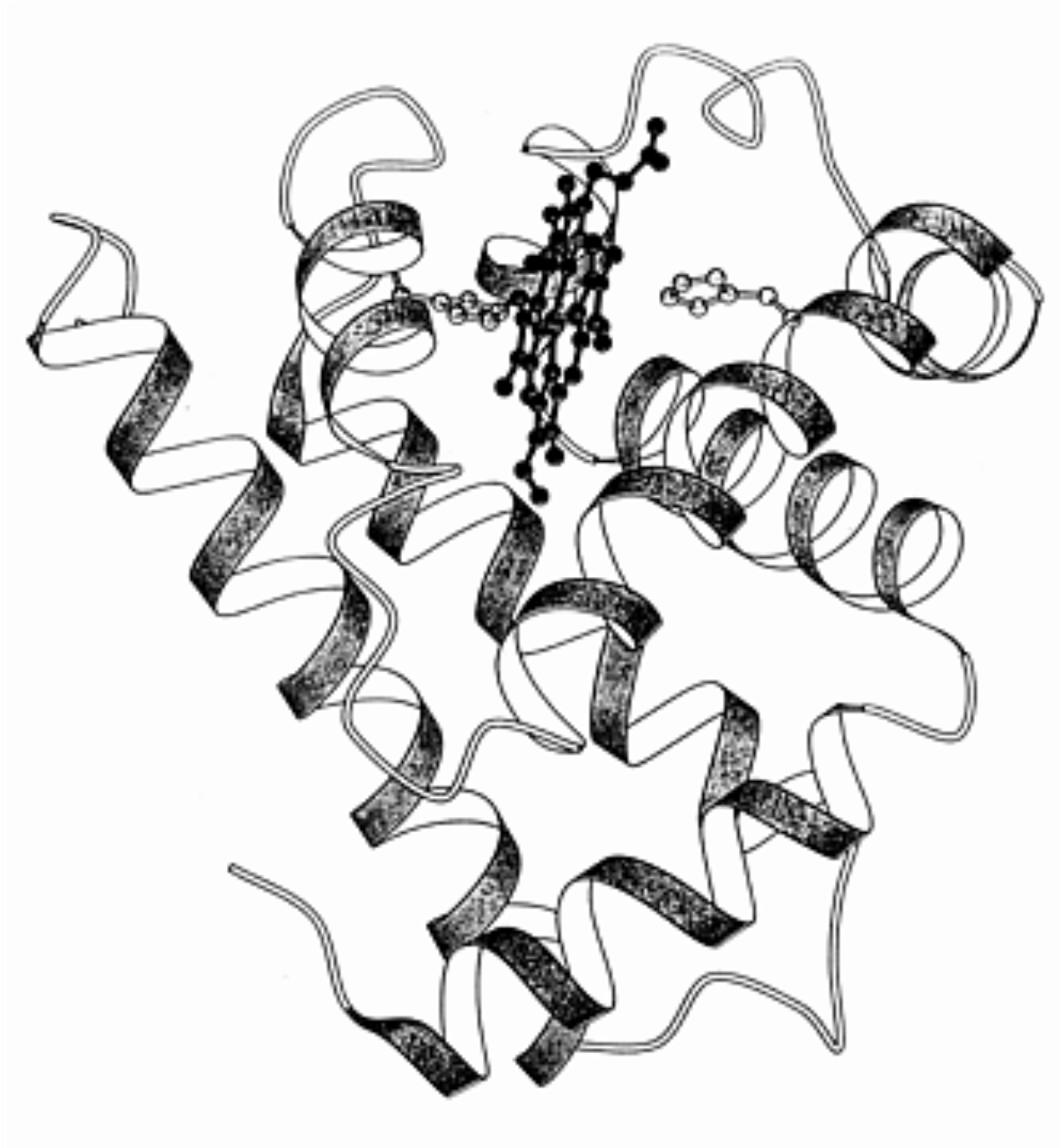

Figure 5.2 Cartoon depiction of one of four components in human hemoglobin.

primary sequence of a protein completely determines both its structure and its function; in practice, however, we will see modeling this functional relationship is quite challenging.

5.2 The Problem of Protein Fold Class Prediction

The mapping from protein sequence *to* protein structure *to* protein function, leads one to consider the inverse mappings. Does each function require a particular structure? Does each structure result from only one primary sequence? The answer to both questions is no. Figure 5.3 shows the primary sequence of Mandelate Racemase matched against the sequence for Muconate Lactonizing Enzyme. Both tertiary structures are very similar—they both are used in the process of food digestion. Proteins found in nature seem to fall into equivalence classes like this with similar structures; further, those with similar structures may have common functions. Biologists refer to these generalized 3D protein shapes as *fold classes*; knowing a protein's fold class tells one a lot about its potential function within a biological system.

As we see in Figure 5.3, the mapping back to primary sequence is also not conserved. The sequences shown have different amino acids in many of the residue positions, and yet fold into very similar structures. A single position within the protein is referred to as a residue. The similarity of two sequences is called their homology. In the figure the sequences are *aligned*—gaps are inserted (shown as dashes) so that the greatest possible number of acids are paired with acids of the same type (denoted as shaded regions). The percentage of identical matches on the shorter sequence is referred to as the *percent sequence homology* between two sequences. In the example above, Mandelate Racemase and Muconate Lactonizing Enzyme have 26% sequence homology.

Two proteins with completely unrelated structures will nonetheless have some level of sequence homology purely by chance. It has been shown that unrelated sequences will

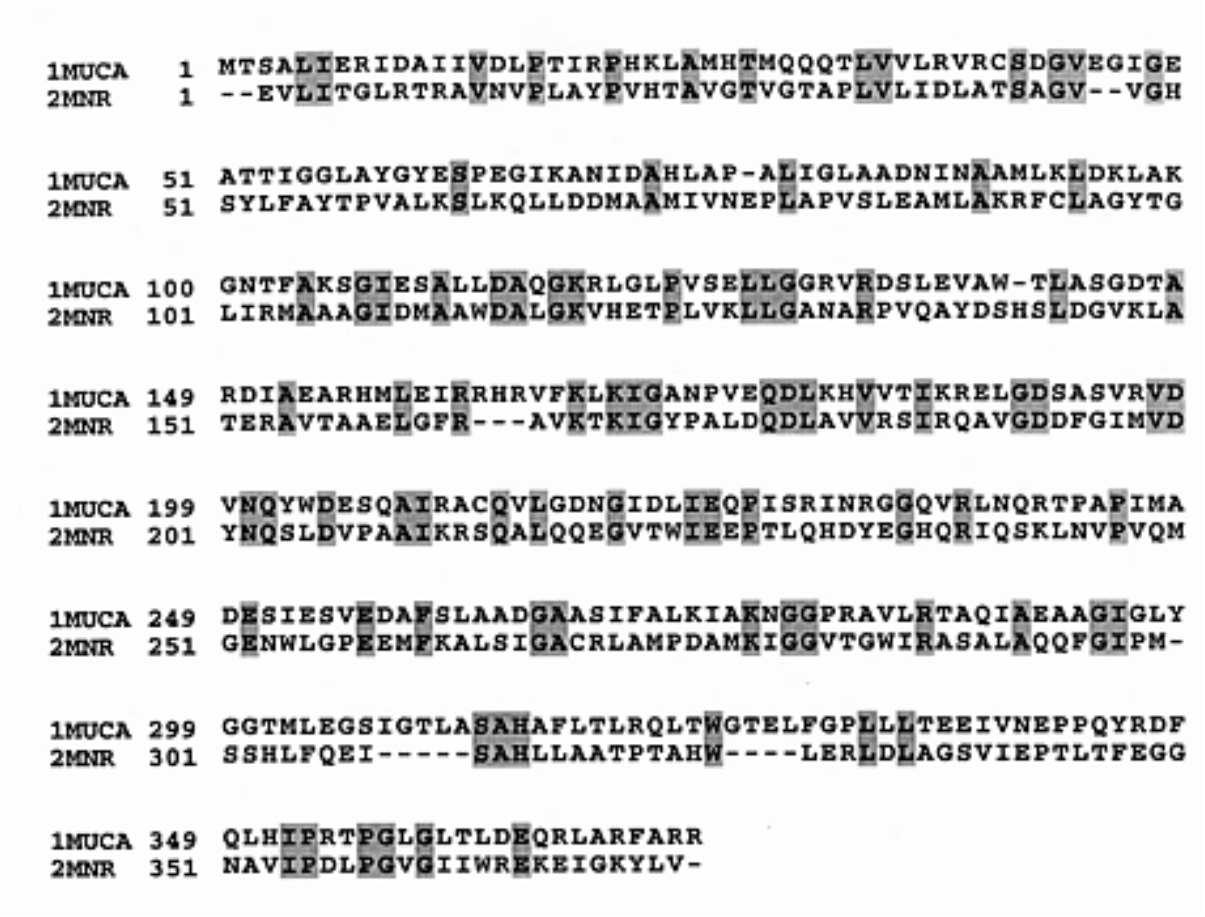

Figure 5.3 Alignment of Mandelate Racemase (2mnr) and Muconate Lactonizing Enzyme (1muc).

Input: A set of *known* sequences (with 3D-structural and fold class information).
A set of *unknown* sequences (with only primary sequence information).
Output: A fold class label for each unknown sequence.

Figure 5.4 Definition of the HHM problem.

have about 15-20% sequence homology on average[56]. Researchers agree that between 25% and 35% homology is needed before one can assume the sequences belong to the same fold. In a number of cases proteins with sequence homologies well below this threshold, however, still turn out to have very similar structures and functions. Proteins in the TIM barrel family, for example, have as little as 10% sequence homology yet share a common tertiary structure. Such proteins are said to have a *hidden* homology, and the problem of finding these hidden relationships is called the problem of *hidden homology modeling* or HHM. It is this problem of modeling these "hidden" homologies that PI has been applied to. For concreteness and simplicity we use the definition shown in Figure 5.4 as our characterization of the HHM problem.

5.3 Previous Work in Hidden Homology Modeling

Because of its central importance, the problem of HHM has received considerable attention by the research community. In our analysis of HHM we do not attempt to survey this work, rather we focus on approaches that rely on domain expertise similar to that used in our application of PI to the problem.

A dynamic-programming algorithm for computing optimal alignments (like the one shown in Figure 5.3) serves as the basis for much of the work in HHM, and our work as well[57]. This algorithm places gaps in its input sequences in order to optimize the alignment score. The alignment score is derived by applying some similarity metric to

all pairs of amino acids in a potential alignment. Two proteins are assumed to be in the same fold class if they can be aligned with high similarity scores between their constituent amino acids. Thus a protein with an unknown fold class can be labeled by noting the fold class of a "similar" protein with a known fold class.

Substitution-Table Methods

The challenge with this approach is to find a biologically motivated definition for amino acid similarity on which to base our alignments. One of the earliest approaches was the use of a substitution table[58]. According to this approach, one constructs a matrix of observed substitutions between amino acids in proteins that are known to be aligned. A similarity matrix is derived from this observed substitution matrix. Two amino acid types that often occur in the same place within matching proteins are given a high similarity score, and those that rarely occur in matching locations are given a low score. This method has met with some success at predicting fold class, in fact this success and its simplicity have lead to its widespread use in aligning protein sequences.

Context-Based Substitution Tables

The chief limitation of the substitution table methods are their reliance on a single similarity number to represent the relationship between these complex objects (amino acids). Consider serine and threonine, for example, are similar in size and electric charge (see Figure 5.1), so in many circumstances they have a high similarity score. Within a corkscrew-like structure (called an α-helix), they behave differently. Serine's particular shape allows it to compete for bonds required to form the α-helix there by destabilizing it. Threonine, while similar in many contexts, does not share this unique shape, so in the α-helix context they behave very differently. Lumping the α-helix context in with all other situations for an amino acid causes strong relationships like the ones described

above to be blurred. In some cases Serine and Threonine are very good matches; in others they are bad matches. By representing these opposing cases by a single "average" similarity we loose that information. At best we are able only to capture weak general trends that are constant over many contexts.

A number of researchers[59, 60, 61] have tried to overcome this blurring by partitioning amino acid positions into groups with a similar context (e.g., "in α-helix"). In this way Serine/Threonine can have one similarity score outside of a helix context and a different score within the helix. The challenge with this type of an approach is to choose the "right" definition of context—one that splits the training data in a way that minimizes blurring, but does not split the data so much that statistical significance is lost. Although existing research was carried out using fixed definitions of context, a natural application of machine learning is to induce the best definition of context from some space of alternatives.

Position-Specific Similarity Metrics

Although the context-based approach increases the specificity of its similarity metric, it still must summarize the many different types of interactions that occur within one context into a single similarity score. The same blurring effect is still present, just on a smaller scale. One approach that overcomes this lack of specificity is to build a specialized similarity metric for each aligned position within the proteins of a single fold class. Such position specific similarity metrics are called *1D-maps* of the protein, since a different set of similarity scores are associated with each amino-acid position within a given fold[62, 63, 64, 65]. This scheme permits the similarity metric to take into account very specific details about that particular position within a fold. For example, the hemoglobin in Figure 5.2 uses two Histidines to bind the heme-group in place. A hemoglobin is simply not hemoglobin unless it can bind a heme-group. The similarity metric for these histidine

positions could reflect this by not allowing Histidine substitutions *in those two particular amino acid positions*. While allowing Histidine substitutions elsewhere, the challenge one faces in building a 1D-map for a particular fold class is the extremely limited data available. Many fold classes have no more than a dozen or so members with known tertiary structure. This means any scoring metric for a particular amino acid position will be based on a very limited set of examples. Thus, using learning to induce a position specific scoring metric for any particular fold class will require an extremely strong bias since the amount of data available is very small. Fortunately, fixed configuration allows biologist's models of amino acid behavior to be directly applied. Thus an induction algorithm, like PI, that can take advantage of such partial models is a good match for this approach.

5.4 Influent Encodings For HHM

There are many questions we seek to address through our application of Plausible Inference. Probably the most important is: how well does our influent vocabulary capture what is known and not known by experts in the task domain? What types of uncertainty in the expert's current understanding can be encoded using the refinement space types described in Chapter 2, and resolved through automated refinement? We answer these questions by looking how knowledge from the HHM domain fits within the framework. Figure 5.5 contains the influent theory used in experiment three described in the next chapter. We will not discuss this theory as a whole here; instead, we will draw examples from this theory showing several general forms of uncertainty that occur again and again across this domain.

Uncertainty In Data Abstraction

$PredictedFoldClass(seq_u) \leftarrow$ (1)
$\quad FoldClass(Max(KnownSeqs, \lambda(x).AlignedScore(x, seq_u)))$
$AlignedScore(seq_k, seq_u) \leftarrow$ (2)
$\quad AlignAlg(seq_k, seq_u, ResSimFn, GapCostFn)/Min(Len(seq_k), Len(seq_u))$

$GapCostFn(GapLength) \prec GOP + GapLength * GEP$ (3)
$GapCostFn(GapLength) \prec_{\texttt{para}} \langle GAP, GOP \rangle \quad GOP + GapLength * GEP$ (4)
$GOP \prec_{\texttt{range}} \{10, 15, 20, 25, 30, 40, 50, 70, 100, 200, 300, 500\}$ (5)
$GEP \prec_{\texttt{range}} \{3, 5, 7, 10, 20, 40\}$ (6)

$ResSimFn(R_k, R_u) \prec$ if $(Id(R_k) = Id(R_u))$ then 100 else 0 (7)
$ResSimFn(R_k, R_u) \prec$ (8)
$\quad ArrayLookup(CtxCostMatrix(ContextFn),\ ContextFn(R_k),\ Id(R_k),\ Id(R_u))$

$CtxCostMatrix(ContextFn) \leftarrow$ (9)
$\quad DayhoffCostFnComputation(APM(TrainingData, ContextFn))$
$ContextFn(res) \prec_{\texttt{th}} \langle ExposureCutoff \rangle \quad SolventExposure(res)$ (10)
$\quad ExposureCutoff \prec_{\texttt{range}} [0 - 150]$ (11)
$ContextFn(res) \prec Str0(res)$ (12)
$ContextFn(res) \prec Chirality(res)$ (13)
$ContextFn(res) \prec StructureIndex(res)$ (14)
$\quad \vdots$

variables:

seq_k, seq_u	;; Protein Sequences with known/unknown tertiary structure.
R_k, R_u, res	;; Amino acids with known/unknown structures.

Procedural Attachments:

$Max(best, list, fn)$	;; Maximal element of a list
$DayhoffCostFnComputation(APMtable)$	;; Dayhoff APM table transform
$APM(MultiAlignments, ContextFn)$	;; Computes point mutation counts
$AlignAlg(seq_k, seq_u, ResSimFn, GapCostFn)$	;; Needleman optimal string align
$Id(res)$	;; Returns amino acid type number

Figure 5.5 Plausible characterization of experiment three

In the characterization of homology modeling given in Figure 5.4, each protein represents a single training instance for the learning task. Each protein is characterized by a pdb[2] data file whose size is between 20 and 60 kilobytes. No learning algorithm can find useful patterns directly from such raw data. We must appeal to experts for intermediate terms computed from this raw data, only then is it possible for learning to make progress by operating on these intermediate terms. We have found the process of abstraction itself can often benefit from automated refinement. For example, consider the notion of amino acid exposure to the surrounding solvent outside of the protein. Biologist's know that an exposed acid's interaction with solvent contributes significantly to the protein's overall energy state, thus an acid's energy contribution is strongly dependant on its solvent exposure. The intermediate binary term $\texttt{Exposed}(AminoAcid)$ is derived from the locations of each atom within each acid within a folded protein. In practice such an intermediate term could never be induced; we must rely on expert-generated algorithms for estimating the square angstroms of the acids surface accessible to solvent molecules. How such a term is computed is a complex, but well-understood process. The volumes required by various atoms are well understood. So determining solvent exposure can be reduced to a geometric reasoning problem about where solvent molecules can fit into the protein. The point at which the qualitative behavior associated with exposed acids asserts itself, however, is not known. This small part of the total model is ideal for automated refinement. The follow pair of influents could be used to encode this uncertainty. The first influent is the procedural attachment used to compute the square angstroms of solvent exposure:

$$\texttt{ExposedSurfaceArea}(res) \leftarrow$$
$$\texttt{TotalSurface}(res) - \texttt{ComputeBuriedSurface}(res, \texttt{RawPDB}(res))$$

The `RawPDB` simply gives the location of every atom in every amino acid. The number of

[2] "Pdb file" refers to the protein data files maintained at the Brookhaven Protein Data Bank

square angstroms of buried surface area for each amino acid is computed by the procedural attachment `ComputeBuriedSurface`, which is some very complex function over the raw data.

The second influent uses the first to split all amino acids into two categories: exposed and buried (not exposed).

$$\texttt{Exposed} \prec_{\texttt{ovr}} [0..150]\ \texttt{ExposedSurfaceArea}$$

Now the `Exposed` predicate can be used to build two context-specific substitution tables (buried and exposed) in a manner similar to what we saw in the previous work. The difference here is that PI can efficiently optimize the threshold parameter, which in the previous work had simply been set using intuition.

This is a concrete example of an ubiquitous task in complex domains. First the raw (high-dimensional) observed data must be abstracted into lower-dimensional, intermediate terms. This process must be well understood since the intermediate term is generally too complex to induce purely from data. Nonetheless this abstraction process, though well understood, may still have parameters that would profit from inductive refinement. Such cases are ideal candidates for Plausible Inference.

Selecting Among Competing Alternatives

The state of the art in protein modeling is best described as a set of competing partial models rather than a unified theory into which all existing expertise can fit. Many current protein modeling techniques will rely solely on only a single aspect (volumetric, geometric, chemical, etc) of protein science, since no unified theory exists[66, 67, 68, 69]. We believe that complex domains in general move through this phase where many loosely connected partial models have been found, but no single model captures what is known. Further, it seems that this period of transition is the period where automated refinement has its greatest opportunity for impact. Prior to this point learning is too unconstrained,

and subsequent to it learning has less potential for impact since a unified model of the domain has already been identified.

A common type of uncertainty when competing partial theories exist is the enumerated set of alternatives. These partial models of the domain will, in some cases, suggest different expansions for a particular intermediate term. Often these alternatives are all generalizations that have basis is observation. The task is not to choose the correct characterization of the domain, but to choose the characterization that best models that particular aspect relevant to that learning task at hand. The expert often knows the general trends that operate in the domain, but does not know to what extent each trend applies to a specific modeling task. That is where automated refinement can be fruitfully applied. Rules #10, #11, ... are an example of enumerative in Figure 5.5. These influents describe different alternatives for computing amino acid similarity. Many of these methods involve computing context-dependant substitution tables like those discussed above. Some of these context-dependant tables partition the amino acids based on computations involving the atom contact patterns found between the acid and its neighbors, other tables use properties like solvent exposure. None of these definitions are "wrong" but it is not clear which best serves as a cost function for optimal string alignment. That is determined by first optimizing each alternative, and then testing the predictive accuracy of each alternative.

Monotonic Refinement Spaces

These same competing partial theories described in the previous section also result in another common type of uncertainty. The Protein domain has several intermediate terms like "delta potential energy" that are predicted by a number of the partial theories. Each of these theories explain how the total potential energy for the protein is increased or decreased as the result of some properties of a group of the amino acids. Some theories

look at how far atom bond lengths and bond angles are from the most favorable position. Other models look at the total number of bonds able to form in a given conformation. Each of these sub-theories looks at completely different aspects of the amino acid, yet experts would agree that both are relevant to determining energy potentials. The best model of an amino-acid's energy contribution probably takes both into account. But how should they be combined? Neither theory discusses its relation with the other. The interaction between these is simply not *known* yet. Nonetheless we expect that both of these sub-energy terms to be positively related to the total difference in potential energy. It further greatly simplifies learning if we can assume that the contribution from each of these terms are *independent*. Though we can rarely guarantee such independence, it is often a useful assumption to make. If the output of one sub-theory is potentially relevant to another sub-theory then the expert should have constructed the first theory in a way that makes it depend (or possibly depends) on the second. In the case of steric and electrostatic contributions we expect them to be relatively independent, so encoding them as a set of linearly related factors is ideal. Each sub-model can be refined independently and the appropriate weight for each can be efficiently induced. Decomposing part of a complex model into a set of linear factors reduces learning complexity to such an extent that it is often the only way the model can be refined given the limited data available.

Injecting Expertise without Learning

As we said in the introduction, PI attempts to "fill in" gaps in expertise currently in use by domain experts. Thus much of the theory generated by PI is taken directly from expertise encoded for the algorithm. There are two ways to encode such expertise. The first is to use a procedural attachment. In this case arbitrary (functional) code is written in an available language (we use LISP) and is invoked within the theory by referring to the name of the procedure within the influent theory. (PI knows to look for a proce-

dural attachment since it has no influent with that predicate as a consequent, and it is not observable.) A good example of this is the `ComputeBuriedSurface` predicate shown above. This term is computed by a very complex (but well-understood geometric reasoning algorithm[70]). The second method for directly including expertise to be used without learning is the definitional influent type ($\leftarrow$). This influent provides an expansion for an intermediate term which can simply be employed to directly compute the term from other terms without requiring any learning. The second influent shown in Figure 5.5 is an example of an definitional influent that computes a normalized alignment score:

$$AlignedScore(seq_k, seq_u) \leftarrow$$
$$AlignAlg(seq_k, seq_u, ResCostFn, GapCostFn)/Min(Len(seq_k), Len(seq_u))$$

Not all the knowledge associated with HHM fit nicely into the PI framework. Still other knowledge seemed to fit into the framework, but would have required the introduction of a new type of refinement space. Chapter 7 discuses some of the limitations of the approach including knowledge that doesn't seem to fit well into an influent theory. That said, we have generally found the repeated use of the same refinement spaces sufficed for encoding a large range of biological expertise in HHM. Each of these groups represents a recurring form of uncertainty in the protein domain. In the next chapter we will discuss the implementation of Plausible Inference used in our application to HMM, and show an experiment in the domain based on the knowledge shown above.

CHAPTER 6

APPLICATION OF PI TO THE PROBLEM OF HIDDEN HOMOLOGY MODELING

The implementation of Plausible Inference described in this Chapter predates the framework described in previous chapters. The constituent refinement space algorithms are implemented as discussed in Chapter 2, but the top-level control described in Chapter 2 is not used. The top-level control used in the experiments we have run, however, are consistent with the framework described in Chapter 2. In the next section we give details about the particular implementation used in our application to HHM.

6.1 Building a Predictive Model of Protein Fold Class

As with any real-world learning task, much effort must be expended on non-learning aspects of the problem before automated learning is even possible. Indeed PI designed for such domains—domains so complex that only a fraction of the entire modeling task can be solved through induction. In order to see the role that learning plays in this practical modeling application we briefly describe the entire implementation, including the domain-specific machinery necessary for this application. Much of what we describe in this subsection is not a part of Plausible Inference, but this is the point, since the

tradeoffs made by our approach are predicated on the assumption that automated model refinement is only responsible for a small fraction of the larger application being constructed.

The basis for all of our experiments is a quadratic-time, optimal string alignment algorithm[57]. This algorithm computes a similarity score for pairs of proteins based on a similarity metric for individual amino-acids. We use this algorithm in turn to compute the pairwise similarity between all proteins in our training data. This $AlignAllPairs$ computation allows us to assign fold class names to unknown proteins as required in our definition of the HHM problem (see Figure 5.4). $AlignAllPairs$ is also the way we test the performance of an induced amino acid similarity function. Thus, this procedure is repeated many times in the course of any given experiment. Because of the large time complexity of the $AlignAllPairs$ procedure, we have implemented it in parallel, in optimized C, on a 512 node CM5 supercomputer. The limited memory on each CM5 processing node forces us to adopt a linear space variant of the basic Needleman-Wunch algorithm[71].

A LISP master process controls the induction. $AlignAllPairs$ requests are transported to a server process on the supercomputer which in turn dynamically partitions these requests for parallel processing on the nodes. When they become available, the results of these computations are merged and sent back to the LISP process. To avoid serializing on the master LISP process, we have implemented a simple futures mechanism to mediate communication with the supercomputer.

Figure 6.1 shows a breakdown of the domain specific modules needed for our application to HHM. Solid lines denote sub-components, and dotted lines denote the flow of data. Data generally flows from left to right in the diagram. Preprocessing modules download and parse protein data files from three sources on the internet into a common attribute/value format for learning. A number of algorithms like the DSSP local struc-

ture analysis package are used to compute higher level terms from the raw data[70]. We have also coded a number of our own intermediate attributes based on geometric patterns in the raw protein data. The attribute Lp3n3, for example, classifies each amino acid in to one of nine categories by looking at the number of polar and non-polar contacts made by the amino acid. All of these attributes and values are transported to the processing node where various combinations of them are used as amino acid match cost functions by *AlignAllPairs*. Some of the entries in this figure represent fairly succinct procedures, like the Lp3n3 procedure. Others however, represent hundreds of lines of source that is will be part of the final induced model; `StringAlign`, `DayhoffTransform`, and `MinRMS` are all examples of such code. The `DayhoffTransform`, for example, computes an amino acid match cost function from a set of observed substitutions. We have argued that all induced models of complex domains must be constructed from this type of large expert-encoded pieces. Here we have many examples, in the HHM domain of these fixed portions of the induced model.

6.2 Experiment 3: Using PI to Model Protein Structure

Our third experiment demonstrates the prediction of protein fold class using our implementation of PI. Figure 6.2 is repeated from the previous chapter, it shows the plausible theory that characterizes this experiment. The machinery described in the previous section is incorporated as a number of procedural attachments, some of which are listed at the bottom of the figure. The first two influents describe how the string alignment algorithm is used to find the fold class of the best matching known sequences. This is a procedural attachment for the Needleman–Wunch alignment algorithm. These influents,

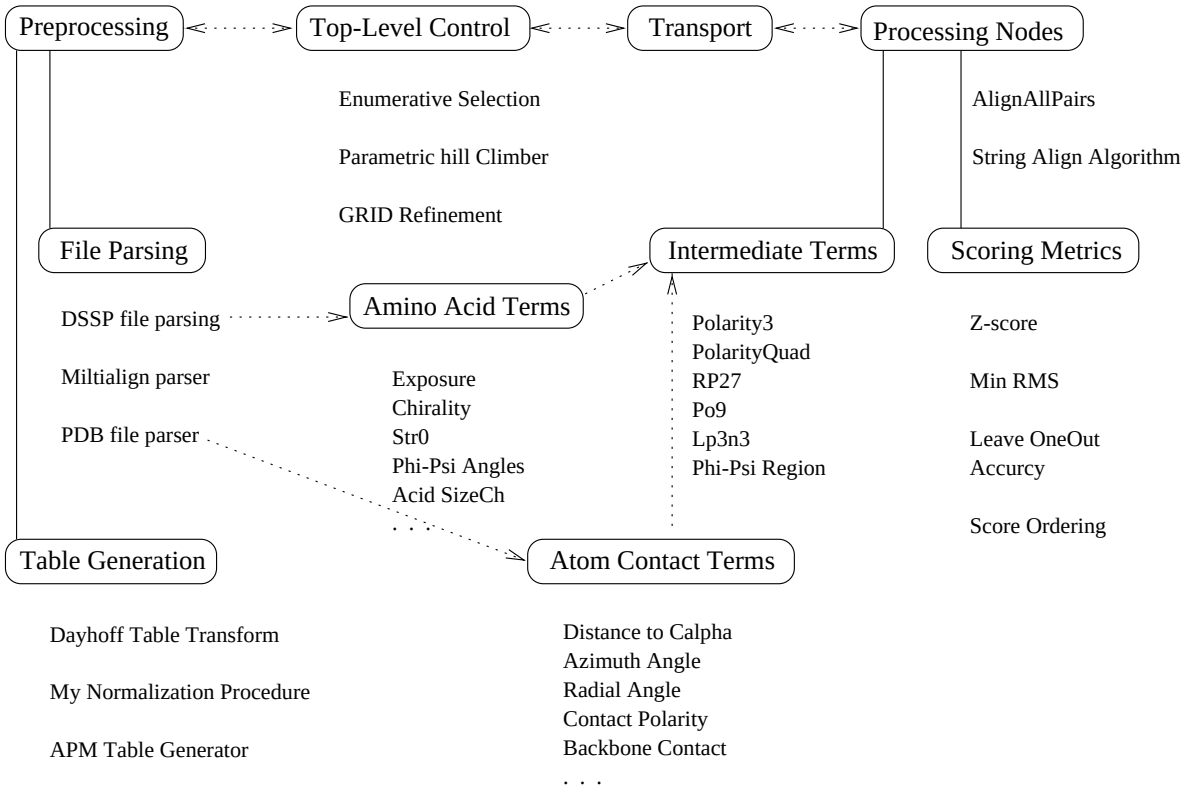

Figure 6.1 A module level breakdown of our implementation of PI

in turn, depend on two functions: a function that computes a local similarity score for a pair of amino acids ($ResSimFn$), and a function that computes the cost of inserting a gap into the alignment ($GapCostFn$). Both of these functions must be optimized through learning. The $GapCostFn$, in turn, depends on two numeric parameters: the cost of opening a gap in the alignment (GOP), and the cost of extending the gap (GEP). Most experts favor this gap function because it penalizes the opening of a new gap more than extending an existing gap (a few large gaps are more biologically plausible than many small gaps). Both the GOP and GEP parameters are determined independently through exhaustive search through the specified range of values. Exhaustive search is efficient in this case, since each space is optimized independently.

Enumerative refinement is used to select the most predictive similarity function ($ResSimFn$). The identity similarity function is defined by influent #7. Influent #8 computes similarity from amino-acid similarity matrix. The similarity matrix is computed from observed substitution patterns in the training data. Procedural attachments take the $KnownSeqs$(aligned sequences in the training data), and compute these similarity matrices using a complex, but completely understood process developed by Dayhoff. The $ContextFn$ shown in Figure 6.2 partitions the pairs of aligned amino acids into a number of different contexts. Similarity tables are then computed independently for each of these contexts. The objective of this partitioning is to separate the different substitution behaviors of the amino acids into different contexts, so that the substitution patterns found in each table will be more predictive over their particular context.

Of course, in order for this partitioning to be effective, the boundaries separating amino-acid contexts must correspond to transitions in the behavior of the amino acids (their substitution patterns). Unfortunately experts do not understand these boundaries well; there is no generally accepted partitioning of environments. Experts do have many hypotheses, however, about which environmental factors have the greatest impact on

$PredictedFoldClass(seq_u) \leftarrow$ (1)
$\quad FoldClass(Max(KnownSeqs, \lambda(x).AlignedScore(x, seq_u)))$
$AlignedScore(seq_k, seq_u) \leftarrow$ (2)
$\quad AlignAlg(seq_k, seq_u, ResSimFn, GapCostFn)/Min(Len(seq_k), Len(seq_u))$

$GapCostFn(GapLength) \prec GOP + GapLength * GEP$ (3)
$GapCostFn(GapLength) \prec_{\texttt{para}} \langle GAP, GOP \rangle \quad GOP + GapLength * GEP$ (4)
$GOP \prec_{\texttt{range}} \{10, 15, 20, 25, 30, 40, 50, 70, 100, 200, 300, 500\}$ (5)
$GEP \prec_{\texttt{range}} \{3, 5, 7, 10, 20, 40\}$ (6)

$ResSimFn(R_k, R_u) \prec$ if $(Id(R_k) = Id(R_u))$ then 100 else 0 (7)
$ResSimFn(R_k, R_u) \prec$ (8)
$\quad ArrayLookup(CtxCostMatrix(ContextFn),\ ContextFn(R_k),\ Id(R_k),\ Id(R_u))$

$CtxCostMatrix(ContextFn) \leftarrow$ (9)
$\quad DayhoffCostFnComputation(APM(TrainingData, ContextFn))$
$ContextFn(res) \prec_{\texttt{th}} \langle ExposureCutoff \rangle \quad SolventExposure(res)$ (10)
$\quad ExposureCutoff \prec_{\texttt{range}} [0 - 150]$ (11)
$ContextFn(res) \prec Str0(res)$ (12)
$ContextFn(res) \prec Chirality(res)$ (13)
$ContextFn(res) \prec StructureIndex(res)$ (14)
$\quad \vdots$

variables:

seq_k, seq_u	;; Protein Sequences with known/unknown tertiary structure.
R_k, R_u, res	;; Amino acids with known/unknown structures.

Procedural Attachments:

$Max(best, list, fn)$	;; Maximal element of a list
$DayhoffCostFnComputation(APMtable)$	;; Dayhoff APM table transform
$APM(MultiAlignments, ContextFn)$	;; Computes point mutation counts
$AlignAlg(seq_k, seq_u, ResSimFn, GapCostFn)$	;; Needleman optimal string align
$Id(res)$	;; Returns amino acid type number

Figure 6.2 Plausible characterization of experiment three (repeated from Figure 5.5)

amino-acid substitution patterns. We have encoded a number of these hypotheses as context functions, for example, the $Str0$ function partitions each residue position within a sequence into four well known structural categories: α-helix, β-sheet, turn, and coil. This $ContextFn$ is used to generate the $AAPstr0Table$ table. This is one of many possible context functions, PI selects among these alternatives using the `Enumerative` refinement space defined by the influents #10, #11, etc. There are 15 different context functions tested in this experiment, many of them are similar to context functions that have been used by others as we saw in Section 5.3. A few of the context functions are novel; they are based on specific chemical bonds associated with the residue position within the fold.

6.2.1 Experimental Results

PI operates by repeatedly selecting and optimizing choice points (refinement spaces) in the background knowledge. The theory shown in Figure 6.2 has three points where refinement is needed: GOP, GEP, and $ContextFn$. We chose initial values of GOP= 20, and GEP= 5, and on its first iteration PI used these to refine its choice of a context function. Figure 6.3 shows the relative ranking of each of these context functions given the initial GOP/GEP values. Once an optimal $ContextFn$ is chosen, the GOP and GEP parameters are iteratively refined by repeatedly optimizing one and then the other. Figure 6.4 shows the results of this optimization process. Each line in the figure represents a single iteration over all possible GOP values or all possible GEP values. The second iteration (illustrated by the line with embedded diamonds) shows the accuracy obtained when the GOP parameter is varied while fixing GEP at its initial value of 5, and fixing the $ContextFn$ at its optimum from the previous iteration. The third iteration (illustrated by the line with embedded squares) varies the GEP parameter while fixing GOP at

	Context Table	Accuracy
0	AAP-EXPOSURE3-TABLE	80.9%
1	AAP-IDENTITY-TABLE	70.4%
2	AAP-BASIC-TABLE	80.9%
3	AAP-STR0-TABLE	65.8%
4	AAP-DM78-TABLE	69.3%
5	AAP-PO9-TABLE	79.9%
6	AAP-LP3N3-TABLE	81.4%
7	AAP-STD-ID-TABLE	76.9%
8	AAP-MOD9-TABLE	80.4%
9	AAP-MOD2-TABLE	80.4%
10	AAP-MOD3-TABLE	79.9%
11	AAP-MOD4-TABLE	80.4%
12	AAP-MOD5-TABLE	80.4%
13	AAP-MOD6-TABLE	79.9%
14	AAP-MOD7-TABLE	79.9%

Figure 6.3 Accuracy at predicting protein fold class using each of the context-based substitution tables with non-optimized gap weights.

its optimum (100) from the previous iteration. The fourth iteration again varies the GOP parameter while fixing GEP to its new optimum (20). The optimal GOP value is unchanged, and so PI terminates the learning process.

6.2.2 Analysis

Notice that this experiment takes advantage of several assumptions we made regarding the interference/non-interference of these three refinement spaces. First, an assumption is made that non-optimal settings of the GOP and GEP parameters will not change the order of the scoring for each of the 15 context functions. This allows us to optimize that parameter without varying GOP and GEP. Because we *did* expect the value of the GOP parameter to affect the optimal setting of the GEP parameter, we interleaved the optimization of these two parameters. Since we had no prior knowledge about the

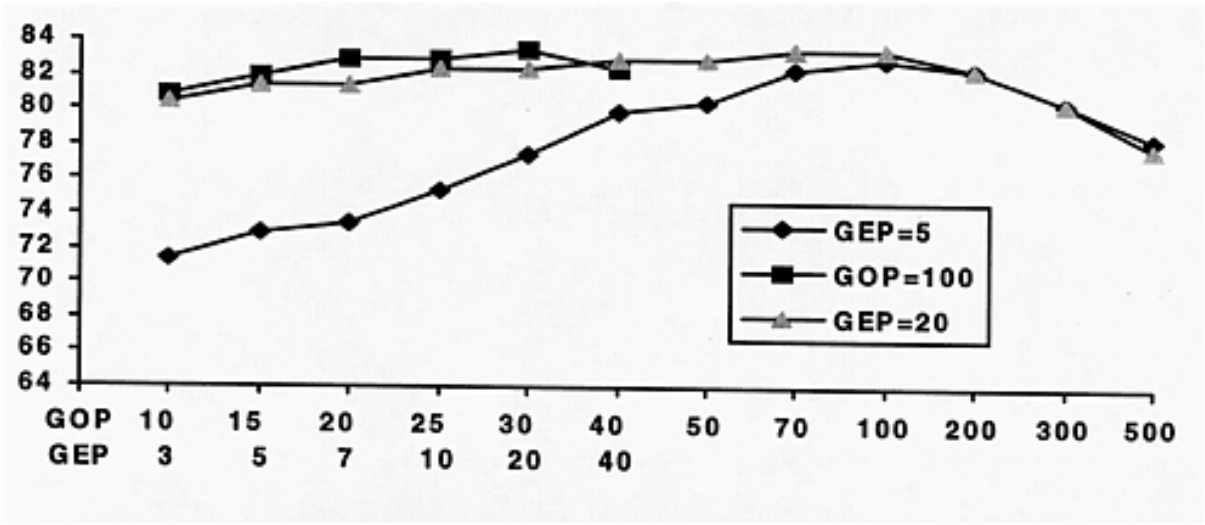

Figure 6.4 Percent predictive accuracy for GOP and GEP values.

smoothness of the GOP/GEP space we chose to simply test all possible values at each iteration. Such a brute force analysis of each refinement space is possible because we had decoupled the three decisions in this induction problem. Applying this brute force approach to the combined space would be prohibitively expensive. Even for this small example an exhaustive evaluation would require over 1,000 $AlignAllPairs$ (approximately 66 hours on a 512 node CM5).

As shown in Figure 6.2, the use of definitional influents and procedural attachment allows the expert complete freedom in specifying those portions to be explicitly encoded and portions to be learned. This turns our to be quite important for the HHM domain. In this experiment, and in others we have run, we make heavy use of the interspersing of complex, but well-understood procedures between relatively simple, but completely unknown, combination or weighting functions. This allows us to focus the scarce training data available to those portions of the complete model that are unknown to the expert.

Decomposing the choices that make up hypotheses in the previous example was important in reducing the inductive complexity of that problem. Such decomposition is, in general, not always possible. It requires that local progress toward a combined optimum

is possible in each sub-refinement space, even when the values chosen in other refinement spaces are not optimal. Nonetheless we found other examples, like the one shown here, where the expert does expect that optimization of different parts of their expertise can be undertaken independently. Notice, if we had believed that GEP and GOP could not be iteratively optimized as shown here, we would have used a different decomposition for this part of the model. Influent #4 in Figure 6.2, for example, illustrates a decomposition for $GapCostFn$ that simultaneously optimizes these two parameters using the `Parametric` function refinement space. In earlier experiments (not shown) we used this computationally less efficient decomposition for the $GapCostFn$, but limited the experiment to a single fixed $ResSimFn$. By comparing this decomposition with our current one, we were able to determine that separating the refinement of the two parameters did not reduce the accuracy obtained. Thus, when we expanded the experiment to include different similarity functions we felt confident in employing only the more efficient decomposition for $GapCostFn$. We believe that this ability to assemble and test a complex learning algorithm quickly is an important benefit of the approach. It allows the expert to quickly "try out" many different parts of a larger model in order to home in on effective decompositions for the modeling task at hand.

CHAPTER 7

ANALYSIS AND CONCLUSIONS

7.1 Specifying Bias Using the Expert's Vocabulary

High performance induction results from a correct, strong bias. It is often difficult however, for a domain expert to make strong assertions about appropriate learning parameters (bias) for two reasons: Many learning systems accept bias that controls the *general* behavior of the learning system, and because those control parameters are specified in a vocabulary familiar to machine learning researches, but which is foreign to the domain expert. Imagine asking a Biologist familiar with the material presented in Chapter 5 whether k-DNF or decision trees best match their domain, and what depth bound or k bound should be used. Their understanding of the domain is not indexed or testing in a way that allows them to answer that question. Ask them however, how `ExposedSurfaceArea` and `AminoAcidHydrophobicity` might relate to an amino-acids energy contribution, and they will have a great deal of very specific (strongly biased) things to say. They will often be able to specify a restricted class of hypotheses to search, and a limited set of sub-terms to consider. Notice that this knowledge is not global, restricting the combination of these terms to the space of all linear thresholds

does not imply any such restriction on other parts of the theory being induced. Further this bias knowledge is indexed by the domain terms themselves, as is the expert's understanding of the domain. (After all, the terms used in the problem decomposition were generated by the expert in the first place.) For these reasons it is important that a learning system is able to utilize bias that applies only to a *localized* portion of the model being induced, and that this region is indexed using *experts* vocabulary. Chapter 5 show many instances were very specialized spaces of hypotheses and limited sets of relevant terms were specified for particular intermediate terms in the experts vocabulary. Indeed it is PI's influent type and parameters that allow it to capture this rich source of strong bias.

Of course, some other learning algorithms also allow specification of bias indexed using the domain expert's vocabulary. As we saw in Chapter 4 they general do not *limit* the hypotheses considered by this knowledge, and it is this limiting of the learning to a simpler space of hypotheses or limiting the relevant term at a point in the theory that affords PI is exceptionally strong bias.

7.2 Viewing All Learning Systems As Modeling Tools

We have described, formally defined, and experimentally documented a number of advantages the modeling-tool perspective has over other inductive learning paradigms. These detailed comparisons are important since they allow us to understand exactly which aspects of the PI framework are reasonable for particular improvements in performance. Nonetheless, these detailed analyses of important differences threaten to occlude the one overarching difference that results from the modeling-tool perspective. In this section we draw a parallel between PI and conventional (complete) inductive learning, and show how the many differences documented in the dissertation stem from a single deeper idea.

We have characterized PI as a learning system able to operate within complex domains, while other complete inductive learning systems cannot. This is an accurate characterization—no complete learning system can learn complex concepts directly from a data set with 50K byte feature vector for each training instance (this is the size of a typical protein data file). Still a Machine Learning practitioner would rightly counter that this characterization does not tell the whole story. Current inductive learning algorithms *are* being applied as modeling tools in complex domains. They are not applied directly to raw data from these domains, but rather to a set of carefully constructed attributes derived from this raw data. Figure 7.1 graphically depicts this larger domain modeling task, and how a complete learning systems fits in to this picture. As with all of our diagrams, the triangles represent sub-components within a model's functional decomposition. The shaded region in the figure is the portion of the model that was induced. The unshaded regions are those functions which the expert has explicitly coded in order to transform the raw data into a set of attributes for which learning is possible. In this way a complete learning system can also be used as a tool that directed by its user. Surprisingly, the control exerted by the expert in this scenario is not contained in any rules, structures or explicit bias presented to the learning system. Control is primarily exerted through the functions defined in the lower part of the figure. To force a complete learning system to only consider particular combinations of the raw attributes, the expert simply doesn't allow the system to "see" other unwanted combinations. Oddly enough, the greatest source of bias for these systems is a set of functions (lower part of figure) that the learning system does not have access to! The practitioner would correctly argue that, aware or not, this *does* allow these inductive learning to be used as tools in complex domains.

The expert's direct contribution to PI's learning can also be graphically depicted as shown above. Figure 7.2 shows shaded and unshaded regions as before. Triangles shaded

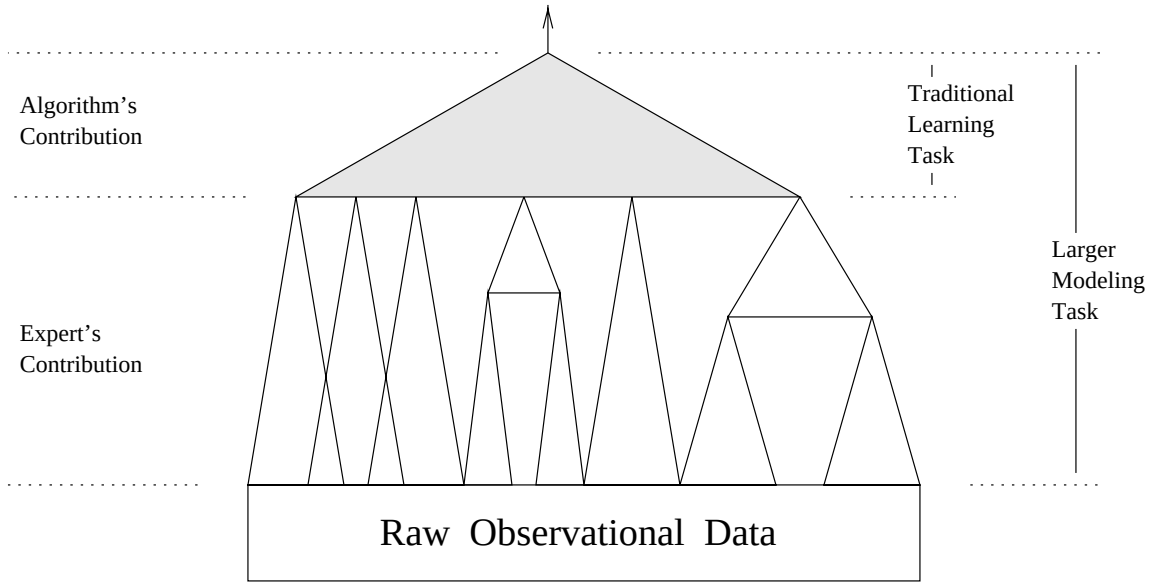

Figure 7.1 Learner and expert contribution to a domain model

at the top represent refinement spaces, like the parametric space, where most of the model is directly encoded by the expert, and learning is used to "fill in a small part of that structure" (set the coefficients of the parametric function, for example).

Viewed in this way, these two approaches do not seem all that different, the total shaded area in the two figures is roughly the same, but PI's shaded regions are simply spread out. Said another way, the learning has been "moved around" a bit, but the same "amount" of learning seems to be happening. We take a moment to make this notion of "amount" of learning formal since it is instrumental in our deeper understanding of the relation of PI to other learning systems. For this discussion we view the learning depicted by these diagrams as a set of independent binary choices made by the learning system. A refinement space (node) in the decomposition that has n hypotheses can be viewed as making $log_2(N)$ independent binary choices. In order to discuss the "amount" of learning happening at different points within these models we coin the term MIB or (Machine Induced Bit). So a refinement space that is selected among 1,024 alternatives would be said to contain 10 MIBs. The combined hypothesis space size for an entire decomposition

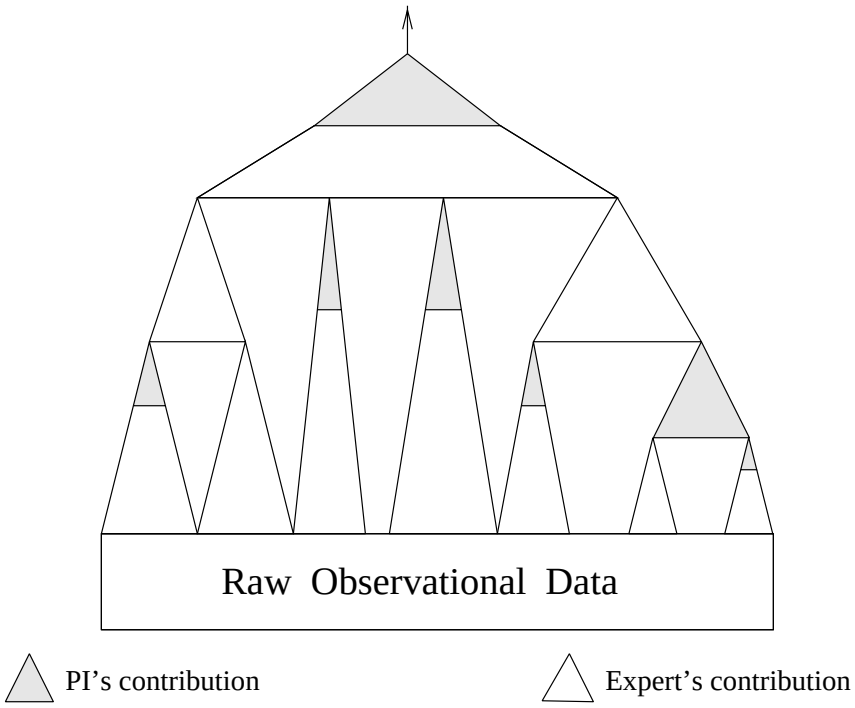

Figure 7.2 PI and Expert contribution to a domain model

with C independent binary choices spread through its nodes is 2^C. The VC-dimension of this space is bounded above by the log_2 of this hypothesis space size[72]. So the VC-dimension is bounded above by $log_2(2^C)$ or simply C. The minimum number of training examples needed to learn a Boolean concept is also bounded by this VC-dimension. So the labelings on a fixed set of training examples of size C cannot be guaranteed to be sufficient for learning over any space of hypotheses constructed from more that C MIBs. This upper bound can only be obtained given an optimal arrangement of hypotheses and training data. Nonetheless we can think of any fixed set of available observations as having some C value representing the number of MIBs it is capable of inducing. Thus one can think of our training data as a resource that allows us to put a fixed number of MIBs (shaded area) into our model, but no matter where we put those bits we only get a fixed "amount" of them.

We see that all inductive learning systems (ours and others) are subject to this upper limit on the amount of learning allowed given a fixed source of training data. It stands to reason then that when training data is limited and the domain is complex, this fixed resource of MIBs would become quite precious. The expert would want to use these MIBs to cover areas of uncertainty as judiciously as possible without wasting them by incidental coverage of implausible areas.

There is no reason to expect that the top of a model's decomposition will have the greatest degree of uncertainty, yet the use of a conventional learning system *force* all MIBs to be placed at the top. (The shaded region at the tope of Figure 7.1 denotes the allocation of MIBs using a complete induction algorithm over a set of derived attributes.) Forcing the expert to allocate all MIBs at the top of the decomposition could be quite wasteful if significant uncertainty occurs lower in the decomposition. An alternative available to the expert in this case is to increase the size of the top MIB region, but this alternative requires an exponentially increasing amount of training data. The other possibility is to rearrange the decomposition so that the uncertain portion occurs lexically at the "top" of the theory. This is not always easy, or even possible. Consider the function `Exposed`(*AminoAcid*) from Chapter 5. It is a very complex function with a single unknown threshold that requires about eight MIBs of learning but is otherwise a completely determined Boolean predicate. This predicate in turn is used to define a context-specific similarity table. The only way to push this uncertainty to the top of the decomposition is to instantiate the function `Exposed` sub-term with all possible 256 different thresholds and present them as 256 different similarity tables. This works but is very wasteful, because there is no way to tell the learning system to select just one of these exposure tables when inducing a best combined tables from a number of sources. The complete learner will consider an exponential number of silly hypotheses that attempt to combine the tables. Indeed this is so wasteful that no practitioner

would resort to this. Instead they would arbitrarily choose a value for the threshold and hope for the best. Perhaps they would choose several values and rerun the whole learning algorithm several times, thereby manually simulating PI's decomposition of a single task into smaller learning tasks. Both alternatives are grim, and in truth most learning systems are simply not applied to problems cannot be reduced to a single region to be learned. This is an unfortunate and totally unnecessary restriction. We see using the PI approach, that it is possible to spread the MIBs out over the whole model to cover the areas most needed.

Notice that we have not claimed (and do not believe) that PI's flexibility in allocating MIBs is optimal. The degree to which its refinement spaces match the uncertainty found in the expert's understanding of the world will determine how efficient PI is over a particular domain. What is being claimed is that, among other things, a learning systems that is used as modeling tools *must* be judged on how well it allows the expert to match the allocation of MIBs with their uncertainty about the domain. We also want to claim, that while not optimal, PI does go along way toward allowing the expert flexibility in the placement of MIBs in the problem decomposition.

There are a number of aspects of the framework described in Chapter 2 that on the surface appear quite orthogonal but in fact are tied together by this deeper notion (flexible, efficient allocation of the learning resource). Decomposing the learning task into a network of refinement spaces allows the expert shift learning to where it is most needed. In some cases the hypotheses considered by a specialized type may be completely subsumed by a more general refinement type. Nonetheless the existence of both refinement space types is important as it allows the expert to select a type that minimize the number of MIBs used to cover each uncertainty within the theory. A set of overriding influents for example, could be encoded using the more general `BooleanCombination` space, but doing so would be wasteful because it would require more data for convergence. The

extensibility of the influent language itself is also motivated by the notion of efficient MIB allocation. The sophisticated user may want to include specialized representations and algorithms if the domain being modeled has a recurring form of uncertainty that is wastefully covered by the existing refinement types. We see all of these aspect of our framework serve a common goal: allowing the expert the greatest possible flexibility in using the training data resource to resolve uncertain aspect of their understand of the domain. In a practical sense we believe these three aspects: decomposition, specialized algorithms, and extensibility all go a long way toward making PI into a learning tool that facilitates efficient, directed use of the learning resource.

7.3 Limitations of the Framework

We have explored a number of advantages of Plausible Inference. Through direct comparison and extended application, we have demonstrated many desirable properties including higher learning performance (computation time and accuracy) scalability, usability, etc. We now turn our attention to the commitments and limitations inherent in the approach.

7.3.1 PI Only Learns From Connected Knowledge

PI allows an expert to select the degree of control they desire over the learning process. A `BooleanCombination` space with all observable attributes as inputs is an example of extremely limited control (weak bias), whereas a real-values threshold with a limited range over a single variable is an example of tightly controlled induction (strong bias). What PI does not permit is for the expert to leave the degree of control *undefined*. This is an obvious consequence of PI's structure: an influent theory explicitly specifies the range of hypotheses to consider at each point in the learning. There is room for tight

or limited control, but no room for undefined control. Some other learning approaches do not have such explicit languages for control and *must* leave the scope of hypotheses considered undefined. A theory refinement system, for example, accepts a set of rules similar to the background knowledge used by PI. Unlike PI, however, these rules do not contain any information to control the scope of the refinement process. The goal of the theory refinement system is to "refine" the input theory into a predictive output theory. Which refinement should to consider and in what order is complete by up to the learning algorithm.

We have shown the advantages PI reaps from having such control. The disadvantages associated with it are two fold: first, the expert is obliged to exercise this control. Even if the expert simply decides to adopt a weakly controlled learning strategy, the burden of making that decision, if nothing else is still on the expert. The second limitation is a little less obvious, but certainly no less important. PI requires the expert to circumscribe (however loosely) the hypotheses to consider at each point within a decomposition. PI has no freedom to escape these explicit bounds and consider alternatives it somehow thinks might be useful. This is a crucial part of how PI can function in complex domains. Unfortunately it is also why PI has no use for knowledge that does not fit somewhere within the expert's decomposition. To understand what this means, imagine a less-than-stellar physicist "expert" that remembers the definition of momentum, but can't really remember how momentum fits into a model of rigid body motion. A theory refinement system could easily accept such knowledge and perhaps even make good use of the momentum chunk in its induction of a model of rigid body motion. PI, on the other hand, would happily accept an influent that defines momentum, but would simply ignore the influent when learning, since it hasn't been told where to try sticking that knowledge in its model of motion, and PI is prohibited from trying unrequested hypotheses.

This inability to incorporate unconnected knowledge is not simply a limitation of the eight refinement types we happen to have defined, but rather is a limitation inherent in the tradeoffs we have made in the approach. We have assumed that the domain is so complex that the number of potential inputs for every node within a problem decomposition is extremely large. Thus, even one completely unconstrained node in a problem decomposition would cause PI to fail; each node must explicitly specify those inputs that may be considered in the induction of that node. Thus an influent's consequent is either mentioned in the problem decomposition and participates in the learning, or its is not mentioned and does not participate. We believe that other learning systems can "get away" with use of disconnected knowledge precisely because they assume the domain is simple enough to permit expansion of nodes within their induced model without expert control—we are not willing to make that concession, so we must live with this limitation.

7.3.2 PI Needs Full Problem Decompositions

PI requires a structure of sub-refinement spaces that stretch all the way from the goal term, `G`, to observable terms, `O`. Without this full decomposition PI cannot do any learning. This is such an obvious limitation resulting from PI's insistence that the expert be in control of the learning process. We consider the ramifications of this limitation because we believe that, while significant, they are much less damning than they first appear. A learning approach that requires a full problem decomposition is suspicious; after all it seems if we had a full problem decomposition we might not *need* the learning system!

There are two methods an expert may use in cases where full problem decompositions are not known. First, if the appropriate sub-decomposition for a term in the full decomposition is not known but a number of alternatives are known, then the expert can

encode that term as explicit selections among these alternatives using the `Enumerative` refinement space type. But as is often the case for such nodes, there may not even be a known set of alternatives. In this case, a weakly controlled refinement space like `BooleanCombination` must be employed to "swallow up" this uncertain portion of the decomposition. The larger the fraction of the entire model swallowed by such an unconstrained refinement type, the lower the efficiency of the overall learning. In the extreme case, the expert provides no structure at all, and declares that the entire learning problem is a single `BooleanCombination` space. In this extreme case, PI's learning behavior becomes indistinguishable from the complete Boolean concept learner used for that refinement space. We believe this graceful degradation in performance as less structure is provided by the expert is a desirable behavior for a learning tool.

Notice that PI's performance drops to, but not below, that of the more conventional complete induction algorithm it is based on. The only way for PI to underperform its constituent refinement algorithms is for the expert to make promises (through the encoded problem decomposition) that are not consistent with the domain itself. Thus, there is no potential for degradation when the expert declares that they "don't know part of the problem decomposition," and thereby encode that part of the problem using a generic (complete) refinement space. Degradation in performance is only possible if the expert declares (through the encoded decomposition) that they are certain about the problem decomposition, and they are wrong. This is exactly how a tool should behave. A tool that is too weak to allow its user to do much damage is also a tool that is too weak to allow its user to do much good.

7.3.3 Limitations On TARGETs Passed between Refinement Spaces

The various influent types described in Chapter 2 cannot be arranged arbitrarily into problem decompositions. Some refinement spaces, the `Threshold` space for example, do not define a RELEVANT subspace function. It is the RELEVANT function that specifies `TARGET` training sets for the subspace below a given space. Since the `Threshold` space has no RELEVANT subspace function, it must always be used directly on fixed combinations of the observable terms. This topology constraint seems like an odd limitation, but it is an unavoidable consequence of the fact that the thresholding operation cannot be inverted. Imagine modifying our car-starting domain so that the `GasLevel` term was not directly observable, but was an intermediate term that also had to be induced. Given TRUE/FALSE `TARGET`s for `CarStarts` one can back propagate to a hypothesized `TARGET` for the `HasGas` term (just use the same TRUE/FALSE labelings). Given TRUE/FALSElabeling for `HasGas`, however, there is no effective way to invert the thresholding operation to generated hypothesized gas levels. We know that the `HasGas` = TRUE cars should have more gas then the other cars, but we cannot assign specific real-valued intermediate observations in any meaningful way. (Should a car that `HasGas` have `GasLevel` = 50% or `GasLevel` = 80%?) Most of the refinement space types we have considered do have a natural inversion of their mapping that can be used to obtain subspace `TARGET`s. These spaces can be used anywhere within the problem decomposition.

This restriction on refinement space decompositions suggests that the `TARGET` mechanism used to propagate information between individual refinement might be too limited. Perhaps a more general communication mechanism would allow the `Threshold` space to effectively propagate information to its sub-refinement space. We will show why this is *not* possible, and argue that the current definition for `TARGET` is as general as possible. According to Figure 2.1 in Chapter 2, a `TARGET` is a vector of "target" labelings for

some intermediate term. Each entry in the vector can optionally be assigned a fractional (rational) weight value. (In our examples we have assumed a uniform weighting and so weights have been ignored, but for generality they are included in the framework.) This aspect of our specification of a generic refinement space is noticeably more constrained, than any other part of our specification, and as we saw in the last paragraph, not all refinement space types can generate this particular form of target. The specification of influent syntax, refinement space inputs, and BEST hypothesis outputs are specified without making assumptions about the specific form of these components, since those assumptions are not necessary to ensure the correct functioning of the framework. Unfortunately the specification of the `TARGET` passed between refinement spaces, however, must be more constrained. The influents used by one space do not need to be understood or interpreted by any other space. `TARGET`s generated by one space, on the other hand, *do* need to be interpreted by the subspaces of that space. Thus it is important that the `TARGET` is fully specified so that the receiving space knows how to interpret and use the information. We chose a weighted set of training data labels because it is the most general specification guaranteed to be consistent with all inductive learning algorithms. Any inductive learning algorithm accepts a set of labeled training examples exactly like those in a `TARGET`. Many of these algorithms also can accept a weighting on the training data as well. If a particular algorithm does not accept weighted training then it can be given multiple copies of each training example in order to simulate weighted training data. So, while we can imagine different alternative representations one might want to employ, this alternative was adopted because it is a least common denominator for communicating information about intermediate attributes between refinement spaces.

7.3.4 Applicability of Plausible Inference

Throughout this dissertation, we have demonstrated a number of benefits relating to the performance, usability, and applicability of Plausible Inference. We have also shown several limitations on the use and effectiveness of the approach. For what class of modeling tasks do the advantages bestowed by the approach out weigh its limitations? We answer this question by discussing the domain properties necessary for effective use of the framework into a description of an ideal domain. This serves as a brief summary of how the advantages and disadvantages of the approach impacts its applicability across different domains.

Our primary objective in the design of the framework shown in Chapter 2 was to minimize the constraints the framework placed on the refinement spaces themselves. This minimal commitment approach allows PI's user-directed problem decomposition to be used with virtually any representation of background knowledge and world representation. No specific assumptions regarding influent syntax or training-data representation are needed, so none are made. First-order attribute/value pairs are sufficient for encoding any digitally encoded set of observations. By convention the influent syntax for the existing refinement spaces follow a common schema. All that is strictly required, however, is that the influents describe some *functional* relationships between the domain terms. Specific interpretation of the each type of influent is done by the refinement spaces themselves (through the RELEVANT, BEST, and REFINE functions). Most inductive learning algorithms learn functional relationships, and can be incorporated in the PI framework. This aspect of the framework is very general.

In order for a domain to benefit from our framework, it is necessary for the refinement space types to occur repeatedly across the domains being modeled. We have made no assumption that the eight refinement space types developed here are sufficient for

all domains, but it is assumed that some small set will suffice for a range of domains. This is an important assumptions, if each refinement space type is developed for a specific part of a single learning task, then the framework degenerates into a task-specific learning algorithm with no reuse. Our application of PI has shown that very general forms of uncertainty (like spaces of parametric functions) do exist. These applications also suggests that such generic forms of uncertainty will often occur repeatedly across a complex domain. The influents developed for modeling protein structure seem to apply equally to any physical domain. Our intuition is that broad domain categories like, physical, financial, planning, etc. will each have a handful of unique refinement space types appropriate primarily for that class. Beyond this intuitions, however, it is very difficult to know in advance, if there are domains where expert uncertainty does not fall into recurring categories. We can only say that recurring refinement space types is a necessary precondition for the use of Plausible Inference.

More constraining than PI's need for recurring uncertainty types is PI's need for expert-encoded problem decompositions. By design PI is strongly guided by these decompositions, thus is it critical that any application of PI be in domains where experts can generate such decompositions. In the previous section we demonstrated a couple of ways for an expert to cover gaps in problem decomposition using weaker refinement space types. Nonetheless, the constraint provided by these decompositions is the source of PI's ability to out perform other learning systems. Domains so poorly understood that expert's cannot even generate partial or hypothesized problem decompositions are not candidates for Plausible Inference.

Though we have shown that, in principle, PI can be applied to a great many learning tasks, there are practical considerations limiting PI's application. Not only is it necessary for problem decompositions to be available, but it must be worth the effort required to encode and use them. This means that the learning task must be too complex for the

application of any weakly-biased learning system. Such systems that do not require encoding of complex influent theories are a preferable when they are applicable. It is also necessary for the domain to be important enough to merit the human resources necessary to encode these expertise even when in it available. Notice this required complexity and problem importance are often both found in domains where a large community of experts are actively working. Indeed it is these modeling tasks "on the frontier of human understanding" for which PI's tradeoffs were designed.

7.3.5 Issues for Continued Study

There are a number of aspects of our work that merit continued exploration. We have broken these into three groups: (1) Formal extensions to the framework that aid in our understanding and use of the framework. (2) Extensions to the framework designed to enhance PI as a practical tool for domain modeling. (3) Further work in the modeling of protein structures.

A semantics for PI

An important consequence of our parallel between inference and induction is the idea that inductive learning systems could (and should) have a formal semantic basis. As we argued earlier, complex domains require complex bias, and encoding this bias involves all of the difficulties associated with more traditional knowledge representation. Because deductive inference has a compositional formal semantics (model theory), an expert is able to understand and encode each sentence in a theory independently. They are also able to understand these sentences directly from their model-theoretic meaning without considering all possible derivations from those sentences. Providing a compositional formal semantics for PI would allow the expert the same ability to understand the influents themselves without having to reason about the operation of PI's induction engine. Such a

semantics would also allow a large influent theory to be understood piece by piece, rather than as a single huge bias specification. Both of these aspects afford a great practical importance to this theoretical addition to the framework.

There are several aspects of our framework that facilitate the development of a compositional semantics. The functional decomposition defined by the influents provides a straight forward basis for a formal semantics for the refinement space. Each refinement space asserts the existence of a functional relationship of a specified form (from its hypothesis space). This functional relationship can also be used to define a semantics for the influents associated with these refinement space types. The threshold influent, for example, can be succinctly characterized as the existence of a functional relationship: $\texttt{A} \prec_{\texttt{th}} \texttt{B}$ *means* $\exists n$ such that $(\texttt{A} \geq n) \leftrightarrow \texttt{B}$. Other influent types will undoubtedly have more complex formal characterizations. Nonetheless we believe this basic approach can be used to provide formal characterizations for other influent types as well, and having a simple compositional semantics for PI would enhance our ability to translate an expert's understanding of a complex domain into an influent theory.

Enhanced top-level control for PI

In addition to these formal additions to our model, there are several pragmatic issues that merit further study. One important, but only lightly studied aspect of the framework, is its top-level control. We have assumed that the tree of refinement spaces is generated using the RELEVANT function, and that spaces from this set are repeated chosen and refined at random. For all tasks we have considered, this procedure is sufficient though not necessarily computationally optimal. We can imagine learning tasks, however, where pathological interactions occur between the optimization for connected refinement spaces. There are a number of top-level control strategies that one might adopt in order to extend

PI's applicability to cover a greater fraction of these domains where these interactions occur.

A pathological interaction occurs when the adoption of an incorrect hypothesis in one refinement space, systematically generates errors in the `TARGET`s or attribute-values passed to other spaces. These errors, in turn, keep those spaces from progressing toward correct refinement. Notice, we expect initial refinements *will* generate error, but we only label them as pathological if these errors distort the information passed to other spaces to such a degree that no progress is possible in those spaces, and an impasse is reached. One solution to the pathological interaction problem is to consider multiple alternatives for each space. This can be done by simply restarting this random process multiple times, and selecting the best decomposition induced. There are also more sophisticated approaches which attack the same basic problem, but attempt to combine information from these alternative decompositions. A beam search could be used in for top-level control. This would maintain the top n decompositions, while randomly refining and testing these alternatives.

A genetic algorithm approach is also very appropriate for the top-level control of the PI framework. We believe the refinement space structure provides an ideal crossover mutation operator. Sub-trees that have generated useful intermediate predictions within one decomposition are likely to produce useful predictions within other decompositions, since the intended function of this sub-component with in all of these structures is the same. This ability to crossover parts from different PI executions is due to the relative independence of the refinement spaces themselves. This independence suggests that rather than running the algorithm repeatedly and discarding all but the best iteration, we might share the best sub-trees generated on successive iterations by simply computing them in parallel and using sub-tree crossover to test combinations from these different runs of the algorithm. Many alternatives are possible for the top-level control, and it is

unlikely that any single control strategy will be optimal for all tasks, nonetheless having several alternatives like at the user's disposal these would increase PI's reach into domains that cannot be decomposed into benignly interacting refinement spaces.

Further work in modeling protein structure

Our application of PI has provided a tantalizing first look at how an automated modeling tool can be used to build models of protein structure. There are a great many tantalizing avenues for continuing work using PI as a modeling tool within the field of computational biology. Physics, chemistry, and biology all suggest very complex and potentially predictive structures, these structures are the idea basis for hypothesized problem decompositions for PI.

A promising and relatively unexplored direction in the field, is to use of simple models of chemical and physical constraints to build fine-grained models of particular protein structures. Biologists and Chemists have made considerable progress in understanding the steric and electro-chemical interactions that occur between neighboring amino acids within a protein. Because it is difficult to incorporate this knowledge into purely statistical approaches, little of this knowledge is currently incorporated into models that predict protein homology. We can use much of this knowledge by incorporating it as an influent theory. These influents make predictions about the propensity for particular amino-acid substitutions. PI is used to refine these influents to maximally separate members and non-members of a fold class family using these substitution predictions.

A second avenue that can be used in independently or in conjunction with the first is the use of bi-layered decompositions. Each position within a fold class has a unique environment because of its position within the folded protein. Models based on this specific environment can be based on this unique environment and thus can be combined into much more accurate predictors. Unfortunately many fold classes have only a few

dozen instances with known structure, this provides very little data for refining models specific to each amino acid position. A promising solution is to use a two-level influent theory: The lower part of this theory attempts to predict various local and secondary structure attributes from the raw data. Many instances of these local structures can be observed over the entire training set. So these models can have a relatively large learning component. The second level in this influent theory deals with the combination of these predictors in a unique way for each amino acid position within a fold class. This step has much less data available, but much less data is required to simply test which of the level-one attributes are predictive for *this particular site* within the fold class. We are very excited about this approach because it seems to make good use of all available data for each prediction made, but still retains some ability to specialized these predictions based on the unique qualities of each specific positions within a fold class. These are two concrete next steps which we believe hold great promise. These two examples, however, only scratch the surface; there is a plethora of theories in this domain that could be efficiently tested and refined using PI.

7.4 Summary

We have explored the use of a structured space of specialized learning algorithms as a modeling tool in a number of different ways. We summarize these avenues of exploration here A formal characterization of our learning tool, PI, is provided in Chapter 2 along with a characterization of eight refinement space type that we use throughout the dissertation. As a further aid to understanding the framework, we also provide a detailed trace of its application to a learning task designed to exercise the important components of the framework in that chapter. The objective of our framework is to match the hypotheses considered by induction as closely as possible to the uncertainty in the expert's current

understanding of the domain itself. To test this ability we encoded an array of different forms of expertise into influent theories. Examples of this are found in Chapters 3, 5, and 6. Effort was made in Chapter 5 to obtain expertise at the forefront of a field of study, since this is the type of domain appropriate for PI's use. We demonstrated empirically the large, practical improvement possible through the use of structure, and specialization in our experiments in Chapter 3. In Chapter 4 we compared PI to a number of other inductive learning paradigms, these comparisons served to place PI among other learning systems on the autonomy verses performance spectrum. Chapter 4 also mapped defeasible inference and knowledge-based inductive learning onto a common framework, where the choices made by each sub-field were directly compared. These various methods of analysis, provide a detailed picture of PI as a learning tool.

7.5 Conclusion

There are four important aspects of this work which we believe have import for the entire field of inductive learning. They follow from our perspective of induction as an expert-controlled modeling tool, they are responsible for the performance improvements documented in this dissertation, and they summarize the contribution of this work. We enumerate them, and then elaborate on each in this final section.

1. PI allows the specification of bias using the domain expert's vocabulary.
2. PI accepts problem decomposition information as bias.
3. PI is a *general* approach that combines different learning algorithms into a problem-*specific*, high-performance learning system.
4. PI allows the expert to allocate the fixed learning resource to portions of the domain model where it is most needed.

As we discussed in the first section of this chapter, PI accepts its bias as a set of influents. These influents set localized parameters for the learning of each expert-specified domain term. Indexing and specifying bias in this manner results in strong bias, as we have seen in the experiments in Chapter 3. It also facilitates the encoding of this strong bias. We have found that a domain expert's ability to generate strong learning bias is greatly increased if that bias is only to be applied to a particular intermediate domain term familiar to the expert.

Learning within the confines of an expert-hypothesized problem decomposition also results in very strongly biased inductive learning. Furthermore, important, complex domains have often been extensively studied, and *have* a body of expertise that can be encoded as hypothesized decompositions. Thus, any learning system designed for such domains would benefit from the ability to accept problem decomposition information.

Related to these first two points is the idea of using specialized learning algorithms as building blocks for complex learning systems. Machine learning researchers have long recognized the tradeoff between the performance and generality of a learning system: either a learning algorithm is very powerful, or very general, but not both. By composing learning algorithms into a complex structure of interrelated algorithms, we are able to construct specialized (and thus potentially powerful) learning algorithms. Any given combination of learning algorithms is strongly-biased and specialized, indeed such a combination has been designed for a *particular* learning task. Nonetheless, the approach itself is general, a *different* specialized combination can be constructed for each learning task. This is probably the most important aspect of our framework. Using it we are able to obtain the high performance of specialized learning algorithms on a task-by-task basis without hand-coding specialized learning algorithms for each new application. Use of a structured set of building-block algorithms provides the high level of specialization without requiring the labor of repeatedly coding these very specialized learners.

The parallel we have drawn between defeasible inference and inductive learning has important consequences for both areas. In the area of defeasible inference, it challenges the fundamental assumption that a minimal-change bias is the only or best way to update beliefs given contrary evidence. Indeed we presented a simple example, typical of many domains, where a generalization bias not minimal-change bias is most appropriate. On the other side, this parallel highlights the need for a world-centric, not learner-centric, semantics for bias knowledge. We believe the practical importance of having a formal semantics for bias knowledge will grow rapidly as the complexity of bias knowledge grows.

Through the simple analysis shown in Section 7.1, we demonstrated that training data can be viewed as a resource (MIB), that can resolve a fixed "amount" of ambiguity in the model being induced. By viewing learning as a tool (part of a larger expert-machine modeling effort), we are able to ask how effectively the learning resources are being utilized in this larger context. We can track down ineffective use of the learning resource by various parts of the entire model. A refinement space with hypotheses deemed implausible by the expert is wasting the learning resource. This criterion provides important guidance for the design of refinement space type, and for the design of effective influent theories for a given learning task.

The utility of this perspective is not limited to our approach. This perspective provides an important criterion for understanding how effectively *any* learning system can be employed as a modeling-tool. Even learning algorithms that provide no explicit control over the allocation of learning resource still have made some allocation of that resource when viewed in this larger context. If that allocation is not efficient, it still has the same negative consequences on modeling performance. Thus, the ability to allocate learning resource is an important criterion by which all learning algorithms should be measured.

The decomposition of learning into a set of simpler learning tasks suggests an interesting agenda for the Machine Learning community. Most researchers agree that no

"universally optimal" learning algorithm is possible; high performance is possible only with high specificity. We have shown, however, that general learning components can be combined to form specialized learning systems. This suggests that further progress in high performance induction is possible through the identification of specific recurring forms of domain uncertainty, and the development of algorithms that are capable of learning models for these recurring forms. This is a pleasing compromise, it allows the pursuit of a general approach to learning, while at the same time still permitting instantiations of that approach to be specialized and thus achieve high performance.

APPENDIX A

SUPPORTING MATERIAL FOR EXPERIMENT THREE

This appendix provides the details underling the experiment in Chapter 6 required to reproduce the result. The first section provides the raw data files from the Brookhaven Protein Database used in the experiment. The second section specifies the steps used in computing and evaluating the similarity tables used in the experiment, including the source code for the procedure that translated raw accepted point mutations (observed substitutions rates) into a similarity function. The third section defines each of the amino-acid similarity functions used in the experiment. And the fourth section provides the similarity tables generated for the substitution table that resulted in the highest predictive accuracy score.

A.1 Protein Sequences Used In Experiment Three

This is the list of protein sequences used in experiment three. These are used to compute observed substitution rates that underlie the similarity functions, and they are used test the combined similarity functions generated by the experiment. These sequences are a subset of the 200 sequences used by Pascarella and Argos[73]. They

were provided to me in machine readable format by Tom Ioerger. Each entry in this list specifies: the Brookhaven Protein Data file used. It specifies the sub-chain selected as well as inclusive starting and ending indices used. Each entry has a fold class label. The learning algorithm's goal is to predict this label, of the "unknown" protein by select the label of a similar "known" protein. Entries with no chain specification assume that the first chain listed in the protein data file is used.

file	**range**		**foldclass**	**file**	**range**		**foldclass**
256b	A		256B	2ccy	A		256B
1cms	1	175	AC-PROT	1cms	176	323	AC-PROT
4ape	2	174	AC-PROT	4ape	175	323	AC-PROT
3app	1	174	AC-PROT	3app	175	323	AC-PROT
2apr	1	178	AC-PROT	2apr	179	325	AC-PROT
4pep	1	174	AC-PROT	4pep	175	326	AC-PROT
2taa	A		BARREL	1wsy	A		BARREL
1tim	A		BARREL	1gox			BARREL
1ypi	A		BARREL	1fcb	A	100 511	BARREL
1abe			BINDING	2liv			BINDING
2lbp			BINDING	2gbp			BINDING
1ca2			CARBONIC	2cab			CARBONIC
3cln			CA-BIND	5cpv			CA-BIND
3icb			CA-BIND	4tnc			CA-BIND
5tnc			CA-BIND	4gcr	1	39	GCR
4gcr	40	87	GCR	4gcr	88	128	GCR
4gcr	129	174	GCR	2gcr	1	39	GCR
2gcr	40	86	GCR	2gcr	87	127	GCR
2gcr	128	173	GCR	1fcb	A 1	99	CYTB
3b5c			CYTB	451c			CYTC
1ccr			CYTC	1cyc			CYTC
5cyt	R		CYTC	3c2c			CYTC
155c	1	122	CYTC	2cy3			CYTC3
2cdv			CYTC3	3dfr			DFR
4dfr	A		DFR	8dfr			DFR
1dhf	A		DFR	1cse	I		EGLIN
2ci2	I		EGLIN	1phh			FAD-NADH
3grs	18	161	FAD-NADH	3grs	186	294	FAD-NADH
4fxc			FCX	1ubq			FCX
4hhb	A		GLOBIN	4hhb	B		GLOBIN
2mhb	A		GLOBIN	2mhb	B		GLOBIN
1fdh	G		GLOBIN	1mbd			GLOBIN

file	range			foldclass	file	range			foldclass
1mbs				GLOBIN	2lhb				GLOBIN
1eca				GLOBIN	2lh1				GLOBIN
1pmb				GLOBIN	1mba				GLOBIN
3hla	A	1	90	HLA-A2	3hla	A	91	182	HLA-A2
2hmz				MHR	2mhr				MHR
2fb4	L	1	109	IGB	2fb4	L	110	214	IGB
2fb4	H	1	118	IGB	2fb4	H	119	221	IGB
2fbj	L	1	106	IGB	2fbj	L	107	213	IGB
2fbj	H	1	118	IGB	2fbj	H	119	218	IGB
1mcp	L	1	113	IGB	1mcp	H	1	122	IGB
1rei	A			IGB	2rhe				IGB
7fab	L	1	109	IGB	7fab	L	104	214	IGB
7fab	H	1	117	IGB	7fab	H	118	220	IGB
2f19	L	1	108	IGB	2f19	L	109	215	IGB
2f19	H	1	123	IGB	2f19	H	124	220	IGB
3hfl	L	1	105	IGB	3hfl	H	1	116	IGB
3hfl	H	117	213	IGB	1cdh				IGB
3hla	A	183	270	IGB	3hla	B	1	99	IGB
4fab	L	1	112	IGB	3hfm	L	1	108	IGB
1mcw	W			IGB	1i1b		3	51	IL
1i1b		50	107	IL	1i1b		108	153	IL
1tgs	I			INHIBIT	3sgb	I			INHIBIT
2ovo				INHIBIT	1ovo	A			INHIBIT
3cna				LTN	2ltn	A	1	108	LTN
2ltn	A	109	181	LTN	2ltn	B	1	47	LTN
3lzm				LZM	2lzt				LZM
2lz2				LZM	1lz1				LZM
1alc				LZM	4mdh	A			NBD
2ldb				NBD	1ldm				NBD
5ldh				NBD	2ldx				NBD
1llc				NBD	8adh				NBD
3gpd	R			NBD	1gpd	G			NBD
1gd1	R			NBD	1fx1				NBD
4fxn				NBD	3adk				NBD
8atc				NBD	9pap				PAP
2act				PAP	1pp2	R			PLIPASE
1bp2				PLIPASE	1p2p				PLIPASE
1pfk	A			KINASE	3pfk				KINASE
2paz				PLASTO	1plc				PLASTO
1azu				PLASTO	2aza	A			PLASTO
7pcy				PLASTO	7rxn				PLASTO
4rxn				RDX	1rdg				RDX

file	range		foldclass	file	range		foldclass
6rxn			RDX	1lmb	3		REPRESSOR
1r69			REPRESSOR	2cro			REPRESSOR
1rhd	1	146	RHD	1rhd	152	293	RHD
1cse			SBT	1sbt			SBT
1tec	E		SBT	2prk			SBT
1ton			S-PROT	2pka			S-PROT
2ptn			S-PROT	2trm			S-PROT
4cha	A		S-PROT	3est			S-PROT
1hne	E		S-PROT	1sgt			S-PROT
2sga			S-PROT	3sgb	E		S-PROT
2alp			S-PROT	2abx	A		TOX
1nxb			TOX	1ctx			TOX
4rhv	1		VIRUS	4rhv	2		VIRUS
4rhv	3		VIRUS	4sbv	A		VIRUS
2mev	1		VIRUS	2mev	2		VIRUS
2mev	3		VIRUS	2tbv	A		VIRUS
2stv			VIRUS	2plv	1		VIRUS
2plv	2		VIRUS	2plv	3		VIRUS
1r1a	1		VIRUS	1r1a	2		VIRUS
1r1a	3		VIRUS	3hvp			VIRUS-PROT
2rsp	A		VIRUS-PROT	7wga	A	1 43	WGA
7wga	A	44 86	WGA	7wga	A	87 129	WGA
7wga	A	130 171	WGA	9wga	A	1 43	WGA
9wga	A	44 86	WGA	9wga	A	87 129	WGA
9wga	A	130 171	WGA	4xia	A		XIA
1xya			XIA				

The second section specifies the steps used in computing and evaluating the similarity tables used in the experiment, including the source code for the procedure that translated raw accepted point mutations (observed substitutions rates) into a similarity function. The third section defines each of the amino-acid similarity functions used in the experiment. And the fourth section provides the similarity tables generated for the substitution table that resulted in the highest predictive accuracy score.

A.2 Steps Used to Compute the Similarity Tables for Experiment Three

The following steps were used in the generation of the similarity tables used in experiment three. The various context function used in each similarity functions is specified in the next section. The source code used to translated a table of substitution counts into a similarity table is also provided. This similarity function computation is based on the computation used by Dayhoff in her 1972 paper on amino acid substitution patterns[58]. It is modified to allow for the asymmetric substitution rates that are possible when context is taken into account. Each amino-acid substitution pair (A, B) generates an entry for $A \rightarrow B$ and $B \rightarrow A$ her tables are symmetric. Because amino acids A and B may be in different contexts these two entries many get partitioned into different context-specific substitution tables and thus result in asymmetric substitution counts tables, and then asymmetric similarity tables. The source code shown at the end of this section reflects this variant of the basic Dayhoff transformation.

Steps used in similarity table generation:

1. The sequence ranges shown in the first section are extracted from Pdb data files taken from the Brookhaven Protein Databank.

2. Proteins from this dataset with the same fold class labels are aligned using the Needleman and Wunch optimal string alignment algorithm[57]. The identity cost function is used with 1.0 assigned to sequence matches and 0.0 assigned to mismatches. A gap open penalty of 3.0 is used, along with a gap extension penalty of 0.1.

3. These alignment are used to generate a set of observed amino acid substitutions. These substitution pairs are partition using each of the amino acid context functions shown in the next section.

4. Each partition generated by a context function, is compiled into an Accepted Point Mutation (APM) table which simply records the number of times each amino acid was observed to be paired with another amino acid in the specified amino-acid context.

5. The Dayhoff transformation whose source code is shown below is applied to each partition associated with context function.

6. When a particular similarity function is used in the experiment, the function itself serves as an index to these tables. So, for example, the Lp3n3 tables shown in section four would be applied to a pair of amino acids as follows: The acid from the known protein is passed to the Lp3n3 function which returns an integer 0 through 8. This selects among the nine similarity tables. Then the known amino acid type is translated into a it canonical id (a number between 0 and 19) and used a the row index for the table, the column index is computed from the canonical id for

the amino acid type in the unknown protein. The number selected by this process is the context-specific "similarity" of these two amino acids according to the Lp3n3 context function.

7. This similarity is again used in the Needleman and Wunch optimal string alignment algorithm when determining the closest match for an unknown protein sequence.

8. The accuracies reported in experiment three are computed by aligning all protein sequences in the dataset shown in section one with each other. Each protein in turn is assumed to be the "unknown" protein and all other proteins are assumed to be "known." The closest matching "known" protein to the "unknown" protein is assumed to be in the same fold class as the unknown protein. If this is correct then that protein is scored as a 1.0 otherwise it is scored as a 0.0. The average of the scores for each protein taken in turn as the unknown protein is averaged, and the result is presented as the accuracy obtained using that particular amino-acid similarity function. These are the accuracy scores reported in Figure 6.3 and Figure 6.4.

```
;;; ------------------------------------------------------------------------
;;; This procedure accepts a 20 by 20 APM matrix and returns a 20 by 20
;;; similarity matrix using the conversion outlined in Dayhoff's 1978 paper
;;;
;;;
;;; Formulas used in the Dayhoff conversion of substitution rates to
;;; similarity matrixes.  Formulas are derived from:
;;;   ``A Model of Evolutionary Change in Proteins''
;;;   M. O. Dayhoff, R. V. Eck, and C.M. Park
;;;
;;; Key:
;;;
;;;   Aij   (Accepted point mutation matrix)
;;;
;;;      This is the observed substitution counts for a set of aligned
;;;      sequences.  E.g. fig. 9-3
;;;
;;;
;;;   m(j)  (Relative Mutability)
;;;
;;;      The mutation rate for a residue relative to the number of occurrences
;;;      of that residue.
;;;      m(j) = Change(j)/Exposure(j), where
;;;         Change(j)    = Num. times j is matched w. non-j res. over all
;;;                        aligned seq. in dataset.
```

```
;;;         Exposure(j)  = Sum over all aligned seq. pairs:
;;;                            Mut_Rate * Num_j_res_in_pair
;;;         Mut_Rate     = Total_Num_Mutations_In_Aligned_Pair /
;;;                        Num_Links_In_Pair * 100
;;;
;;;
;;;   Fi    (Normalized freq. of res. in Accepted point mutation matrix)
;;;
;;;      Approx. avg. composition of family * mut_rate
;;;      Fi = sum_bent_Aij(i) / sum_i sum_j Aij / mj(i) * scaling_factor
;;;
;;;
;;;   Mij   (Mutation probability matrix)
;;;
;;;      Probability that the amino-acid in col j is replaced by acid in row i.
;;;
;;;      Mij = L m(j) * Aij  /  Sum_i Aij        (For non-diagonal ele)
;;;      Mjj = 1 - L * m(j)                      (For diagonal ele)
;;;
;;;      Avg. Changes = 100 * Sum_i Fi Mii
;;;      Set L st. expected PAM = 100 - Avg_Changes
;;;
;;;
;;;   M-pa  (PAM Adjusted Mutation prob matrix)
;;;
;;;
;;;      PAM adjustment for testing vs. training evolutionary distances
;;;
;;;      M-pa = Mij ^ 4
;;;
;;;
;;;   Rij   (Relatedness Odds Matrix)
;;;
;;;      The odds that a particular paring occurred as a mutation
;;;      divided by the odds that it occurred by chance (not a pair).
;;;      (by ``chance'' takes into account relative occurrence of acids)
;;;
;;;      Rij = Mij/Fi    (Rij = Rji)
;;;
;;;
;;;   MRij  (Modified odds matrix)
;;;
;;;      The log of the odds matrix.
```

```
;;;      This was used by Dayhoff as a similarity measure for seq. matching.
;;;
;;;      MRij = ln Rij
;;;
;;;   Aij, m(j) given.  ->Fi  ->Mjj ->L ->Mij  ->Rij ->MRij
;;; ------------------------------------------------------------------------

(defvar *dayhoff-relative-mutabilities*
 (build-res-prop-vector :size 21 :plist '(
  Ser 1.49   Asp  .90   Gly  .48
  Met 1.22   Thr  .90   Phe  .45
  Asn 1.11   Ins  .84   Arg  .44
  Ile 1.10   Val  .80   Leu  .38
  Glu 1.02   Lys  .57   Tyr  .34
  Ala 1.00   Pro  .56   Cys  .27
  Gln  .98   His  .50   Trp  .22 )))

;;; Updates and returns min and max to reflect values in 'array'
;;; Ignores the last row and col of array.
(defun day-array-min-max (array min max &aux (size 21))
  (unless (equal (array-dimensions array) (list size size))
    (error "unexpected array dimensions"))
  (domat (e array i j) (when (and (numberp e) (< i (1- size)) (< j (1- size)))
      (unless min (setf min e max e))
      (if (< e min) (setf min e))  (if (> e max) (setf max e)) ))
  (format T " [~A ~A].~%" min max)
  (values min max))

;;; Uses min and max to scale the _entire_ vect
(defun day-centi-scale-vect (vect min max &aux ele)
 (unless (and min (> max min))
   (error "Lousy min max vals.  2342"))
 (loop for i from 0 to (1- (length vect))   for ele = (svref vect i) do
   (setf (svref vect i)
     (if (keywordp ele)  ele   (round (/ (* 100.0 (- ele min)) (- max min))))))
 vect)


;;; ------------------------------------------------------------------------
;;; DAY-NORM matrix
;;; MAIN TRANSFORMATION PROCEDURE.
;;; WARNING: Destroys contents of matrix, by zeroing diagonal
;;; ------------------------------------------------------------------------
```

```
(defun day-norm (matrix &key (emr .4) (pam-adj 4) (verbose nil)
                 &allow-other-keys &aux
                 bent-sum1 bent-sum2 tmp
                 sum Fi-sum Fi-mj-sum
                 Aij-sum size Aij mj Fi L Mij M-pa Rij MRij)
  (declare (special qq))
 (domat (ele matrix row col) (incf (aref matrix row col)))
   (setf
    Aij     matrix                           ; Accepted Point Mutation counts
    EMR     EMR                              ; Expected Mutation Rate [0-1]
    pam-adj pam-adj                          ; PAM adj. for training -> testing
    size    (array-dimension Aij 0)          ; Size of square Aij matrix
    Aij-sum 0                                ; Sum of ele in Aij matrix
    mj      *dayhoff-relative-mutabilities*  ; Relative Mutabilities vector
    Fi      (make-array size)                ; Normalized res freq. in Aij
    Fi-sum  0
    Fi-mj-sum 0
    L       nil                              ; Proportionality constant
    Mij     (make-array (list size size))    ; Mutation Probability Matrix j->i
    M-pa    nil                              ; PAM adjusted Mij
    Rij     (make-array (list size size))    ; Relatedness Odds Matrix
    MRij    (make-array (list size size)))   ; Modified Relatedness Odds Matrix

   (cond
    (t (format t "DAY-ASYMETRIC-NORM: 0")
       (setf bent-sum1 #'day-unbent-sum-row bent-sum2 #'day-unbent-sum-col)))
    ;; Set diagonal elements to 0 just in case it is needed.
    (dotimes (i size) (setf (aref Aij i i) 0))

 (format t "1")
   (dotimes (i size)                         ; Compute Aij-sum (lower triangle)
    (dotimes (j i)
     (incf Aij-sum (aref Aij i j))))
 (format t "2")
   (setf sum 0)                              ; Compute Fi
   (dotimes (i size)
    (setf (aref Fi i)
     (/ (float (funcall bent-sum1 Aij i size)) (* Aij-sum (aref mj i))))
    (incf sum (aref Fi i)))
   (dotimes (i size)                         ; Normalize Fi
    (setf (aref Fi i) (/ (aref Fi i) sum)))
 (format t "3")
    (dotimes (i size)                        ; Compute Fi-sum & Fi-mj-sum
```

```
    (incf Fi-sum (aref Fi i))
    (incf Fi-mj-sum (* (aref Fi i) (aref mj i))))
(format t "4")
   (setf L (/ (float emr) Fi-mj-sum))      ; Compute L
(format t "5")
  (dotimes (i size)                        ; Compute Mij
   (dotimes (j size)
    (let ((numerator (* L (aref mj j) (aref Aij i j)))
  (denom     (funcall bent-sum2 Aij j size)))
     (setf (aref Mij i j) (e/ numerator denom)) )))
(format t "6")
  (dotimes (j size)                        ; Compute Mjj elements
   (setf (aref Mij j j)
     (- 1 (* L (aref mj j)))))
(format t "7")
  (setf M-pa (e-mat-exp Mij pam-adj))      ; Compute M-pa
(format t "8")
  (dotimes (i size)                        ; Compute Rij
   (dotimes (j size)
    (setf (aref Rij i j)
  (e/ (aref M-pa i j) (aref Fi i)))))
(format t "9")
  (dotimes (i size)                        ; Compute MRij
   (dotimes (j size)
    (setf tmp (aref Rij i j)
  (aref MRij i j)
  (e-log (aref Rij i j)))))
(format t ".~%")
  (when verbose
   (format T "Dayhoff Transform --  EMR ~D,  PAM-adj ~D~%~%APM Matrix (Aij)~%"
   emr pam-adj)
;;   (display-res-matrix Aij :order *dayhoff-res-order*)
   (vtab Aij)
   (format T "   Aij sum ~D, size ~D,  Fi sum ~D,  Fi*mj sum ~D,  L ~D~%~%"
   Aij-sum size Fi-sum Fi-mj-sum L)
(read-char)
   (format T "Normalized frequences of residues in APM matrix (Fi)~%")
   (dotimes (i size)
    (format T "   ~A  ~D~%" (bio-id2short-name i) (aref Fi i)))
(read-char)
   (format T "Relative Mutabilities (mj)~%")
   (dotimes (i size)
    (format T "   ~A  ~D~%" (bio-id2short-name i) (aref mj i)))
```

```
 (read-char)
    (format T "~%Mutation Rate Matrix (Mij)~%")
 ;;  (display-res-matrix Mij :order *dayhoff-res-order* :scale 10000 :clip nil)
    (vtab Mij :scale 10000 :clip nil)
    (format T "~%~%PAM Adjusted Mutation Rate Matrix (M-pa)~%")
 ;;  (display-res-matrix M-pa :order *dayhoff-res-order* :scale 100 :clip T)
    (vtab M-pa :scale 100 :clip T)
    (format T "~%~%Relatedness Odds Matrix (Rij)~%")
 ;;  (display-res-matrix Rij :order *dayhoff-res-order* :scale 10 :clip T)
    (vtab Rij :scale 10 :clip T)
    (format T "~%~%Modified Relatedness Odds Matrix (MRij) Sim Matrix~%")
 ;;  (display-res-matrix MRij :order *dayhoff-res-order* :scale 100 :clip T))
    (vtab MRij :scale 100 :clip T))
   MRij)


;;; Sums the Ith row of Aij except Aii
(defun day-unbent-sum-row (Aij i size &aux (sum 0))
  (dotimes (j size)
   (unless (eql i j) (incf sum (aref Aij i j))))
  sum)

;;; Sums the Jth col of Aij except Ajj
(defun day-unbent-sum-col (Aij j size &aux (sum 0))
  (dotimes (i size)
   (unless (eql i j) (incf sum (aref Aij i j))))
  sum)

;;; Adds the Ith row and col in lower triangle of Aij
(defun day-Aij-i-sum (Aij i size &aux (sum 0))
  (dotimes (j size)
   (cond ((< j i) (incf sum (aref Aij i j)))
         ((> j i) (incf sum (aref Aij j i)))))
  sum)
```

A.3 Amino-Acid Similarity Functions Used in Experiment Three

In experiment three PI selected between a number of different amino acid function similarity functions. Each of these function are described in this section. Similarity function that were computed from a context function are described by specifying the context function. These context functions were used to generate the similarity table by partitioning substitution pairs as described in the previous section.

The EXPOSURE3 function is based on the amino-acid's exposure to external solvent molecules. The number of square angstroms of solvent exposure is calculated from each amino acid by a program called DSSP[70]. This real-valued parameter is broken into three regions: exposure levels less than 10, those between 10 and 100, and those greater than or equal to 100.

The IDENTITY similarity function simply returns a 1.0 if the two amino acids are of the same type, and 0.0 otherwise.

The BASIC similarity function assumes that all amino acids are in one standard context. This is the degenerate context function, and it is equivalent to simply using Dayhoff's method[58] for taking substitution counts and converting them into a similarity function as described in the previous section.

The STR0 context function splits amino acids into four categories: α-helix, β-sheet, turn, and coil. This function is computed using the DSSP secondary chain specification. All forms of helix ("H", "G", and "I") are labeled context 0. Sheets, strands, and β-bridges ("E", and "B") are all labeled as context 1. Turns ("T") are labeled as context 2, and all others ("S" and " ") are labeled as context 3.

The DM78 similarity function is the table reported by Dayhoff in 1978. It way computed in the same way as our BASIC similarity function was computed. The only difference is the raw data used in generating the table.

The PO9 context function splits the amino acids into nine categories. The atoms of an amino acid are inspect to determine if any are bonding with atoms from other acids. The atoms are assumed to be bonded if they are less than or equal to 3.5 angstroms apart. Bonds emanating from atoms in the backbone of the amino acid are ignored. A bond is labeled as "polar-polar" if both atoms that make up the bond are polar. The number of "polar-polar" bonds are counted and used to select on of three categories: zero, one, or many. The number of all other bonds are also counted and used to select from the same three categories. The cross product of these two three element sets define the nine PO9 contexts returned by the function.

The LP3N3 context function also splits amino acids into nine categories. It also splits the non-backbone bonds into two categories. This function simply looks are the polarity of the atom in the amino acid whose context is being computed. This splits bonds in to polar and non-polar. Each of these sets are counted as above and used to select from: zero, one, or many. As before the cross product of these two sets has nine elements, these are the possible values for the LP3N3 context function. The context number is computed by taking the number of polar contacts compressing them to 0, 1, or 2 and multiplying by three. Then the number of non-polar contacts is also compressed to 0, 1, or 2, and add in. The result is an integer between 0 and 8.

The STDID similarity function partitions amino acids into 20 different contexts based on the amino acid type. This table is used as a control for the experiment.

Each of the MOD* functions were used as a control in the experiment. They are pseudo random context functions, that test the degree of degradation in accuracy that occurs as a result of using smaller training sets that result when data is partitions into

separate contexts. Each of the MODX functions splits the substitutions pairs into X partitions. This is done by taking the `mod` base X of the amino acids index within the protein.

A.4 The Lp3n3 Similarity Table Generated in Experiment Three

Here is an example of the similarity matrices generated in Experiment three. This is the LP3N3 table. The context number is computed as described in the previous section. The tables are labeled 0 through 8 corresponding to each of the nine contexts. The rows and columns of each table represent the 20 amino acids in alphabetic order according to their one-letter amino acid type.

[T:0 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	21	-34	18	18	-6	17	-1	16	8	-11	15	16	10	18	-4	15	18	16	-57	-20	17
CYS	-34	346	-46	-24	-57	-57	-28	-50	-43	-94	-44	-43	-48	-43	-88	-31	-40	-37	-x	-70	-70
ASP	25	-32	76	33	-13	19	2	19	15	-12	21	24	18	27	2	21	20	21	-46	-24	16
GLU	24	-28	31	50	-14	20	8	20	17	-9	19	24	17	27	8	20	23	19	-48	-20	15
PHE	0	-73	-11	-3	210	-10	-22	7	-24	-4	13	-8	-27	-2	-47	5	-5	-12	-73	-28	-28
GLY	-9	-77	-12	-13	-47	1	-31	-15	-26	-50	-16	-10	-31	-14	-43	-11	-15	-18	-91	-58	-17
HIS	-12	-48	-9	-8	-40	-31	334	-12	-2	-28	5	23	-27	-20	-17	-4	-9	-16	-x	-42	-2
ILE	19	-18	18	16	11	13	2	41	12	4	20	18	6	18	-13	16	17	28	-51	-20	34
LYS	8	-66	9	19	-27	5	-3	6	124	-26	8	8	-4	21	-3	11	12	7	-72	-48	10
LEU	-17	-88	-19	-18	-20	-29	-53	3	-33	102	9	-11	-36	-19	-51	-10	-15	-7	-81	-66	-17
MET	20	-20	16	18	12	16	1	20	9	10	36	19	11	18	6	17	20	21	-35	-13	11
ASN	20	-27	22	21	3	15	15	19	11	-7	16	39	9	22	2	17	24	17	-67	-25	9
PRO	21	-37	9	32	10	32	4	28	11	-14	22	16	410	19	-20	27	12	15	-26	-21	18
GLN	25	-8	24	27	-7	16	11	19	13	-7	22	22	7	75	11	20	26	21	-70	-27	22
ARG	-1	-48	-7	-2	-25	-20	-39	-5	-7	-51	1	1	-21	2	253	5	-8	-17	-x	-40	6
SER	21	-26	24	21	0	20	3	18	14	-3	17	20	19	22	8	18	23	19	-45	-7	23
THR	25	-13	25	30	-4	20	-8	22	20	-7	23	29	11	31	3	27	96	20	-57	-26	31
VAL	23	-27	19	20	-3	16	-6	35	3	10	22	22	0	20	-11	21	21	81	-57	-17	20
TRP	-58	-93	-58	-39	-69	-56	-68	-61	-83	-x	-42	-58	-61	-38	-x	-48	-59	-64	332	-37	-29
TYR	-22	-x	-23	-31	-20	-27	-54	-19	-44	-57	-30	-20	-55	-26	-40	-3	-13	-23	-45	254	-15
INS	31	38	9	22	-19	20	-14	23	16	-20	16	19	10	47	24	29	17	23	-72	-26	321

[T:1 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	3	-42	-1	0	-28	-13	-17	-2	-10	-31	-3	1	-8	3	-23	-2	-1	0	-80	-39	5
CYS	-28	317	-43	-33	-67	-91	-67	-30	-66	-87	-19	-33	-44	-53	-81	-36	-32	-47	-x	-x	-37
ASP	10	-76	138	22	-27	0	2	8	4	-24	7	22	4	14	-19	12	13	4	-66	-44	-9
GLU	17	-36	25	77	-7	-9	-7	12	15	-18	12	19	8	15	-1	14	16	13	-84	-30	12
PHE	1	-77	-22	-12	174	-42	-34	1	-36	-35	0	0	-18	-22	-49	-2	-12	-3	-76	-25	-18
GLY	-45	-33	-28	-5	-48	516	-47	-41	-32	-24	-19	-2	-40	-21	-23	-19	-28	-33	-41	-36	18
HIS	-5	-53	6	-6	-68	-33	335	-18	-11	-65	-20	-19	-17	45	-48	-3	-17	-20	-91	-81	-5
ILE	22	-32	18	20	-6	-6	-1	35	5	5	24	21	12	17	-10	19	20	28	-59	-20	27
LYS	9	-78	3	8	-40	-32	-10	-3	123	-41	2	7	-7	12	-10	6	6	-9	-91	-59	4
LEU	-15	-91	-23	-20	-35	-53	-41	-2	-37	75	6	-17	-35	-25	-53	-12	-24	-8	-x	-74	-27
MET	22	-35	17	20	5	-4	-6	29	8	16	41	23	14	18	-9	20	18	24	-45	-29	18
ASN	3	-41	12	11	-13	5	-13	-2	-5	-27	1	68	-3	7	-26	5	9	-1	-78	-39	22
PRO	-1	-77	-2	-1	-41	-32	-38	-10	-21	-46	-11	-5	55	-3	-42	2	-3	-14	-70	-62	-25
GLN	13	-66	16	19	-19	-8	-5	4	4	-28	7	13	-3	110	-3	13	14	4	-78	-25	30
ARG	-14	-52	-14	0	-83	-44	-44	-27	-7	-72	-32	-16	-46	-3	234	-7	-14	-26	-92	-64	-32
SER	16	-45	20	15	-7	1	-3	14	17	-7	13	21	13	18	1	15	20	15	-52	-14	30
THR	20	-50	17	25	-16	-10	-18	15	11	-16	16	24	11	23	-13	21	84	14	-69	-32	26
VAL	15	-56	9	9	-17	-14	-16	21	-5	-10	14	13	0	13	-19	12	12	39	-81	-34	9
TRP	-85	-x	-55	-72	-79	-x	-x	-92	-x	-x	-x	-62	-73	-68	-x	-69	-64	-x	329	-39	-25
TYR	-28	-x	-30	-37	-29	-59	-52	-37	-61	-83	-45	-26	-51	-29	-73	-20	-27	-37	-99	236	-39
INS	5	-22	0	19	-26	-31	39	23	-19	-9	18	27	9	4	-43	14	15	-1	-69	-61	396

[T:2 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	251	25	18	12	-5	-14	10	15	0	0	13	24	19	33	18	11	9	15	-21	-27	51
CYS	-43	592	-32	-23	-46	-72	-12	-34	-49	-82	-1	-24	-36	-18	-42	-39	-43	-48	-42	-68	94
ASP	-12	49	236	23	-16	-14	37	-16	-2	-35	-17	-2	-23	7	-17	-15	-5	-14	-78	-52	6
GLU	6	-15	9	51	-12	2	-7	0	21	-14	1	5	10	12	14	1	10	3	-56	-35	-4
PHE	-5	-58	-6	-4	55	-30	-15	-3	-20	-22	-1	-2	-11	-2	-37	-1	-6	-4	-61	-8	8
GLY	4	-7	-19	-14	-17	488	59	3	-19	-36	-2	2	-8	-12	-41	9	7	-14	-15	-37	56
HIS	-7	-56	1	-16	-28	-21	279	-29	-12	-60	-19	3	-43	-3	-35	-12	-19	-30	-90	-45	-20
ILE	19	-24	17	17	6	6	1	18	12	10	16	16	15	18	1	14	18	22	-43	-17	13
LYS	0	-57	1	12	-35	-15	-4	-7	102	-28	-5	-1	-9	4	6	-1	-1	-7	-80	-52	-15
LEU	0	-68	-2	-1	-17	-27	-30	8	-12	47	12	2	-13	2	-33	1	-2	6	-75	-50	-12
MET	24	-6	22	21	15	6	-3	23	19	26	23	21	20	23	7	20	23	24	-32	-18	12
ASN	-3	-35	-1	-12	-27	-12	23	-15	-19	-18	-14	77	-14	-10	1	-15	-9	-16	-71	-11	0
PRO	1	-60	-2	-6	-30	-15	-23	-11	-10	-37	-8	-5	58	-3	-21	-3	-2	-9	-83	-57	-26
GLN	-1	-72	8	7	-28	-12	10	-5	9	-18	-6	0	-11	70	4	-6	2	-4	-56	-35	-16
ARG	-8	-81	10	-11	-55	-27	-15	-11	3	-45	-12	-13	-21	2	263	-8	2	-11	-96	-51	33
SER	21	1	21	19	32	13	22	18	15	2	18	20	31	20	17	23	25	20	-8	5	31
THR	10	-19	14	9	-9	-9	-6	4	6	-10	3	10	19	14	3	8	83	6	-76	-24	22
VAL	12	-35	10	11	-5	-3	-4	12	4	-2	8	8	7	11	-5	7	11	18	-60	-30	4
TRP	-52	-90	-48	-45	-52	-57	-84	-47	-70	-80	-53	-32	-56	-70	-78	-37	-53	-58	221	-52	-6
TYR	-16	-79	-21	-12	-6	-40	-17	-18	-29	-50	-25	-11	-36	-24	-53	-5	-14	-25	-55	142	-2
INS	15	63	19	21	9	-6	33	17	9	-12	23	19	17	23	13	14	14	20	9	-9	417

[T:3 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	377	-2	-1	-9	-56	-39	53	10	2	-65	25	7	-51	-10	15	7	14	-33	-69	-72	28
CYS	-72	233	-x	-79	-x	-x	-x	-55	-x	-x	-51	-68	-x	-x	-x	-72	-93	-79	-x	-x	-x
ASP	5	-70	65	18	-41	-38	-22	11	-22	-52	8	12	-25	7	-41	16	5	-4	-x	-67	1
GLU	3	-72	15	88	-43	-32	-42	8	-7	-61	5	-1	-22	5	-43	6	0	-4	-99	-74	-4
PHE	-69	12	-54	-43	512	-x	-40	-38	-84	-x	28	-44	-89	-40	-78	-53	-70	-76	-69	-96	36
GLY	-29	-10	-40	-34	-77	474	-45	-28	-75	-x	64	-29	-49	-32	-78	1	-28	-22	-83	-94	26
HIS	-17	-94	-11	-26	-68	-72	243	-31	-57	-87	-27	-24	-51	-24	-69	0	-31	-20	-x	-x	-33
ILE	-22	-7	2	-22	47	-80	-35	347	-59	-18	13	8	-25	-2	-65	-9	-30	-29	-76	-51	21
LYS	-22	-x	-16	-3	-80	-48	-75	-9	177	-84	-17	-11	-57	-2	-55	-3	-12	-27	-x	-84	-28
LEU	-30	-29	-77	-68	-x	-x	-73	-29	-x	476	-4	-66	-80	-65	-x	-53	-66	-99	-x	-83	1
MET	-9	-5	3	8	-50	-29	-45	12	7	-51	244	19	11	-6	-37	11	4	-4	-97	-53	66
ASN	14	-62	23	21	-29	-23	-7	23	-11	-34	16	67	-18	15	-25	28	23	6	-66	-45	6
PRO	-54	26	-40	-29	-64	-98	-26	-24	-68	-x	39	-31	508	-26	-63	-39	-55	-61	-55	-81	48
GLN	1	-75	8	8	-46	-32	-25	-4	-3	-52	3	14	-39	97	-35	10	9	4	-x	-73	-7
ARG	-37	-x	-51	-23	-73	-61	-78	-32	-38	-x	-17	-32	-61	-16	210	-4	-14	-43	-x	-x	-68
SER	-7	-85	-10	-12	-47	-39	-36	-10	-35	-55	-8	-6	-29	-12	-51	-9	-4	-17	-x	-62	-7
THR	9	-87	9	10	-40	-33	-33	9	-24	-46	14	15	-18	6	-34	22	38	-3	-93	-44	0
VAL	-33	44	-20	-11	-44	-76	-7	-6	-47	-86	51	-12	-52	-8	-43	-20	-33	483	-35	-60	63
TRP	-x	-x	-x	-x	-x	-x	-x	-98	-79	-x	-x	-92	-x	-99	-x	-98	-x	-x	286	-x	-x
TYR	-58	-x	-79	-77	-47	-x	-x	-55	-98	-x	-41	-68	-x	-35	-x	-54	-66	-72	-x	241	-90
INS	-31	46	-19	-9	-42	-73	-5	-5	-45	-83	51	-11	-50	-7	-41	-19	-31	-37	-33	-58	476

[T:4 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	374	-42	-19	-10	-35	-43	45	1	-56	-26	-2	-27	-61	49	13	-21	-35	-34	-86	-40	15
CYS	-35	186	-51	-57	-96	-92	-x	-33	-87	-x	-27	-43	-95	-52	-x	-19	-36	-57	-x	-x	55
ASP	10	-75	88	32	-39	-16	-5	9	-8	-39	10	26	-11	11	-34	19	13	5	-91	-47	-19
GLU	13	-77	27	89	-32	-24	-18	15	-2	-39	12	17	-21	22	-14	14	14	0	-70	-43	-14
PHE	-32	-2	-25	-17	554	-81	-11	-12	-55	-93	36	-14	-50	-14	-55	-14	-29	-43	-60	-75	65
GLY	-38	-25	-10	-25	-26	519	-28	-22	-37	-x	21	-22	-3	-22	-32	-15	-5	-48	-81	-88	42
HIS	-20	-x	-18	-5	-44	-66	195	-16	-22	-68	-27	-9	-58	-3	-52	-5	-26	-37	-x	-84	-29
ILE	12	5	-10	-6	26	-57	-3	395	22	-27	26	-2	-34	-3	-38	1	-10	-23	-49	-56	55
LYS	-13	-94	-7	0	-59	-46	-45	1	130	-63	-3	1	-33	-7	-39	5	-4	-16	-98	-61	-28
LEU	-28	-56	-15	-21	-87	-73	-55	-21	-85	501	-4	-44	-82	18	-91	-28	-40	-71	-x	-80	13
MET	21	-61	9	17	1	-6	2	43	-3	-21	160	24	15	32	-9	31	29	23	-87	-39	50
ASN	21	-64	32	26	-21	-5	3	23	4	-27	24	63	-11	18	-20	27	28	13	-73	-35	6
PRO	-40	-22	-32	-1	-61	-65	-27	-23	-63	-54	18	19	510	-25	-67	-21	-36	-10	-27	-87	43
GLN	5	-92	2	14	-29	-27	-22	6	-2	-48	9	2	-11	93	-21	5	6	0	-65	-50	-27
ARG	-14	-x	-10	-19	-76	-84	-56	-33	-30	-88	-6	-17	-50	-22	218	-13	-25	-36	-x	-85	-63
SER	-1	-63	-5	-4	-36	-25	-26	-2	-21	-42	-2	-2	-19	-4	-33	-5	1	-7	-92	-49	-16
THR	8	-66	2	7	-37	-26	-23	10	-12	-42	15	10	-10	11	-31	14	30	7	-92	-52	-9
VAL	4	30	8	14	-14	-42	20	19	-19	-55	57	17	-16	17	-21	19	6	511	-27	-39	87
TRP	-x	-x	-x	-98	-x	-x	-x	-79	-x	-x	-64	-x	-84	-89	-x	-95	-x	-x	225	-x	-x
TYR	-58	-x	-46	-43	-40	-89	-95	-55	-93	-82	-64	-30	-84	-55	-88	-35	-47	-56	-87	220	-39
INS	23	-4	11	6	-23	-21	-13	23	-14	-42	19	20	-31	13	-34	30	26	6	-65	-38	101

[T:5 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	138	68	5	5	28	14	11	12	-2	16	6	6	12	7	-8	5	4	2	-16	13	33
CYS	-12	517	-5	2	-28	-44	11	2	-18	-57	33	1	-18	6	-22	-8	-9	-17	-26	-24	78
ASP	28	-26	45	30	-5	14	25	26	27	11	25	28	31	31	22	26	28	23	-19	6	18
GLU	26	-20	29	28	0	19	19	24	25	7	20	24	16	28	14	22	25	25	-36	7	19
PHE	19	52	25	30	492	-10	39	30	14	-22	52	29	14	33	10	22	22	15	5	7	95
GLY	22	54	27	33	7	484	41	32	17	-18	53	31	17	35	13	25	25	18	8	11	96
HIS	11	-43	15	13	-13	1	82	10	7	-26	3	18	0	12	-11	9	9	11	-63	-11	3
ILE	37	62	38	38	25	19	47	93	34	23	36	38	32	39	30	36	37	35	23	28	62
LYS	14	-46	15	16	-21	-3	7	12	41	-11	14	12	3	15	6	11	14	11	-44	-20	6
LEU	10	44	16	22	-6	-20	31	22	4	506	48	21	4	25	1	13	13	5	-4	-2	92
MET	13	-20	15	12	1	-1	9	11	16	13	8	10	5	13	0	9	11	15	-39	-3	7
ASN	17	-5	20	17	-6	16	13	15	16	-1	12	16	10	17	8	14	16	20	-33	1	14
PRO	24	55	29	33	9	17	42	32	19	-15	50	32	455	35	15	26	26	20	10	13	91
GLN	23	-29	23	24	-6	9	20	19	19	5	15	19	10	26	14	18	21	22	-51	2	16
ARG	19	-37	14	31	-34	-13	10	10	21	-9	26	12	11	19	150	19	23	15	-64	-30	-6
SER	41	65	42	42	31	27	50	41	40	17	38	41	38	42	36	88	41	40	29	34	56
THR	6	-30	6	5	-16	-4	-2	3	6	-17	0	3	0	6	-6	2	5	8	-57	-9	2
VAL	39	67	42	44	25	17	53	43	35	5	49	43	34	45	31	39	40	344	24	28	86
TRP	-30	-96	-32	-31	-59	-58	-54	-24	-33	-60	-14	-25	-41	-32	-65	-23	-27	-37	102	-41	-18
TYR	-15	-66	-18	-19	-12	-35	-34	-11	-30	-56	-18	-15	-33	-15	-43	-11	-14	-16	-60	33	-13
INS	39	67	42	43	26	18	53	43	36	6	47	42	34	44	31	39	40	37	25	29	317

[T:6 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	227	18	18	20	-4	-24	-6	23	-12	-35	53	22	-6	20	3	17	8	3	-32	-35	59
CYS	-69	318	-58	-52	-82	-x	-86	-47	-96	-x	4	-46	-88	-51	-75	-57	-72	-79	-x	-x	0
ASP	1	-88	10	-3	-39	-34	-33	-3	-24	-48	-11	5	-24	-4	-42	4	-1	-1	-87	-63	-20
GLU	10	-x	17	36	-33	-1	-40	11	-11	-28	-2	8	-10	-1	-25	10	4	4	-80	-34	-3
PHE	-30	-19	-20	-16	324	-69	-48	-11	-56	-81	36	-10	-49	-14	-37	-20	-33	-40	-73	-79	34
GLY	-26	-14	-16	-11	-40	322	-43	-7	-52	-76	39	-6	-44	-10	-32	-16	-29	-36	-68	-74	38
HIS	3	-53	2	-18	-54	-19	264	-14	-39	-26	18	35	-56	-13	-50	-7	-13	-37	-96	-39	12
ILE	13	20	20	22	-1	-20	-3	197	-9	-32	50	23	-4	22	5	19	10	5	-29	-31	58
LYS	-15	-4	-6	-2	-29	-53	-32	2	315	-65	46	3	-34	-1	-22	-6	-18	-25	-58	-63	46
LEU	-42	-30	-32	-27	-56	-82	-59	-22	-68	325	27	-21	-61	-25	-48	-31	-45	-52	-85	-91	24
MET	14	20	20	22	0	-18	-2	23	-7	-30	155	23	-2	22	6	20	12	7	-28	-29	55
ASN	10	-89	14	6	-26	-20	-10	6	-17	-32	5	14	-19	8	-25	12	6	2	-88	-47	-2
PRO	-16	-5	-7	-3	-30	-54	-33	1	-42	-66	45	2	316	-2	-23	-7	-19	-26	-59	-64	45
GLN	16	-50	19	27	-31	-22	-11	13	-6	-47	15	16	2	33	-7	15	13	13	-78	-54	-14
ARG	-29	-x	-28	-21	-66	-67	-64	-17	-42	-78	-46	-25	-54	-9	64	-12	-28	-39	-x	-77	-56
SER	19	23	23	24	4	-12	3	25	-2	-23	41	24	3	24	10	63	16	12	-23	-24	53
THR	7	15	15	17	-7	-28	-10	20	-16	-40	53	20	-10	18	-1	14	254	-1	-36	-39	58
VAL	9	-11	1	7	-28	-47	-28	6	-9	-32	37	7	-26	25	15	23	-10	249	-61	-59	36
TRP	-85	-72	-74	-68	-99	-x	-x	-63	-x	-x	-10	-62	-x	-67	-91	-73	-89	-96	309	-x	-15
TYR	-50	-38	-40	-35	-64	-90	-68	-30	-77	-x	20	-29	-69	-33	-57	-39	-54	-61	-93	324	17
INS	3	12	11	14	-11	-32	-13	17	-21	-44	52	17	-14	15	-4	11	0	-5	-39	-43	268

[T:7 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	306	33	14	14	-13	-34	3	21	-21	-49	45	32	12	11	-17	14	17	-6	-32	-36	132
CYS	-x	343	-68	-85	-x	-x	-x	-53	-x	-x	-48	-79	-55	-44	-70	-61	-85	-73	-x	-x	41
ASP	-3	-74	15	4	-36	-29	-40	4	-26	-41	-1	8	-30	-3	-36	7	-5	-10	-94	-54	-14
GLU	23	-67	24	40	-17	-17	-15	24	-2	-42	20	24	-5	32	-5	23	17	13	-90	-48	-4
PHE	-38	13	-24	-22	418	-80	-29	-12	-62	-94	28	-15	-53	-22	-52	-29	-42	-47	-63	-75	43
GLY	-34	17	-20	-18	-43	416	-25	-8	-57	-89	31	-11	-48	-17	-48	-24	-37	-43	-59	-70	46
HIS	-17	-x	-19	-13	-39	-75	239	-17	-26	-78	-28	7	-32	-20	-45	-10	-37	-24	-67	-51	-41
ILE	8	51	17	18	-3	-29	12	297	-13	-42	49	21	-6	18	-7	15	5	0	-19	-27	69
LYS	-23	27	-10	-7	-32	-64	-15	2	410	-77	39	-2	-37	-7	-37	-14	-26	-31	-48	-59	55
LEU	-51	1	-36	-33	-60	-93	-41	-23	-74	418	18	-26	-65	-33	-64	-41	-54	-60	-75	-87	33
MET	10	52	18	19	-1	-25	14	25	-10	-39	255	21	-3	19	-4	17	8	3	-17	-25	67
ASN	-6	-84	1	-2	-37	-27	-29	0	-28	-46	0	3	-27	-10	-37	0	-5	-14	-72	-52	-13
PRO	-24	26	-11	-8	-33	-65	-16	1	-47	-78	38	-3	411	-8	-38	-15	-27	-33	-49	-60	54
GLN	3	-97	-4	1	-47	-36	-28	-9	-22	-62	-9	-5	-9	22	-28	0	0	-4	-97	-63	-15
ARG	-22	-x	-17	-18	-75	-67	-60	-25	-50	-74	-17	-15	-67	-14	62	-11	-27	-29	-x	-98	-39
SER	5	45	12	13	-6	-28	9	18	8	-41	33	14	-7	12	-9	128	4	-1	-22	-27	96
THR	1	46	11	13	-10	-37	7	21	-21	-51	50	17	-13	13	-13	9	353	-7	-26	-35	68
VAL	-4	43	7	9	-15	-43	2	17	-26	-57	48	13	-18	9	-19	4	-7	375	-30	-40	66
TRP	-94	-41	-79	-76	-x	-x	-84	-64	-x	-x	-20	-68	-x	-75	-x	-84	-98	-x	401	-x	-7
TYR	-59	-7	-44	-41	-68	-x	-49	-31	-83	-x	11	-34	-74	-41	-73	-49	-62	-68	-83	417	25
INS	-34	11	-23	-21	-44	-73	14	-13	-56	-57	17	-17	-17	-22	-48	-4	-16	-42	-60	-3	333

[T:8 9x21x21] Day-Lp3n3-Table

[x100]	ALA	CYS	ASP	GLU	PHE	GLY	HIS	ILE	LYS	LEU	MET	ASN	PRO	GLN	ARG	SER	THR	VAL	TRP	TYR	INS
ALA	302	59	31	33	17	-6	39	35	6	-18	50	33	14	33	21	31	24	20	10	-3	87
CYS	-46	455	-36	-30	-54	-88	-25	-27	-72	-x	10	-28	-60	-31	-49	-37	-50	-56	-59	-77	49
ASP	16	-61	28	22	-19	-3	-5	18	-8	-29	20	21	-3	13	5	18	16	12	-74	-28	14
GLU	20	-40	23	30	-12	-4	0	18	0	-19	20	18	2	22	0	18	17	12	-56	-39	8
PHE	-10	31	-1	4	452	-49	10	7	-34	-61	39	6	-23	3	-13	-2	-13	-18	-24	-40	78
GLY	-6	34	3	8	-14	449	13	11	-30	-57	42	9	-19	7	-9	2	-9	-14	-20	-36	81
HIS	-11	-x	-9	-20	-25	-31	69	-4	-39	-57	-13	-3	-37	-6	-32	-4	-12	-17	-92	-56	-8
ILE	28	60	32	33	19	-3	40	253	8	-15	48	34	16	33	23	32	26	22	11	-1	84
LYS	4	43	12	16	-4	-34	22	19	436	-46	47	18	-9	16	0	12	1	-4	-10	-26	86
LEU	-21	21	-12	-6	-29	-61	0	-3	-46	457	31	-4	-35	-7	-24	-12	-24	-30	-34	-52	70
MET	29	60	33	34	20	0	40	35	10	-13	185	34	18	34	24	33	27	24	12	1	80
ASN	2	-48	9	4	-26	-10	-10	4	-13	-24	10	8	-9	-1	-10	4	5	4	-63	-26	-9
PRO	3	42	11	15	-5	-35	21	18	-21	-47	47	17	437	15	-1	11	0	-5	-11	-27	86
GLN	20	-54	18	21	-8	-16	3	14	0	-21	19	15	8	26	7	17	15	12	-79	-31	13
ARG	-20	-93	-21	-16	-62	-57	-45	-20	-31	-63	-21	-18	-43	-17	11	-15	-23	-25	-x	-79	-37
SER	31	60	34	34	22	4	40	35	14	-8	40	34	20	34	26	46	29	26	14	4	72
THR	23	58	29	31	14	-10	37	33	2	-22	52	32	11	31	19	28	344	16	7	-6	89
VAL	19	55	26	28	10	-15	35	31	-2	-28	52	30	7	28	15	25	17	378	4	-10	90
TRP	-63	-19	-52	-46	-70	-x	-40	-42	-88	-x	-4	-43	-76	-47	-65	-53	-66	-72	449	-94	35
TYR	-10	-27	-35	-26	-60	-90	-34	11	-71	-76	1	-31	-33	33	-51	-20	-30	-35	-81	408	30
INS	20	56	26	29	11	-14	35	31	-1	-26	51	30	8	29	16	26	18	13	5	-9	365

REFERENCES

[1] J. Quinlan, *C4.5: Programs for Machine Learning*. Morgan Kaufmann: Palo Alto, CA, 1993.

[2] J. Pearl, "From Bayesian networks to causal networks," in *Probabilistic Reasoning and Bayesian Belief Networks* (A. Gammerman, ed.), (Henley-on-Thames), pp. 1–31, Alfred Waller, 1995.

[3] J. Pearl, "Probabilistic reasoning using graphs," in *Proceedings of the International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems* (R. R. Y. B. Bouchon, ed.), vol. 286 of *LNCS*, (Paris, France), pp. 200–204, Springer, June–July 1986.

[4] S. Russell and B. Grosof, "A declarative approach to bias in concept learning," in *Proceedings of the Sixth National Conference on Artificial Intelligence*, pp. 505–510, 1987.

[5] T. Davies and S. Russell, "A logical approach to reasoning by analogy," in *Proceedings of the Tenth International Joint Conference on Artificial Intelligence*, pp. 264–270, 1987.

[6] M. Pazzani, "Integrated learning with incorrect and incomplete theories," in *Proceedings of the Fifth International Conference on Machine Learning*, pp. 291–297, 1988.

[7] A. Ginsberg, S. Weiss, and P. Politakis, "Automatic knowledge base refinement for classification systems," *Artificial Intelligence*, vol. 35, pp. 197–226, 1988.

[8] A. Ginsberg, "Knowledge base refinement and theory revision," in *Proceedings of the Sixth International Conference on Machine Learning*, pp. 260–265, 1989.

[9] G. Towell, J. Shavlik, and M. Noordewier, "Refinement of approximate domain theories by knowledge-based neural networks," in *Proceedings of the Eighth National Conference on Artificial Intelligence*, pp. 861–866, 1990.

[10] R. S. Michalski, "Pattern recognition as rule-guided inductive inference," *PAMI*, vol. 2, no. 4, pp. 349–361, 1980.

[11] N. Nilsson, *Principles of Artificial Intelligence*. Palo Alto, CA: Morgan Kaufmann, 1986.

[12] D. Ackley, "An empirical study of bit vector function optimization," in *Genetic Algorithms and Simulated Annealing* (L. Davis, ed.), Los Altos, CA: UC Press, 1987.

[13] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, pp. 671–680, 1983.

[14] F. Romeo and A. Sangiovanni-Vincentelli, "A theoretical framework for simulated annealing," *Algorithmica*, vol. 6, pp. 302–345, 1991.

[15] N. Littlestone, "Learning quickly when irrelevant attributes abound," *Machine Learning*, vol. 2, pp. 285–318, 1988.

[16] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry.* MIT Press: Cambridge, MA, 1969.

[17] D. E. Rumelhart, G. E. Hinton, and J. L. McClelland, "A general framework for parallel distributed processing," in *Parallel Distributed Processing: Explorations in the Micro-Structure of Cognition* (D. E. Rumelhart and J. L. McClelland, eds.), pp. 46–73, MIT Press, 1986.

[18] M. Genesereth and N. Nilsson, *Logical Foundations of Artificial Intelligence.* Morgan Kaufmann: Palo Alto, CA, 1987.

[19] J. R. Quinlan, "Discovering rules from large collections of examples: A case study," in *Expert Systems in the Micro Electronic Age* (D. Michie, ed.), Edinburgh, Scotland: Edinburgh University Press, 1979.

[20] L. Rendell, "A general framework for induction and a study of selective induction," *Machine Learning*, vol. 1, no. 2, pp. 177–226, 1986.

[21] N. Lavrac and S. Dzeroski, *Inductive Logic Programming: Techniques and applications.* Ellis Horwood, Chichester, 1993.

[22] J. W. Lloyd, *Foundations of Logic Programming.* SPRINGER, 1984.

[23] R. Quinlan, "Determinate literals in inductive logic programming," in *Proceedings of the Eighth International Workshop on Machine Learning*, pp. 442–446, Morgan Kaufmann: San Mateo, CA, 1991.

[24] G. E. Hinton and T. J. Sejnowski, "Learning and relearning in boltzmann machines," in *Parallel Distributed Processing, Vol. I*, pp. 282–317, MIT Press, 1986.

[25] J. Zurada, *Introduction to artificial neural systems.* St. Paul, MN: West Publishing, 1992.

[26] M. Pazzani, C. Brunk, and G. Silverstein, "A knowledge-intensive approach to learning relational concepts," in *Proceedings of the Eighth International Workshop on Machine Learning*, pp. 432–436, Morgan Kaufmann: San Mateo, CA, 1991.

[27] D. Ourston and R. Mooney, "Changing the rules: A comprehensive approach to theory refinement," in *Proceedings of the National Conference of Artificial Intelligence*, (Boston, MA), pp. 815–820, 1990.

[28] D. Oblinger and G. DeJong, "Dynamic bias induction," in *American Association for Artificial Intelligence Fall Symposium Series on Relevance (AAAI-94)*, (New Orleans, LA), pp. 164–167, 1994.

[29] D. Oblinger and G. DeJong, "An alternative to deduction," in *The Thirteenth Annual Conference of the Cognitive Science Society*, (Cambridge, Mass.), 1991.

[30] R. S. Michalski, I. Mozetic, J. Hong, and N. Lavrac, "The aq15 inductive learning system: An overview and experiments," *Proceedings of the International Meeting on Advances in Learning*, 1986.

[31] D. B. Induction, "Daniel oblinger and gerald dejong," Tech. Rep. UIUC-AI-BI-9403, Beckman Institute, University of Illinois, 1994.

[32] C. Green, "Application of theorem proving to problem solving," in *Proceedings of the First International Joint Conference on Artificial Intelligence*, pp. 219–239, 1969.

[33] J. Devore, *Probability and statistics for engineering and the sciences*. Brooks/Cole, 1982.

[34] S. Rajamoney, "A computational approach to theory revision," in *Computational Models of Scientific Discovery and Theory Formation* (J. Shrager and P. Langley, eds.), Morgan Kaufmann: San Mateo, CA, 1990.

[35] D. Wilkins, "Knowledge base refinement as improving an incomplete and incorrect domain theory," in *Machine Learning, an Artificial Intelligence Approach, Volume III* (Y. Kodratoff and R. Michalski, eds.), pp. 493–513, Morgan Kaufmann: San Mateo, CA, 1990.

[36] R. Quinlan, "Learning logical definitions from relations," *Machine Learning*, vol. 5, pp. 239–266, 1990.

[37] S. Muggleton and W. Buntine, "Machine invention of first-order predicates by inverting resolution," in *Proceedings of the Fifth International Conference on Machine Learning*, pp. 218–229, 1988.

[38] D. Montomery and E. Peck, *Introduction to linear regression analysis*. New York: Wiley, 1992.

[39] G. Clarke and D. Cooke, *A basic course in statistics*. E. Arnold, 1992.

[40] W. Cohen, "The generality of overgenerality," in *Proceedings of the Eighth International Conference on Machine Learning*, pp. 490–494, June 1991.

[41] R. Mooney and D. Ourston, "A multistrategy approach to theory refinement," in *Machine Learning: A Multistrategy Approach* (R. Michalski and G. Tecuci, eds.), vol. 4, Morgan Kaufmann: San Fransisco, 1994.

[42] W. Cohen, "Rapid prototyping of ILP systems using explicit bias," in *Proceedings of the International Joint Conference of Artificial Intelligence workshop on ILP*, 1993.

[43] S. Muggleton, ed., *Inductive Logic Programming*. No. 38 in APIC, London: Academic Press, 1992.

[44] S. Muggleton, "Inverse entailment and progol," *New Generation Computing Journal*, vol. 13, pp. 245–286, 1995.

[45] M. desJardins, "Probabilistic evaluation of bias for learning systems," in *Proceedings of the Eighth International Conference on Machine Learning*, pp. 495–499, June 1991.

[46] D. H. Fisher, "Conceptual clustering, learning from examples, and inference," in *Machine Learning: Proceedings of the Fourth International Workshop*, pp. 38–49, Morgan Kaufmann, 1987.

[47] P. Cheeseman, J. Kelly, M. Self, J. Stutz, W. Taylor, and D. Freeman, "Autoclass: A baysian classification system," in *Proceedings of the Fifth International Workshop on Machine Learning*, pp. 54–64, 1988.

[48] J. deKleer, "Qualitative and quantitative reasoning in classical mechanics," Tech. Rep. TR 352, MITAI, 1975.

[49] K. D. Forbus, "Qualitative process theory," *Artificial Intelligence*, vol. 24, pp. 85–168, 1984.

[50] P. Clark, "Representing arguments as background knowledge for constraining generalisation," in *European Working Session on Learning 1988*, pp. 37–44, 1988.

[51] R. Reiter, "A logic for default reasoning," *Artificial Intelligence*, vol. 13, no. 1-2, pp. 81–113, 1980.

[52] D. L. Poole, R. G. Goebel, and R. Aleliunas, "Theorist: A logical reasoning system for defaults and diagnosis," in *he Knowledge Frontier: Essays in the Representations of Knowledge* (N. Cercone and G. McCalla, eds.), pp. 331–352, New York: Springer Varlag, 1987.

[53] J. McCarthy, "Circumscription - a form of non-monotonic reasoning," *Artificial Intelligence*, vol. 13, no. 1, pp. 27–39, 1980.

[54] J. McCarthy, "Applications of circumscription to formalizing common-sense knowledge," *Artificial Intelligence*, vol. 28, pp. 89–116, 1986.

[55] C. Anfinsen, "Principles that govern the folding of protein chains," *Science*, vol. 181, pp. 230–233, 1973.

[56] D. Neidhart, G. Kenyon, J. Gerlt, and G. Petsko, "Mandelate racemase and muconate lactonizing enzyme are mechanically distinct and structurally homologous," *Nature*, pp. 347–692, 1990.

[57] S. Needleman and C. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of Molecular Biology*, vol. 48, pp. 443–453, 1970.

[58] M. Dayhoff, R. Eck, and C. Park, "A model of evolutionary change in proteins," in *Atlas of Protein Sequence and Structure* (M. Dayhoff, ed.), vol. 5, National Biomedical Research Foundation: Silver Springs, MD, 1972.

[59] J. Bowie, R. Luthy, and D. Eisenberg, "A method to identify protein sequences that fold into a known three-dimensional structure," *Science*, vol. 253, pp. 164–170, 1991.

[60] J. Bowie and D. Eisenberg, "Inverted protein structure prediction," *Current Opinions in Structural Biology*, vol. 3, pp. 437–444, 1993.

[61] T. R. Ioerger, "The context-dependence of amino acid properties," in *Proceedings of the 5th International Conference on Intelligent Systems for Molecular Biology* (T. Gaasterland, P. Karp, K. Karplus, C. Ouzounis, C. Sander, and A. Valencia, eds.), (Menlo Park), pp. 157–166, AAAI Press, June21–26 1997.

[62] J. Overington, D. Donnelly, J. Johnson, A. Sali, and T. Blundell, "Environment-specific amino acid substitution tables: Tertiary templates and prediction of protein folds," *Protein Science*, vol. 1, pp. 216–226, 1992.

[63] M. Brown, R. Hughey, A. Krogh, I. Mian, K. Sjolander, and D. Haussler, "Using dirichlet mixture priors to derive hidden markov models for protein families," in *Proceedings of the First International Conference on Intelligent Systems for Molecular Biology*, pp. 47–55, 1993.

[64] J. Ponder and F. Richards, "Tertiary templates for proteins: Use of packing criteria in the enumeration of allowed sequences for different structural classes," *Journal of Molecular Biology*, vol. 193, pp. 775–791, 1987.

[65] T. Ioerger, L. Rendell, and S. Subramaniam, "Searching for representations to improve protein sequence fold-class prediction," *Machine Learning*, vol. 21, pp. 151–175, 1995.

[66] F. Richards, "Folded and unfolded proteins: An introduction," *Protein Folding*, pp. 1–58, 1992.

[67] J. Richardson, "The anatomy and taxonomy of protein structure," *Advances in Protein Chemistry*, vol. 34, pp. 167–336, 1981.

[68] S. Subramaniam, D. Tcheng, H. Ragavan, and L. Rendell, "Knowledge-based methods for protein structure refinement and prediction," in *Proceedings of the Fourth international conference of software engineering and knowledge engineering*, (Capri, Italy), June 1992.

[69] R. Luethy, J. Bowie, and D. Eisenberg, "Assessment of protein models with three-dimensional profiles," *Nature*, vol. 356, pp. 83–85, 1992.

[70] W. Kabsch and C. Sander, "Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometric features," *Biopolymers*, vol. 22, pp. 2577–2637, 1983.

[71] E. Myers and W. Miller, "Optimal alignments in linear space," *CABIOS*, vol. 4, pp. 11–17, 1988.

[72] D. Haussler, "Quantifying inductive bias: Ai learning algorithms and valiant's learning framework," *Artificial Intelligence*, vol. 36, pp. 177–221, 1988.

[73] A. P. Pascarella S., "A data bank merging related protein structures and sequences," *Protein Engineering*, vol. 5, no. 2, pp. 121–137, 1992.

VITA

Dan Oblinger was born in Cincinnati Ohio in 1965. He studied both Mathematics and Computer Science at Northern Kentucky University. He was awarded student of the year in both fields and graduated summa cum laude with a Bachelors of Science in 1987. He attended the Ohio State University where he completed a Masters of Science in Computer Science in 1989. He studied Machine Learning under the direction of Dr. Gerald DeJong, and Computational Biology under the direction of Dr. Shankar Subramaniam both at the Beckman Institute in Urbana-Champaign Illinois. These studies culminated in a Doctorate of Philosophy in Computer Science from the University of Illinois in 1998. He is now a Member of the Research Staff at the IBM T.J. Watson Research Center in New York.

His professional interests include Knowledge-Based Machine Induction, Data Mining, Artificial Intelligence, and Bioinformatics.